

c) *Warum werden im Schichtenmodell Schnittstellen benötigt?*

Das Schichtenmodell beschreibt Schichten, die Schnittstellen exportieren (also genau spezifizierte Dienstleistungen den darüber liegenden Schichten zur Benutzung anbieten) und/oder Schnittstellen importieren (also von darunterliegenden Schichten angebotene Schnittstellen benutzen). Die definierten Schnittstellen gewähren dabei die reibungslose Zusammenarbeit der verschiedenen Schichten.

d) *Das Mach-Betriebssystem besteht neben dem Kernel aus verschiedenen Modulen. Doch unterstützen die meisten modernen unixoiden Systeme das Laden von Modulen zur Laufzeit.*

Eine Übersicht über alle aktuell geladenen Module lässt sich in einem Terminalfenster wie folgt ausgeben:

- Unter Linux: `lsmod`
- Unter FreeBSD: `kldstat`
- Unter Solaris: `modinfo`
- Unter OS X: `kextstat`

Überzeugen Sie sich auf einem beliebigen der aufgeführten Systeme, dass tatsächlich Module geladen sind.

- *Was ist bei diesen Betriebssystemen der Unterschied zu besagten modularen Komponenten im Mach-Betriebssystem?*

Die Module sind häufig nur Treiber oder mehr oder weniger abgegrenzte Subsysteme (also keine deutliche Kapselung), die bei Bedarf in den Kern geladen werden können, um diesen zu erweitern. Im Unterschied zu MACH laufen die Module im *kernel mode*, nicht im *user mode*: der Kernel ist somit trotzdem ein monolithischer Kernel (außer OS X: nutzt XNU als Hybridkernel. Die *kernel extensions* (kexts) laufen allerdings trotzdem im *kernel mode*).

- *Welche Vor- und Nachteile ergeben sich daraus?*

Vorteile:

- Codeteile des Kerns während der Laufzeit sind austauschbar (z. B. für Sicherheitspatches ohne Neustart oder Treiberaktualisierungen)
- Der Kernel kann klein bleiben und den Treiber nur bei Bedarf nachladen, also ist weniger Arbeitsspeicher nötig
- Ein relativ generischer Kernel kann sich durch passende Treiber leicht der tatsächlichen Hardware anpassen
- Ein Ansprechen von Modulfunktionen benötigt weniger Kontextwechsel als beim MACH-Modell und ist daher schneller: Der Weg „Nutzerprogramm zu Kernel“ ist im Gegensatz zu „Nutzerprogramm zu Kernel (IPC) zu Modul im user mode“ schneller

Nachteile:

- Weil Module im *kernel mode* laufen können Programmierfehler leicht zu Systemabstürzen führen, weil die Speicherzugriffe unkontrolliert möglich sind. Bei MACH würde ein einzelnes abgestürztes Modul nicht das ganze Betriebssystem abstürzen lassen.

- e) *Informieren Sie sich über GNU Hurd. Welches grundlegende Konzept wird bei Hurd verfolgt und wodurch zeichnet es sich aus?*

Bei Hurd wird das Microkernel-Modell ähnlich dem von MACH verfolgt. Genauer gesagt ist Hurd ein Betriebssystem auf Basis des MACH -Microkernels. Das MACH -Modell zeichnet sich dadurch aus, dass nur die allernotwendigsten Funktionen im Betriebssystemkern (der im *kernel mode* läuft) vorhanden sind: „die Verwaltung der Dienste, die selbst aber aus dem Kern ausgelagert werden, und die Kommunikation zwischen den Diensten“ (siehe [Abschn. 1.3](#)).

Ein Großteil des Betriebssystems läuft also in Form von klar abgegrenzten Diensten im *user mode*. Dies bedeutet eine Kapselung der Systemdienste und folglich deren leichte Austauschbarkeit.

9.2 Lösungen zu Kap. 2

Aufgabe 2.1-1 Betriebsarten

Nennen Sie einige Betriebsarten eines Betriebssystems (inklusive der drei wichtigsten).

- *Stapelbetrieb (Batch)*: Der vollständige Auftrag wird dem Rechner übergeben und von ihm ohne Unterbrechung abgearbeitet. In dieser strengen Form ist dies heute nicht mehr üblich, jedoch gibt es immer noch Batch-Läufe, die parallel zum eigentlichen Dialogbetrieb ablaufen können.
- *Mehrprogrammbetrieb, Dialogbetrieb*: Beim Mehrprogrammbetrieb laufen mehrere Programme nahezu gleichzeitig. Dies wird durch ein geeignetes Verwaltungssystem erreicht.
- *Mehrbenutzerbetrieb*: Hier wird nicht nur ein Benutzer, sondern mehrere mit ihren Prozessgruppen gleichzeitig vom Betriebssystem bedient.
- Beim *Echtzeitbetrieb* unterliegt die Programmausführung strengen zeitlichen Restriktionen.
- Weiterhin unterscheidet man häufig zwischen *user mode* und *kernel mode*. Der *kernel mode* bietet nicht die Schutzmechanismen des *user mode* wie z. B. Speicherschutz etc., weil das Betriebssystem in diesem Modus arbeitet und grundsätzlich auf alle Speicherbereiche aller Benutzer zugreifen darf/können muss, sonst wäre eine sinnvolle Speicherverwaltung nicht möglich.

Aufgabe 2.1-2 Prozesse

- a) *Erläutern Sie nochmals die wesentlichen Unterschiede zwischen Programm, Prozess und thread.*
- *Programm*: Ein Programm besteht aus dem Programmcode und Daten. Es kann sich im Speicher oder auf einem externen Datenträger befinden. Ein Programm kann sich aus mehreren Prozessen zusammensetzen oder mehrere Prozesse starten.

- *Prozess*: Zu einem Prozess gehören die Zustands- und Steuerinformationen eines Programms wie etwa Informationen über geöffnete Dateien, Registerinhalte, Stack, Speicherzuordnungstabellen usw. Er stellt die „Hülle“ für ein Programm dar.
 - *Thread*: Ein *thread* ist ein sogenannter „Leichtgewichtsprozess“, da er nur wenige, eigene Zustandsinformationen besitzt. Das Erzeugen und Entfernen eines *threads* geht daher wesentlich schneller als bei einem Prozess. Ein *thread* benutzt die meisten Zustandsinformationen (Dateien, virtueller Adressraum usw.) vom Prozess, zusammen mit den anderen *threads*, und kann deshalb nur im Kontext eines Prozesses laufen. So kann er z. B. Speicher mit anderen *threads* teilen.
- b) Welche schwerwiegenden Konsequenzen kann es haben, wenn ein Betriebssystem keine Prozesse, sondern lediglich Threads unterstützen würde?

Sehr großes Problem z. B. Speicherzugriffe untereinander: alle laufenden Threads teilen sich denselben Ausführungskontext, also insbesondere denselben Speicherbereich. Somit kann jeder Thread jedem anderen Thread völlig unkontrolliert beliebige Daten überschreiben, Programmabstürze sind vorprogrammiert (Achtung: Wortwitz ☺).

- c) Wie sehen im UNIX-System des Bereichsrechners ein Prozesskontrollblock (PCB) und die „user structure“ aus? Inspizieren Sie dazu die Include-Dateien `/usr/include/sys/proc.h` und `/usr/include/sys/user.h` und charakterisieren Sie grob die Einträge.

In `/usr/include/sys/proc.h` ist die Struktur des PCB enthalten, die dauerhaft im Speicher gehalten wird. Die Einträge enthalten Informationen zu folgenden Bereichen:

- Strukturheader (Zeiger zu anderen PCBs)
 - Prozesspriorität
 - Zeit (verbrauchte CPU-Zeit, Gesamtzeit, ...)
 - Signale (Masken und ignorierte, abgefangene bzw. anfallende Signale)
 - Multiprozessorunterstützung (Affinität, Status, letzter Prozessor, ...)
 - Identitäten (UserId, GroupId, SessionId, ProzessId, ElternId)
 - Zeiger zu Verwandten (Zeiger zu Elternprozess, jüngstem Kind, Geschwistern usw.)
 - Speicherbelegung (Größe, Lage von Stack+Code+Daten)
 - Daten von *Shared Memory* (*page table offsets*, Prozessliste, erste/letzte Adresse, Zugriffsrechte, gesperrt ja/nein)
 - Daten des ausgelagerten Prozesses (Plattenadresse von stack, code, data segment).
- Im Gegensatz dazu sind in `/usr/include/sys/user.h` alle Daten des Prozesskontextes enthalten, die nicht im Hauptspeicher sind, wenn der Prozess ausgelagert ist. Neben einer Struktur, die Identitäten enthält (effektiver/reeller UserId, GroupId usw.) ist dies insbesondere die *user*-Struktur. Diese besteht aus folgenden Bereichen:
- Daten des letzten Betriebssystemaufrufs (Parameter, Systemaufrufsnummer, Rücksprungadresse, Fehlercode)
 - Speichergröße (Code, Daten, Stack)
 - Signale (Masken, Signalstack, blockierte Signale)

- Dateimanagement (Zeiger zur Liste offener Dateien, Zahl offener Dateien, Wurzelverzeichnis, jetziges Verzeichnis, Maske für neue Dateien)
- Zeit und Statistik
- Quotenkontrolle
- *shared memory*-Daten (Größe, Sperrinformation usw.)
- *Audit*-Information
- *User*-Stack.

Aufgabe 2.1-3 Prozesszustände

a) Welche Prozesszustände durchläuft ein Prozess?

Eine grobe Einteilung lässt sich durch *rechenbereit*, *rechnend* und *blockiert* geben. Viele Betriebssysteme kennen weitere Zwischenzustände wie etwa

- nicht existent
- untätig (*idle*)
- angehalten (*stop*)
- im Löschungsvorgang (*zombie*).

b) Was sind typische Wartebedingungen und Ereignisse?

Typische Wartebedingungen sind:

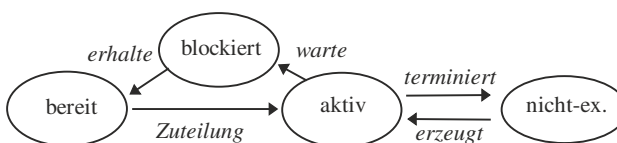
- Der Prozess wartet in der *bereit*-Liste darauf, Prozessorzeit zugewiesen zu bekommen.
- Ein Editor wartet typischerweise auf eine Eingabe vom Benutzer, d. h. auf einen Tastendruck oder die Auswahl eines Menüpunkts. Dies entspricht einem Warten des Editorprozesses in einer I/O-Warteschlange.
- Das Dienstprogramm „Clock“ wartet auf
 - Timer-Events, um z. B. den Sekundenzeiger weiter zu bewegen
 - eine Nachricht von einem anderen Prozess
 - Daten vom Benutzer (Interaktion).

In diesem Fall wartet der Prozess auf mehrere Ereignisse in einer allgemeinen Ereignisschlange; die Art des Ereignisses wird beim Aufwecken dem Prozess bekanntgegeben.

Typische Ereignisse sind z. B. gesendete Signale (Kill-Signal in UNIX), Freigabe von I/O-Geräten, Zuteilung von Rechenzeit.

c) Ändern Sie die Zustandsübergänge im Zustandsdiagramm [Abb. 2.2](#) durch ein abweichendes, aber ebenso sinnvolles Schema. Begründen Sie Ihre Änderungen.

Ein anderes Übergangsschema könnte sein



Hierbei erhält jeder Prozess, sobald er erzeugt wurde, auch die Prozessorkontrolle.

Ein weiteres mögliches Schema besteht darin, dass nur bereite Prozesse terminiert werden können und erzeugte Prozesse sofort die Kontrolle erhalten. Dies dreht die Erzeugungs-/Terminierungsverbindungen in Abb. 2.2 gerade um.

Allgemein gilt, dass für ein gutes Schema jeder Prozess nur für sich selbst verantwortlich sein sollte und nicht andere Prozesse beeinflussen darf. Dies entspricht einer Trennung der Benutzer und ihrer Prozesse. Ausnahme: Elternprozesse dürfen ihre Kinder kontrollieren (sie stammen meist vom selben Benutzer).

Aufgabe 2.1-4 UNIX-Prozesse

Wie ließe sich in UNIX mit der Sprache C oder MODULA-2 ein Systemaufruf „ExecuteProgram(prg)“ als Prozedur mit Hilfe der Systemaufrufe `fork()` und `waitpid()` realisieren?

Eine mögliche Lösung würde, analog zur *shell*, folgendermaßen aussehen:

```
INT ExecuteProgram (prg);
*CHAR prg;
{ INT pid, status;
  pid := fork();                      (* erzeuge Kopie dieses Prozesses *)
  IF (pid=0)
    THEN execve(prg, "", 0)           (* Kind: überlade mit Programm *)
    ELSE waitpid(pid, status, 0)      (* Eltern: warte auf Ausf.ende *)
  END;
  RETURN status;
}
```

Aufgabe 2.1-5 Android activities

Recherchieren Sie,

- a) welche Zustände eine activity unter Android annehmen kann, und wofür diese verwendet werden.
- b) Wie werden dabei die aktuell nicht sichtbaren activities verwaltet?
 - a) Zustände sind
 - *resumed/running*: Aktivität ist im Vordergrund und hat den Fokus
 - *paused*: Aktivität ist nicht im Vordergrund, ist aber immer noch sichtbar (und wird nur verdeckt; die Aktivität ist immer noch an den Fenstermanager angehängt), kann nur bei extremer Speicherknappheit beendet werden.
 - *stopped*: Aktivität ist im Hintergrund, ist nicht mehr an den Fenstermanager angehängt, aber immer noch ‚am Leben‘, kann jederzeit wegen Speicherknappheit beendet werden.

Quelle: <https://developer.android.com/guide/components/activities.html>

b) Android verwaltet sog. *Tasks*, die in der Taskübersicht angezeigt werden. Jeder Task zeigt dabei *Activities* an, wenn eine *Activity* eine neue *Activity* aufruft, wird diese angezeigt und verdeckt die bisherige *Activity* (und pausiert bzw. stoppt diese somit).

Besonderheit: „Even though the activities may be from different applications, Android maintains this seamless user experience by keeping both activities in the same task.“

Interessant ist nun die Verwaltung der nicht-laufenden *Activities*: Jeder Task hat einen *Back Stack*, auf dem die *Activities* des Tasks gespeichert sind. Drückt der Nutzer die Zurück-Taste, so wird die aktive *Activity* zerstört und durch die oberste *Activity* auf dem *Back Stack* ersetzt. Wird eine neue *Activity* gestartet, so wird diese zuoberst auf den Stack gepusht.

Quelle: <https://developer.android.com/guide/components/tasks-and-back-stack.html>

Aufgabe 2.1-6 Threads

Realisieren Sie zwei Prozeduren als Leichtgewichtsprozesse, die sich wechselseitig die Kontrolle übergeben. Der eine Prozess gebe „Ha“ und der andere „tschi!“ aus.

In MODULA-2 ist die Verwendung von Prozeduren als LWP mit Hilfe der Konstrukte NEWPROCESS und TRANSFER leicht möglich.

Die Realisierung folgt in Grundzügen Dal Cin (1988):

```
MODULE HaTschi;

FROM SYSTEM      IMPORT ADDRESS, ADR, BYTE, SIZE,
                  NEWPROCESS, TRANSFER;

FROM Storage     IMPORT ALLOCATE;
FROM Terminal    IMPORT Write, WriteLn, WriteString;

TYPE COROUTINE = ADDRESS;

VAR Ha, Tschi, main : COROUTINE;
    stack1, stack2: ARRAY[1..1024] OF BYTE;  (*Arbeitsbereiche*)

PROCEDURE writeHa;
BEGIN
  LOOP
    WriteString („Ha“); TRANSFER (Ha, Tschi)
  END;
END writeHa;

PROCEDURE writeTschi;
BEGIN
  LOOP
    WriteString („tschi!“); WriteLn; TRANSFER (Tschi, Ha)
  END;
END writeTschi;
```

```

BEGIN (* --- main --- *)
    NEWPROCESS(writeHa, ADR(stack1), SIZE(stack1), Ha);
    NEWPROCESS(writeTschi, ADR(stack2), SIZE(stack2), Tschi);
    TRANSFER (main, Ha);
END HaTschi.

```

Natürlich kann man dieses Beispiel noch anreichern, beispielsweise dadurch, dass die Zahl der „Ha“-Ausgaben zufallsmäßig variiert wird, bevor die Kontrolle nach „tschi!“ übergeben wird.

Aufgabe 2.1-7 Kernel Threads

Auf einem Linux-System mit verschlüsselter Festplatte tauchen bei Eingabe von `ps -ef` in einem Terminal unter anderem die Einträge `[kcryptd]` und `[dmccrypt_write]` auf. Dabei handelt es sich um Kernel-Threads.

Recherchieren Sie, was Kernel-Threads auszeichnet und erläutern Sie in eigenen Worten, worin der Unterschied zwischen Kernel-Threads und den in der Vorlesung vorgestellten Threadarten liegt.

Kernel-Threads laufen im *kernel mode* ab, haben also uneingeschränkten Zugriff auf Alles. *Lightweight* und *heavyweight* Threads laufen dagegen im *user mode* ab. *Lightweight* Threads werden vom Nutzerprogramm selbst verwaltet und nur innerhalb ihres Prozesses zugeteilt, da sie dem Betriebssystem nicht bekannt sind; *Heavyweight* und *Kernel-Threads* werden dagegen vom Betriebssystem verwaltet und eingesetzt, da sie dem bekannt sind.

Zumindest unter Linux können Kernel-Threads zudem nicht vom Dispatcher unterbrochen werden, sondern laufen, bis sie sich selbst schlafen legen (Quade und Kunst 2016, Kapitel 6.4). *Heavyweight* Threads kann der Dispatcher dagegen beliebig unterbrechen, sofern präemptives Scheduling verwendet wird.

Aufgabe 2.1-8 Prozesse vs. Threads

Ihr Arbeitgeber hat Sie damit beauftragt, für die firmeninterne Verwendung eine integrierte Entwicklungsumgebung (IDE) zu programmieren. Für welche der folgenden Anforderungen ist es sinnvoller, Prozesse zu verwenden, zur Umsetzung welcher sollten besser Threads verwendet werden? Begründen Sie Ihre Antwort.

- Einer Ihrer Kollegen ist ein begnadeter Programmierer, macht jedoch sehr viele Syntaxfehler. Um diese so früh wie möglich zu erkennen, fordert er, dass die neue IDE ständig den aktuellen Stand des Codes einer Syntaxanalyse unterzieht.
- Zwei Ihrer Kollegen arbeiten an mehr als einem Projekt gleichzeitig und möchten daher die IDE mehrfach öffnen sowie die Fenster auf verschiedenen Bildschirmen darstellen können, um einen besseren Überblick zu haben. Allerdings trauen die beiden Ihren Programmierfähigkeiten nicht, weshalb sie betonen, dass bei einem

Absturz durch einen Programmierfehler in der IDE nicht die Änderungen aller Projekte verloren sein dürfen, sondern höchstens eines einzigen Projekts. Es soll also jeder Programmierfehler höchstens eine einzige IDE gleichzeitig zum Absturz bringen können.

- Im ersten Fall werden Threads verwendet, da ein Zugriff auf dieselben Daten wie die IDE benötigt wird.
- Im zweiten Fall sind Prozesse sinnvoll, da mehrere Instanzen verwendet werden, die keine Daten miteinander teilen müssen. So geht beim Absturz auch höchstens ein Prozess zugrunde. Die Mehrbildschirmgeschichte dient nur der Verschönerung des Szenarios.

Aufgabe 2.2-1 Scheduling

- a) *Was ist der Unterschied zwischen der Ausführungszeit und der Bedienzeit eines Jobs?*

Zu der reinen Prozessorzeit, die ein Job benötigt (die Bedienzeit) kommt noch die Zeit hinzu, die der Job in Warteschlangen warten muss, beispielsweise weil eine Platte noch nicht richtig positioniert war, um einen Datenblock zu lesen, oder weil der Prozessor noch Verwaltungsarbeit erledigen musste. Die Gesamtzeit vom Start bis zum Jobende ist dann die Ausführungszeit.

- b) *Mac OS X ist Apples erstes Betriebssystem, das präemptives Scheduling unterstützt: bis einschließlich Mac OS 9 wurde non-präemptives Scheduling verwendet. Erläutern Sie, worin sich diese beiden Schedulingvarianten unterscheiden.*

Beim non-präemptiven Scheduling läuft ein Prozess, bis er fertig ist oder von sich aus den aktiven Zustand verlässt, indem er auf ein Ereignis wartet und damit die Kontrolle freiwillig an einen anderen Prozess abgibt.

Beim präemptiven Scheduling kann dagegen jeder Prozess vorzeitig vom Scheduler unterbrochen werden, um einem anderen Prozess Rechenzeit zuzuteilen. So wird unter anderem verhindert, dass ein einzelner Prozess allen anderen Prozessen die Rechenzeit vorenthält, was die Fairness erhöht.

- c) *Geben Sie den Unterschied an zwischen einem Algorithmus für ein Foreground/background-Scheduling, das RR für den Vordergrund und einen preemptiven Prioritätsschedul für den Hintergrund benutzt, und einem Algorithmus für Multi-level-feedback-Scheduling.*

Im ersten Fall gibt es zwei Prioritätsstufen: Vordergrund und Hintergrund. Ist ein Job im Vordergrund, so bleibt er dort auch; das Umschalten geschieht mit dem RR-Algorithmus. Ist noch Zeit frei, so wird auch der Job aus dem Hintergrund bedient.

Im Unterschied dazu gibt es im zweiten Fall mehrere Prioritätsstufen, zwischen denen die Jobs wechseln. Ein Job, der initial eine geringe Priorität besitzt, bleibt also im ersten Fall immer bei der geringen Priorität; im zweiten Fall kann er aber automatisch in eine höhere Prioritätsstufe aufrücken.

Aufgabe 2.2-2 Scheduling

Fünf Stapelaufträge treffen in einem Computer fast zur gleichen Zeit ein. Sie besitzen in ihrer Reihenfolge geschätzte Ausführungszeiten von 10, 6, 4, 2 und 8 Minuten und die Prioritäten 3, 5, 2, 1 und 4, wobei 5 die höchste Priorität ist. Geben sie für jeden der folgenden Schedulingalgorithmen die durchschnittliche Verweilzeit an. Vernachlässigen sie dabei die Kosten für einen Prozesswechsel.

- a) *Round Robin (Berücksichtigen Sie die FIFO-Reihenfolge)*
- b) *Priority Scheduling*
- c) *First Come First Serve*
- d) *Shortest Job First*

Nehmen Sie für a) an, dass das System Mehrprogrammbetrieb verwendet und jeder Auftrag einen fairen Anteil (Zeitscheibe in FIFO-Reihenfolge!) an Prozessorzeit erhält. Nehmen Sie an, dass die Zeitscheibenlänge unendlich kurz ist verglichen mit der Joblänge.

Die fünf Prozesse seien mit Job A bis E bezeichnet. Damit ergibt sich die folgende Situation:

Zeit	10	6	4	2	8
Job	A	B	C	D	E
Priorität	3	5	2	1	4

- *Round Robin*

Da wir die Wechselzeiten vernachlässigen können und nur kurze Zeitscheiben vorliegen, laufen die einzelnen Prozesse quasiparallel ab. Allerdings wird die Leistung des Prozessors, die jeder Prozess abbekommt, durch die Anzahl der anderen, „gleichzeitig“ laufenden Prozesse entsprechend verringert. Zuerst ist Job D nach 2 Zeiteinheiten fertig. Da aber parallel 5 Jobs laufen, benötigt er die fünffache Zeit, also $5 \times 2 = 10$ Einheiten. Dann ist Job D tatsächlich fertig.

Nun betrachten wir den nächsten Job, der zu Ende gehen wird: Job C. Ihm fehlen noch 2 Zeiteinheiten Laufzeit nach dem Ende von D. Da aber parallel 4 Jobs laufen, benötigt er in Wirklichkeit 4 mal so viel Zeit, also 8 weitere Zeiteinheiten. Damit geht Job C nach Beginn nach insgesamt $8 + 10 = 18$ Zeiteinheiten zu Ende.

Analog wird die zusätzliche Laufzeit von B nach Jobende C zu $3 \times 2 = 6$ Zeiteinheiten ermittelt, so dass also B nach $18 + 6 = 24$ Einheiten zu Ende geht, usw. Es ergibt sich die Summe 110.

Round Robin

A						30
B						+ 24
C						+ 18
D						+ 10
E						+ 28
	10	8	6	4	2	110

=> mittlere Verweildauer = $110 / 5 = 22$

- *Priority Scheduling*

B		E		A		C	D
6	+	14	+	24	+	28	+ 30 = 102

=> mittlere Verweildauer = $102/5 = 20.4$

- *FCFS*

A		B		C	D	E
10	+	16	+	20	+ 22	+ 30 = 98

=> mittlere Verweildauer = $98/5 = 19.6$

- *Shortest Job First*

D	C	B	E	A
2	+ 6	+ 12	+ 20	+ 30 = 70

=> mittlere Verweildauer = $70/5 = 14$

Aufgabe 2.2-3 Adaptive Parameterschätzung

Beweisen Sie: Wird eine Größe a mit der Gleichung

$$a(t) = a(t-1) - 1/t(a(t-1) - b(t))$$

aktualisiert, so stellt $a(t)$ in jedem Schritt t den arithmetischen Mittelwert der Größe $b(t)$ über alle t dar.

Beweis durch Induktion von t nach $t+1$:

Induktionsanfang ($t=1$): $a(1) = a(0) - 1(a(0) - b(1)) = b(1)$

Induktionsvoraussetzung: $a(t)$ sei arithmetischer Mittelwert über alle $b(t)$

Induktionsdurchführung:

$$\begin{aligned}
 a(t+1) &= a(t) - \frac{1}{t+1}(a(t) - b(t+1)) \\
 &= a(t) - \frac{1}{t+1}a(t) + \frac{1}{t+1}b(t+1) = \frac{1}{t+1}a(t) + \frac{1}{t+1}b(t+1) \\
 &= \frac{\sum_{i=1}^t b(i)}{t+1} + \frac{1}{t+1}b(t+1) = \frac{\sum_{i=1}^{t+1} b(i)}{t+1} \text{ Q.E.D.}
 \end{aligned}$$

Aufgabe 2.2-4 Echtzeitscheduling

Um die Ruine eines Atomkraftwerks gefahrlos untersuchen zu können, möchte ein Forscherteam eine autonome Drohne einsetzen. Diese besteht aus verschiedenen Komponenten, die von einem Prozessor gesteuert werden müssen. Dazu muss dieser die folgenden Aufgaben periodisch bearbeiten:

- Alle 50 ms müssen die Beschleunigungssensoren abgefragt und daraus die Ansteuerungsparameter der Rotoren berechnet werden, was 10 ms dauert. Geschieht dies nicht rechtzeitig, so stürzt die Drohne ab und ist verloren.
- Alle 100 ms müssen zur Wegfindung die Abstandssensoren ausgelesen werden, was 10 ms dauert. Geschieht dies nicht rechtzeitig, so kann die Drohne gegen eine Wand fliegen und abstürzen.
- Alle 60 s muss der aktuelle Akkustand überprüft werden, um gegebenenfalls die Rückkehr zum Ausgangspunkt einzuleiten. Diese Überprüfung dauert insgesamt 3 s, da hier zur verlässlichen Vorhersage mehrere Messungen nötig sind. Geschieht dies nicht rechtzeitig, so muss die Drohne im schlimmsten Fall in einem Gebiet notlanden, das stark verstrahlt ist und somit nicht von Menschen betreten werden kann, weshalb die Drohne somit auch verloren wäre.
- Alle 100 ms muss der Temperatursensor ausgelesen werden, um zu verhindern, dass die Drohne zu weit in Richtung des heißen Reaktorkerns fliegt. Dies dauert 5 ms. Geschieht dies nicht rechtzeitig, so fallen wichtige Komponenten möglicherweise aus, was den Absturz der Drohne zur Folge hat.
- Ebenfalls alle 100 ms muss der Geigerzähler abgefragt werden, um die aktuellen Strahlungswerte zu erhalten. Dies dauert 15 ms. Geschieht dies nicht, so kann die Drohne in sehr stark kontaminierte Gebiete fliegen, in denen die Strahlung Fehlfunktionen im Prozessor auslösen kann, die zum Absturz führen können.
- Alle 10 Sekunden soll die integrierte Kamera ein Foto der Umgebung aufnehmen und speichern, was 750 ms dauert. Obwohl dies für die Flugsicherheit der Drohne nicht entscheidend ist, wird gefordert, dass diese Zeitschranke unbedingt eingehalten werden muss, da geschätzt wurde, dass die Entwicklungskosten der Drohne sehr hoch sein werden und ein Drohnenflug mit zu wenigen Bildern nicht genügend Ergebnisse liefert.
- Alle 100 ms muss aus den vorliegenden Daten die Flugroute neu berechnet werden, was 25 ms dauert. Geschieht dies nicht rechtzeitig, so geht die Drohne höchstwahrscheinlich verloren.
- Alle 50 ms müssen die Ansteuerungsparameter der Rotoren an eine separate Flugkontrolleinheit übertragen werden, was 5 ms dauert. Geschieht dies nicht, so stürzt die Drohne ab.
- Aufgrund der zu erwartenden Strahlung, die Inhalte des Arbeitsspeichers verändern könnte, soll der gesamte Arbeitsspeicher immer dann, wenn Zeit zur Verfügung steht, einer Fehlerkorrektur unterzogen werden. Ein Durchlauf durch den gesamten Arbeitsspeicher dauert dabei 20 s. Wegen der guten Abschirmung der Elektronik wird in der durch die Akkukapazität begrenzten maximalen Drohnenflugzeit jedoch statistisch weniger als ein Fehler pro Flugmission erwartet, weshalb diese Anforderung als nicht drohnenmissionskritisch eingestuft wird.
- Nach Möglichkeit soll in regelmäßigen Abständen eines der gespeicherten Bilder per Funk an die Basisstation übertragen werden. Die Forscher hoffen, somit bereits während des Fluges mit einer ersten Auswertung der Drohnenmission beginnen zu können, sehen diese Anforderung jedoch nicht als kritisch an.

Überprüfen Sie, ob sich diese Anforderungen mit den geplanten Komponenten realisieren lassen oder ein neuer Entwurf mit einem schnelleren Prozessor nötig ist. Falls sich die Anforderungen realisieren lassen, so geben Sie ein Schedulingkonzept dafür an und begründen Sie es.

Nicht-kritisch sind RAM-Scrubbing sowie Bildübertragung, kritisch alle anderen Aufgaben. Die Auslastungen der kritischen Tasks sind in der obigen Reihenfolge:

- 20,0 % (10 ms/50 ms) Beschleunigung
- 10,0 % (10 ms/100 ms) Abstand
- 05,0 % (3 s/60 s) Akku
- 05,0 % (5 ms/100 ms) Temperatur
- 15,0 % (15 ms/100 ms) Geigerzähler
- 07,5 % (750 ms/10.000 ms) Kamera
- 25,0 % (25 ms/100 ms) Flugroute
- 10,0 % (5 ms/50 ms) Ansteuerung

Die Summe ist 97,5 %, was die Anforderungen gerade noch alle erfüllt. Die restlichen 2,5 % können dann für RAM-Scrubbing und Übertragung von Bildern verwendet werden.

Hierzu bietet sich ein *foreground-background*-Scheduling an: Im Hintergrund werden die nicht-kritischen Tasks bearbeitet, die jedoch jederzeit von kritischen Tasks unterbrochen werden können, und im Vordergrund werden die kritischen Tasks bearbeitet.

Als Schedulingalgorithmus im Vordergrund könnte man beispielsweise *Guaranteed Percentage Scheduling* (GPS) einsetzen und folglich die Anzahl der Zeitscheiben pro Task proportional zur CPU-Auslastung des Tasks wählen.

Wichtig dabei ist jedoch, dass präemptives Scheduling verwendet wird: Bei nicht-präemptivem Scheduling würde der Akkustand in einem Stück ausgelesen (3 Sekunden!), was unter anderem zum 60-fachen Auslassen des Beschleunigungssensor-Tasks führen würde.

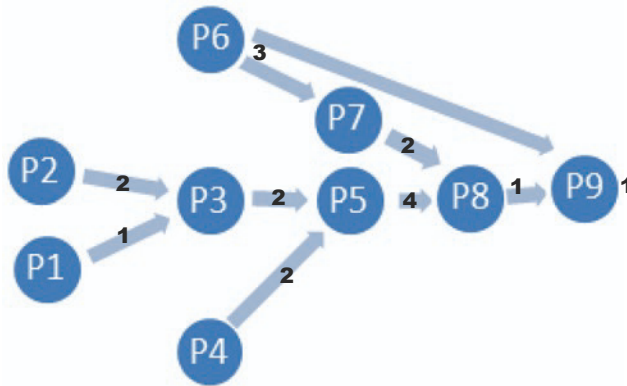
Aufgabe 2.2-5 Multiprozessor-Scheduling

Gegeben seien die Präzedenzen $P1 \gg P3$, $P2 \gg P3$, $P3 \gg P5$, $P4 \gg P5$, $P5 \gg P8$, $P6 \gg P7$, $P6 \gg P9$, $P7 \gg P8$, $P8 \gg P9$ mit den Prozessdauern $t(P1) = 1$, $t(P2) = 2$, $t(P3) = 2$, $t(P4) = 2$, $t(P5) = 4$, $t(P6) = 3$, $t(P7) = 2$, $t(P8) = 1$, $t(P9) = 1$.

- Zeichnen Sie den dazugehörigen Präzedenzgraphen und geben Sie den kritischen Pfad samt seiner Länge an.
- Erstellen Sie für dieses Szenario sowohl einen Earliest Schedul
- als auch einen Latest Schedul und zeichnen Sie die zugehörigen Gantt-Diagramme.

Ihnen stehen unbegrenzt viele Prozessoren zur Verfügung.

Der Präzedenzgraph ist folgendermaßen:



Der Präzedenzgraph ist nicht eindeutig; P4 und P6 können, müssen aber nicht später anfangen. Der kritische Pfad verläuft von $P2 \rightarrow P3 \rightarrow P5 \rightarrow P8 \rightarrow P9$ und hat die Länge 10. Also kann kein Schedul existieren, der eine kürzere Abarbeitungszeit als 10 erreicht.

Gantt-Diagramm des *Earliest-Schedule*:

CPU 3	P6								
CPU 2	P4								
CPU 1	P2		P7						
CPU 0	P1		P3	P5			P8	P9	

Gantt-Diagramm des *Latest-Schedule*:

CPU 2			P4						
CPU 1	P2		P6	P7					
CPU 0	P1		P3	P5			P8	P9	

Man beachte: Es benötigt eine CPU weniger.

Aufgabe 2.2-6 Paralleles Scheduling

- a) Sei ein Programm gegeben mit 40 % sequentiellem, nicht parallelisierbarem Code. Wie groß ist der maximale Speedup?

Bei 40 % sequentiellem Code liegt 60 % paralleler Code vor. Mit der Formel für den Speedup:

$$\text{Speedup} = 1 + \frac{\text{paralleler Anteil}}{\text{sequentieller Anteil}}$$

$$\text{erhält man hier Speedup} = 1 + \frac{60\%}{40\%} = 2,5$$

- b) Angenommen, ein weiteres Betriebsmittel A' ist verfügbar. Wie kann man dann das Gantt-Diagramm in [Abb. 2.12](#) so abändern, dass weniger Zeit verbraucht wird?

Gegeben:

A	A1	A2	A3	A4	A5			A6
B		B1		B4			B2	B3
C			C1		C3	C2		

30 Zeiteinheiten

Dann lässt sich mit einem zusätzlichen A' folgende Bearbeitung erreichen:

A'	A1	A4		A5			A6
A	A2	A3					
B		B1		B4		B2	B3
C			C1	C2	C3		

23 Zeiteinheiten

Aufgabe 2.3-1 Race Conditions

- a) Was wäre, wenn in [Abschn. 2.3.1](#) zuerst Prozess A die Kontrolle erhält und nach Schritt (2) die Umschaltung erfolgt?

Prozess A hängt am Anker, so dass er als nächstes die Kontrolle erhält. Dies hat zum einen zur Folge, dass Prozess B die Kontrolle erhält, obwohl der Prozess in einer ganz anderen Schlange hängt, sobald Prozess A die Kontrolle abgibt und sich wieder aushängt. Zum anderen sind alle Prozesse, die in der abgehängten Schlange warten, auf ewig blockiert und vergessen.

- b) Gibt es noch weitere Möglichkeiten der Fehlererzeugung?

Ja. In [Abschn. 2.3.1](#) wird nur auf die Möglichkeit Bezug genommen, dass beim Aushängen von Prozess B nach Schritt (2) umgeschaltet wird und nach der Aktivität von A, die die Warteschlange geändert hat, mit den alten Daten weitergearbeitet wird. Aber auch wenn für Prozess B nach Schritt (1) umgeschaltet wird, sind die Daten fürs Aushängen veraltet: Anstelle von A, das am Anker hängt, wird dort C eingehängt, so dass nun A blockiert außerhalb der Schlange existiert und nie wieder aktiviert wird.

Auch für das Einhängen führt eine Umschaltung nach Schritt (1) zu einem Fehler; allerdings zum selben wie die obige Umschaltung nach Schritt (2).

Aufgabe 2.3-2 Gegenseitiger Ausschluss: *mutual exclusion*

Formulieren Sie die Prozeduren *betreteAbschnitt* (Prozess: INTEGER) und *verlasseAbschnitt* (Prozess: INTEGER), die die Lösung von Peterson für das *mutual exclusion*-Problem enthalten. Definieren Sie dazu zwei globale, gemeinsame Variablen *Interesse[1..2]* und *dran*. Könnte eine Verallgemeinerung auf *n* Prozesse existieren und wenn ja, wie?

Für zwei Prozesse:

```
VAR
    dran      : INTEGER;
    Interesse: ARRAY[1..2] OF BOOLEAN;

PROCEDURE enterSection(process: INTEGER);
BEGIN
    int anderer;
    anderer    := 3 - process; /* process=1,2*/
    Interesse[process] := TRUE;
    dran       := process;

    WHILE ((dran = process) AND Interesse[anderer])
    DO /* leere Anweisung: busy wait */;
END;

PROCEDURE leaveSection(process: INTEGER);
BEGIN
    Interesse[process] = FALSE;
END;
```

Für *n* Prozesse stammt die klassische Lösung von Lamport (1974). Sein Algorithmus wird „Bäcker-Algorithmus“ genannt, weil er die Zuteilung des Verkaufspersonals zu den zahlreich erscheinenden Kunden in einer Bäckerei simuliert. Dabei ist es üblich, dass jeder Kunde, der hereinkommt, sich zuerst eine Nummer zieht und dann so lange wartet, bis seine Nummer an der Reihe ist. Ziehen zufälligerweise mehrere Kunden die gleiche Nummer (was in der Realität nicht sein kann), so wird als nächster der älteste Kunde (der Prozess mit der kleinsten Prozessnummer) bedient. Die Lösung lautet wie folgt:

```
(* n= number of processes *)
VAR
    choosing: ARRAY[0..n-1] OF BOOLEAN; (*init FALSE*)
    number:   ARRAY[0..n-1] OF INTEGER;  (* init 0 *)

PROCEDURE enterSection(process:INTEGER);
BEGIN
    VAR j:INTEGER;
    choosing[process]:=TRUE;
    number[process] :=max(number)+1;(* select maximal
                                     number currently used *)
    choosing[process]:=FALSE;
```

```

FOR j:=0 TO n-1 DO BEGIN
    WHILE choosing[j]DO (*leerer Befehl:busy wait*);
    WHILE (number[j]<>0)
        AND ((number[j]<number[process]) OR
            ((number[j]=number[process])AND(j<process))
        ) DO (* leerer Befehl: busy wait *);
END;
END;

PROCEDURE leaveSection(process:INTEGER);
BEGIN
    number[process] := 0;
END;

```

Aufgabe 2.3-3 Synchronisation

Betrachten Sie die folgenden fünf Prozesse, die gleichzeitig ausgeführt werden. Jeder Prozess besteht aus vier Abschnitten, wobei Abschnitte der Typen A bis G jeweils korrespondierend kritisch sind:

P1	P2	P3	P4	P5
A	D	B	G	A
B	E	E	H	D
C	A	E	G	E
D	D	F	A	H

- Welche Abschnitte müssen Sie tatsächlich mit Semaphoren synchronisieren?
- Wie viele Semaphore benötigen Sie?

Synchronisiert werden müssen die Abschnitte A, B, D, E.

Nicht synchronisiert werden müssen die Abschnitte C (da nur P1), F (da nur P3), G (da nur P4), H (als nicht kritisch bewertet).

Folglich werden insgesamt 4 Semaphore benötigt.

Aufgabe 2.3-4 Semaphore-Operationen

- Wie würden Sie die Semaphore-Operationen P und V formulieren, wenn s die Anzahl der jeweils wartenden Prozesse enthalten soll?

```

TYPE Semaphor= RECORD
    value: INTEGER;
    list: ProcessList;
END;

VAR s: Semaphor;

PROCEDURE P(VAR s: Semaphor);
BEGIN

```

```

    s.value := s.value + 1;
    if (s.value > 0) THEN
        einhängen(MY_ID, s.list);
        sleep;
    END;
END P;

PROCEDURE V(VAR s: Semaphore);
VAR PID: ProcessId;
BEGIN
    if (s.value > 0) THEN
        PID := aushängen(s.list);
        wakeup(PID);
    END;
    s.value = s.value - 1;
END V;

```

Initialisiert mit -1 enthält `s.value` jeweils die Anzahl der wartenden Prozesse in der Warteschlange `s.list`.

- b) *Wie ändert sich die Lösung des Erzeuger-Verbraucher-Problems dadurch?*
 Nimmt `s.value` einen negativen Wert an, so stellt bereits dessen Betrag (der negierte Wert) die Anzahl der wartenden Prozesse dar. Die Lösung aus [Abschn. 2.3.5](#) kann deshalb mit den neu definierten P und V ohne Änderung übernommen werden.
- c) *Wie muss man `s` verändern, wenn mehrere Betriebsmittel existieren?*
`s.value` muss mit der Anzahl der Betriebsmittel initialisiert werden (bzw. der negativen Anzahl!).

Aufgabe 2.3-5 Prozesssynchronisation

Wie lautet die Synchronisation der Betriebsmittel aus [Abb. 2.13](#) mit Hilfe von Semaphoren?

```

PROCESS A1; BEGIN                                TaskBody; V(b1);                                END A1;
PROCESS A2; BEGIN                                TaskBody; V(a3);                                END A2;
PROCESS B1; BEGIN P(b1);                          TaskBody; V(c1); V(b41);                      END B1;
PROCESS A3; BEGIN P(a3);                          TaskBody; V(b42); V(c21);                      END A3;
PROCESS A4; BEGIN                                TaskBody; V(c22); V(c22);                      END A4;
PROCESS C1; BEGIN P(c1);                          TaskBody; V(a5);                                END C1;
PROCESS B4; BEGIN P(b41); P(b42); TaskBody; V(c3);                                END B4;
PROCESS C2; BEGIN P(c21); P(c22); TaskBody; V(b2);                                END C2;
PROCESS A5; BEGIN P(a5);                          TaskBody; V(b31);                                END A5;
PROCESS C3; BEGIN P(c3);                          TaskBody; V(b32);                                END C3;
PROCESS B2; BEGIN P(b2);                          TaskBody; V(b33);                                END B2;
PROCESS B3; BEGIN P(b31); P(b32); P(b33); TaskBody; V(a6);                      END B3;
PROCESS A6; BEGIN P(a6);                          TaskBody;                                END A6;

```

Aufgabe 2.3-6 Spinlocks

- a) Was ist der Unterschied zwischen den Operationen Blockieren und Spinlock, um auf das Eintreten eines Ereignisses zu warten?

Beim Blockieren wartet der Prozess, bis das gewünschte Ereignis eintritt und er vom Scheduler wieder geweckt wird. Der Prozess benötigt während des Wartens keine Rechenzeit.

Beim *Spinlock* wartet der Prozess *aktiv* durch ständiges Abfragen, ob das Ereignis eingetreten ist. Dabei wird während des Wartens für die Abfrage Rechenzeit benötigt.

- b) Unter welcher Bedingung kann es vertretbar sein, Spinlocks zu verwenden?

Der Einsatz von *Spinlocks* kann aber trotzdem sinnvoll sein, wenn beispielsweise die Hardware das Eintreten des Ereignisses nicht durch einen Interrupt signalisieren kann und/oder wenn es klar ist, dass die Wartezeit auf das Eintreten sehr kurz sein wird. In diesem Fall ist der Overhead durch Blockieren und den damit einhergehenden Threadwechsel größer als das kurzzeitige aktive Abfragen. Ein Beispiel dafür ist, ein Signal an eine Hardware zu senden und auf die ACK-Bestätigung zu warten. Hardwarespezifikationen garantieren das ACK innerhalb von sehr kurzer Zeit. Wären dies beispielsweise nur wenige Nanosekunden, dann müssen nur wenige CPU-Takte „unnötig“ mit Abfragen verbracht werden.

In Betriebssystemkernen werden häufig *Spinlocks* eingesetzt, weil die entsprechenden Methoden nicht unterbrochen werden können dürfen. In diesen Fällen ist auf eine sehr sorgfältige Programmierung zu achten, um Verklemmungen auszuschließen.

Aufgabe 2.3-7 Erzeuger-Verbraucher-Synchronisation

Gegeben sei der folgende unvollständige Programmcode zur Lösung des Erzeuger-Verbraucher-Problems. Ordnen Sie die folgenden 8 fehlenden Befehle den Zeilen (10, 11, 14, 15, 19, 20, 23, 24) im Code zu.

s.mutex.acquire()	gehört in Zeile _____ 11 oder 20
s.mutex.acquire()	gehört in Zeile _____ 20 oder 11
s.mutex.release()	gehört in Zeile _____ 14 oder 23
s.mutex.release()	gehört in Zeile _____ 23 oder 14
s.empty.acquire()	gehört in Zeile _____ 19
s.empty.release()	gehört in Zeile _____ 15
s.full.acquire()	gehört in Zeile _____ 10
s.full.release()	gehört in Zeile _____ 24

Code:

```

1 REGAL = []
2 REGAL_KAPAZITAET = 5
3
4 s_empty   = threading.Semaphore(REGAL_KAPAZITAET)
5 s_full    = threading.Semaphore(0)
6 s_mutex   = threading.Semaphore(1)
7
8 def verbraucher():
9     while True:
10         # Zeile 10
11         # Zeile 11
12         portion = REGAL.pop()
13         print('Lager hat noch "%s", %i Portionen
14         übrig.' % (portion, len(REGAL)))
15         # Zeile 14
16         # Zeile 15
17
18     def erzeuger():
19         while True:
20             # Zeile 19
21             # Zeile 20
22             portion = "McBurger"
23             REGAL.append(portion)
24             # Zeile 23
25             # Zeile 24

```

Aufgabe 2.3-8 Das readers/writers-Problem

Entwerfen Sie einen Pseudocode, um das zweite readers/writers-Problem zu lösen.

Wie beim ersten readers/writers-Problem benötigen wir mehrere Semaphore: eine zur Synchronisation der Leser, andere für die Schreiber. In unserem Fall sind diese wieder binär.

Der Pseudocode fürs Schreiben ist

```

P(WriteSem)
    writecount := writecount + 1          # Erster Schreiber:
    IF writecount = 1 THEN P(TrySem)      # Leser blockieren
V(WriteSem)

P(RWSem)                                # Hier warten alle Schreiber
    Writing_Data()
V(RWSem)

P(WriteSem)
    writecount := writecount - 1          # Letzter Schreiber:
    IF writecount = 0 THEN V(TrySem)      # Leser erlösen
V(WriteSem)

```

und fürs Lesen

```

P(TrySem)
  P(ReadSem)
    readcount := readcount + 1          # Erster Leser:
    IF readcount = 1 THEN P(RWSem)      # Schreiber blockieren
  V(ReadSem)
V(TrySem)
...
Reading_Data()
...
P(ReadSem)
  readcount := readcount - 1          # Letzter Leser:
  IF readcount = 0 THEN V(RWSem)      # Schreiber erlösen
V(ReadSem)

```

Man sieht, dass zwar Leser die Schreiber während des Lesens ausschließen können, aber wenn die Schreiber einmal dran sind, wird solange geschrieben, bis sie fertig sind. Die zusätzliche Absicherung des Lesers durch das zusätzliche Semaphor `TrySem` vermeidet die Situation, dass beim Blockieren der Leser durch einen Schreiber, etwa durch `P(ReadSem)`, eine Verklemmung eintritt: Der letzte Leser könnte dann ausgeschlossen sein, kann `RWSem` nicht mehr zurücksetzen und das Schreiben bliebe blockiert.

Aufgabe 2.3-9 Chinesische (dinierende) Philosophen

N Philosophen sitzen um einen Esstisch und wollen speisen. Auf dem Tisch befindet sich eine große Schale mit Reis und Gemüse. Jeder Philosoph meditiert und isst abwechselnd.

Zum Essen benötigt er jeweils 2 Stäbchen, auf dem Tisch liegen aber nur N Stäbchen, so dass er sich ein Stäbchen mit seinem Nachbarn teilen muss. Also können nicht alle Philosophen zur selben Zeit essen; sie müssen sich koordinieren. Wenn ein Philosoph hungrig wird, versucht er dazu in beliebiger Reihenfolge das Stäbchen links und rechts von ihm aufzunehmen, um mit dem Essen zu beginnen. Gelingt ihm das, so isst er eine Weile, legt die Stäbchen nach seinem Mahl wieder zurück und meditiert. Dabei sind allerdings Situationen denkbar, in der alle gleichzeitig agieren und sich gegenseitig behindern.

Geben Sie eine Vorschrift für jeden Philosophen (ein Programm in einem MODULA oder C-Pseudocode) an, das dieses Problem löst!

Lösung (vgl. Tanenbaum 1995):

```

#define FOREVER      while(1)
#define              N
#define LINKS        (i-1)%N
#define RECHTS        (i+1)%N
#define DENKEND      0
#define HUNGRIG      1
#define ESSEND       2

typedef int Semaphore;
int status[N];
Semaphore mutex = 1;
Semaphore s[N];

void philosoph(int i)
{
    FOREVER { /* Hauptaktionen: Denken und Essen */
        denke();
        nimm_Gabeln_auf(i);
        iß();
        lege_Gabeln_fort(i);
    }
}

void nimm_Gabeln_auf(int i)
{
    P(mutex);
    status[i] = HUNGRIG;
    test(i);          /* versuche beide Gabeln zu bekommen */
    V(mutex);
    P(s[i]);          /* blockiere falls Gabeln nicht frei waren */
}

void lege_Gabeln_fort(int i)
{
    P(mutex);
    status[i] = DENKEND;
    test(LINKS);      /* teste ob linker oder rechter Nachbar */
    test(RECHTS);     /* jetzt essen kann */
    V(mutex);
}

void test(int i)
{
    if (status[i] == HUNGRIG
        && status[LINKS] != ESSEND
        && status[RECHTS] != ESSEND)
    {
        status[i] = ESSEND; V(s[i]);
    }
}

```

Aufgabe 2.3-10 Begriffe

a) Was ist der Unterschied zwischen Blockieren, Verklemmen und Verhungern von Prozessen?

- *Blockieren*: Ein Prozess allokiert sich ein Betriebsmittel BM, um damit zu arbeiten. Eventuell belegt er es länger als nötig – vielleicht vergisst er auch einfach, es nach der Benutzung freizugeben (z. B. Speicher bei Konstruktoren/Destruktoren in C++). Wenn nun ein anderer Prozess ebenfalls dieses BM benötigt, kann er entweder mit einer Fehlermeldung in der Art „Fehler – BM nicht verfügbar“ abbrechen oder warten, bis das BM vom ersten Prozess freigegeben wird – was möglicherweise sehr lange dauern kann. Der zweite Prozess wird im letzten Fall durch den ersten Prozess blockiert.
- *Verklemmen*: Zwei oder mehr Prozesse halten jeder für sich ein BM belegt und versuchen, ein weiteres BM zu belegen, das aber von einem anderen Prozess belegt ist. Dabei entsteht ein Zyklus in den Abhängigkeiten. Da keiner der Prozesse seine Ressource vorzeitig freigibt, warten alle Prozesse nun ewig: Sie sind verklemmt. Eine solche Situation muss von außen gelöst werden.
- *Verhungern*: Ein Prozess erhält, obwohl er rechenbereit ist, keine Rechenzeit zugeteilt, beispielsweise weil ihm immer wieder Prozesse mit höherer Priorität vorgezogen werden.

b) Worin liegt der Unterschied zwischen aktivem und passivem Warten?

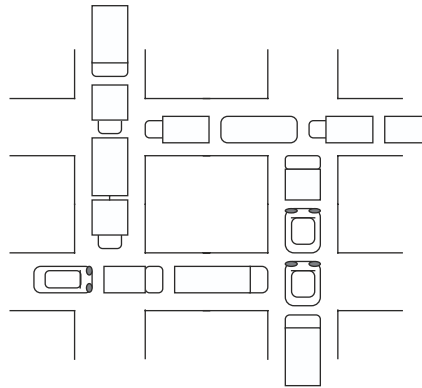
- „Aktives Warten“ bedeutet, dass der Prozess in einer Warteschleife auf das Eintreten eines Ereignisses oder Freiwerden eines Betriebsmittels BM wartet, also *busy waiting* durchführt. Dies kostet Prozessorzeit, die für andere Prozesse sinnvoller genutzt werden könnte.
- „Passives Warten“: Ein Prozess, der auf das Eintreten eines Ereignisses wartet, legt sich „schlafen“, d. h. er nimmt den Zustand *blockiert* an und wird erst dann durch das Betriebssystem in die Liste der rechenbereiten Prozesse eingehängt, wenn das erwartete Ereignis eingetreten ist.

Man könnte meinen, dass das Problem beim passiven Warten nur ins Betriebssystem verlagert wurde. Dies ist jedoch nicht der Fall, da das Betriebssystem sowieso die Kontrolle über die Betriebsmittel und die Ereignisse haben muss und somit eine Verwaltung der Wartebedingungen kaum zusätzlichen Aufwand bedeutet. Es stellt nur eine geringe Extrabelastung dar, wenn beim Freiwerden einer Ressource die darauf wartenden Prozesse überprüft werden.

c) Geben Sie ein praktisches Beispiel für eine Verklemmung an.

Linksabbiegen auf einer Kreuzung (vgl. Duden):

Die Fahrzeuge (Prozesse) fordern das Betriebsmittel „Straßenfläche“ an. A das von B, B das von C, C das von D und D wiederum das von A. In der folgenden Abbildung ist dies am Beispiel einer verklemmten Straßenkreuzung mit Autos und Lastwagen gezeigt.



Aufgabe 2.3-11 Verklemmung/Blockierung

Ein Student S1 hat ein Buch A in der Bibliothek ausgeliehen. In Buch A findet er einen Literaturhinweis auf Buch B. Deshalb möchte er auch Buch B ausleihen. Dies ist momentan von Student S2 ausgeliehen, der wiederum in Buch B einen Verweis auf Buch A findet. Also versucht er, Buch A auszuleihen. Stellt diese Situation eine Verklemmung, eine Blockierung oder keines von beiden dar? Bitte begründen Sie Ihre Antwort.

Diese Situation stellt nicht, wie man leicht glauben könnte, eine Verklemmung (Dead-lock) dar, weil die Bedingung der gegenseitigen Ununterbrechbarkeit nicht gegeben ist. Die Studenten können durch Vormerken des jeweils anderen Buchs von der zentralen Betriebsmittelverwaltung „Bibliothek“ dazu gezwungen werden, ihr Buch zurückzugeben, da sie das Buch nur für eine feste Zeitspanne (Zeitscheibe) belegen können und bei einer Vormerkung ein Buch nicht wieder verlängert werden kann. Somit stellt diese Situation formal nur eine Blockierung dar: A ist so lange blockiert, bis B das Buch abgegeben hat

Außerdem ist aber ein menschlicher Benutzer in der Lage, mit dem bisher entliehenen Buch weiterzuarbeiten. Er ist also auch nicht, wie ein wartender Prozess, richtig blockiert.

Dies könnte man mit der Situation in einem Programm zum Zeichnen von Funktionswerten vergleichen, dessen Aktionen parallel arbeiten: Der eine Subtask berechnet die Funktionswerte, der andere Subtask öffnet ein Anzeigefenster und stellt die berechneten Werte dar. Wenn aufgrund von Speichermangel die Anzeige jedoch zurzeit nicht möglich ist, könnte der Anzeigeprozess jetzt den Berechnungsprozess blockieren. Diesen kann man aber mit Hilfe eines Puffers so konstruieren, dass er auch dann weiterarbeitet, wenn der Anzeigeprozess momentan keine Daten entgegennimmt. Am Ende der Berechnung hat der Benutzer über ein Menü dann nochmals die Chance, sich die Daten aus dem Puffer anzeigen zu lassen.

Eine Blockierung, die sehr lange andauert, kommt einem menschlichen Benutzer wie eine Verklemmung vor. Es ist aus der Sicht des Benutzers ohne Kenntnisse der inneren Zusammenhänge auch nicht festzustellen, ob eine Verklemmung oder nur eine Blockierung vorliegt.

Aufgabe 2.3-12 Monitore

a) *Implementieren Sie das Leser/Schreiber-Problem als Monitorlösung.*

```

monitor ReadWrite
var readers      : integer;
    readers_wait : integer;
    writers_wait : integer;
    writing       : boolean;
    ok_to_read, ok_to_write : condition;

procedure startread;
begin
    if writing or (writers_wait <> 0) then
    begin
        readers_wait := readers_wait + 1;
        wait(ok_to_read);
        readers_wait := readers_wait - 1;
    end;
    readers := readers + 1;
    signal(ok_to_read);
end;

procedure endread;
begin
    readers := readers - 1;
    if readers = 0 then signal(ok_to_write);
end;

procedure startwrite;
begin
    if (readers <> 0) or writing then
    begin
        writers_wait := writers_wait + 1;
        while ((readers <> 0) or writing) wait(ok_to_write);
        writers_wait := writers_wait - 1;
    end;
    writing := true;
end;

procedure endwrite;
begin

```

```

    writing := false;
    if (readers_wait <> 0)
        then signal(ok_to_read)
        else signal(ok_to_write);
end;

begin (* Initialisierung des Monitors *)
    readers := 0;
    readers_wait := 0;
    writers_wait := 0;
    writing := false;
end; (* monitor *)

procedure readprocess
begin
    repeat
        ReadWrite.startread;
        Reading_Data();
        ReadWrite.endread;
    until false;
end;

procedure writeprocess
begin
    repeat
        ReadWrite.startwrite;
        Writing_Data();
        ReadWrite.endwrite;
    until false;
end;

```

b) Was sind die Vor- und Nachteile einer Monitorlösung?

Ein Monitor entlastet den Programmierer bei der Benutzung und korrekten Anwendung von Semaphoren. Werden die Semaphore-Operationen P und V in Programmen nicht in der richtigen Reihenfolge verwendet oder wird eine von beiden vergessen (was in großen Projekten schwer zu überblicken ist), wird die Fehlersuche entsprechend erschwert.

Weiterhin sind die Operationen in einer zentralen Stelle (dem Monitor) zusammengefasst und bilden so etwas ähnliches wie einen ADT (Abstrakten DatenTyp). Zusätzlich werden vom ADT die Semaphore automatisch verwaltet. Es ist also das Verbergen von Information (*information hiding*) möglich. Außerdem wird die Wartung erleichtert, da Änderungen nur im Monitor erfolgen müssen.

Eine Monitorlösung ist leicht nachvollziehbar und somit sicherer gegenüber logischen Fehlern. Ein Monitor besitzt aber auch Nachteile:

- Er muss direkt im Compiler implementiert werden; als Bibliotheksfunktion ist dies nicht möglich. Dies erschwert die Portabilität der Programme.

- Die Wartezustände der Prozesse im Monitor machen es schwierig, bei Zeitüberschreitung, Abschalten des Systems oder Prozessabbrüchen alle Wartezustände konsistent aufzulösen.
- Es kann eine Sequenzialisierung entstehen, wenn die Lese-/Schreiboperationen im Monitor selbst vorgenommen werden. (Nicht im obigen Beispiel!)

Aufgabe 2.3-13 Banker-Algorithmus

a) Welche beiden Reihenfolgen wären im Beispiel für den Banker-Algorithmus für die fünf Prozesse auch möglich?

Nach dem Beispiel sind die existierenden Betriebsmittel mit E, die bestehende Belegung mit B und die gewünschte zusätzliche Belegung mit C gegeben:

E = (6 3 4 2)

B: P_1 3 0 1 1

P_2 0 1 0 0

P_3 1 1 1 0

P_4 1 1 0 1

P_5 0 0 0 0

C: P_1 1 1 0 0

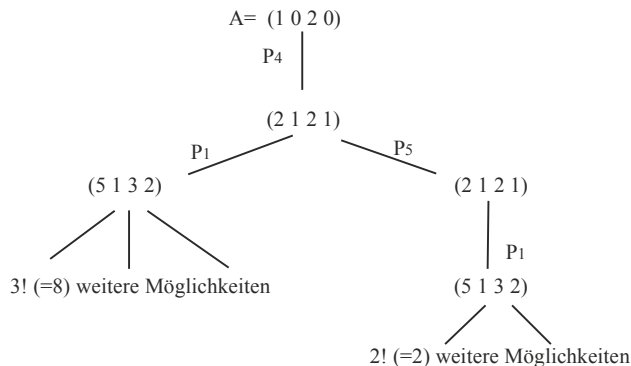
P_2 0 1 1 2

P_3 3 1 0 0

P_4 0 0 1 0

P_5 2 1 1 0

In der folgenden Abbildung ist die Entwicklung der aktuellen Belegung A visualisiert; die Übergänge dazwischen sind mit einer Linie symbolisiert, neben die der Name des aktivierten Prozesses geschrieben ist.



Da auch mehrere bereit sein können, verzweigt sich das Diagramm je nach Entscheidung.

b) Angenommen, Prozess P_1 bekommt ein Bandlaufwerk zusätzlich zugestanden, ist das System dann verklemmungsbedroht?

Falls Prozess P_1 ein weiteres Bandlaufwerk zugestanden bekommt, kommt lediglich der Pfad P_4 – P_5 – ... nicht mehr in Frage. Das System ist also nicht verklemmungsbedroht.

Aufgabe 2.3-14 Verklemmungstest

Gegeben seien fünf Prozesse $P1, \dots, P5$ mit den angegebenen Anforderungen an die Betriebsmittel A, ..., E, wobei ein Eintrag „a/b“ bedeutet, dass der Prozess derzeit a Einheiten des Betriebsmittels hält und noch zusätzliche b Einheiten anfordert.

	Betriebsmittel				
	A	B	C	D	E
P1	1/0	2/4	1/1	1/0	1/0
P2	0/2	0/1	0/3	0/0	0/1
P3	2/0	0/2	1/2	0/2	3/2
P4	0/1	0/4	0/1	0/2	0/0
P5	1/3	1/3	0/1	1/1	1/4

Betriebsmittel	maximal verfügbar
A	4
B	7
C	3
D	2
E	5

Überprüfen Sie, ob in diesem Szenario eine Verklemmung vorliegt. Falls ja, so geben Sie an, wie diese behoben werden kann.
Zunächst einmal kann P1 zu Ende laufen, weil alle seine Anforderungen erfüllbar sind.
Tabelle:

	Betriebsmittel				
	A	B	C	D	E
P2	0/2	0/1	0/3	0/0	0/1
P3	2/0	0/2	1/2	0/2	3/2
P4	0/1	0/4	0/1	0/2	0/0
P5	1/3	1/3	0/1	1/1	1/4

Dann kann P4 zu Ende laufen:

	Betriebsmittel				
	A	B	C	D	E
P2	0/2	0/1	0/3	0/0	0/1
P3	2/0	0/2	1/2	0/2	3/2
P5	1/3	1/3	0/1	1/1	1/3

Dann kommt es jedoch schon zur Verklemmung: kein Prozess kann alle seine Anforderungen erfüllt bekommen: P2 kann keine 2 A erhalten, P3 kann keine 2 D erhalten, P5 kann keine 4 E erhalten.

Verklemmung beheben:
Abbrechen von P2 bringt nichts, da P2 ohnehin keine Betriebsmittel hält. Man könnte jedoch P3 abbrechen. Dann kann P2/P5 durchlaufen, im Anschluss P5/P2. Am Ende kann P3 neu gestartet werden und durchlaufen.

Alternativ: P5 abbrechen. Dann kann P3 durchlaufen, im Anschluss P2. Am Ende kann P5 neu gestartet werden und durchlaufen.

Aufgabe 2.3-15 Verklemmungsfreiheit

Beweisen oder widerlegen Sie:

In einem System mit $n > 1$ verschiedenen Betriebsmitteltypen, in dem jeder Prozess direkt nach seinem Start nacheinander (jedoch in beliebiger Reihenfolge) für jeden dieser Typen alle $m_i \geq 0$ maximal benötigten Betriebsmittel auf einmal anfordern muss und diese erst bei Prozessende wieder freigegeben werden können, sind Verklemmungen unmöglich.

Die Aussage ist falsch.

Gegenbeispiel:

Die Betriebsmittel A und B seien je einmal verfügbar. P1 fordert A an, erhält A. Scheduler unterbricht. P2 fordert B an, erhält B. Scheduler unterbricht. P1 fordert B an, blockiert. Scheduler unterbricht. P2 fordert A an, blockiert.

Es entsteht eine Verklemmung.

Das Problem ist im Grunde die beliebige Reihenfolge der Belegung. Wenn diese festgeschrieben wäre, kann es zu keiner Verklemmung kommen, da die 4. Bedingung (zyklische Wartebedingung) von Coffman et al. verhindert wird.

Aufgabe 2.3-16 Sichere Systeme

- a) *Ein Computersystem habe sechs Laufwerke und n Prozesse, wobei ein Prozess jeweils zwei Laufwerke benötigt. Wie groß darf n sein für ein sicheres System?*

Ein sicheres System darf höchstens fünf Prozesse erlauben, da so stets gegeben ist, dass ein Prozess mit zwei Laufwerken terminieren kann. Nach seiner Beendigung stehen dann diese zwei Laufwerke anderen Prozessen zur Verfügung.

- b) *Für große Werte für m Betriebsmittel und n Prozesse ist die Anzahl der Operationen, um einen Zustand als „sicher“ zu klassifizieren, proportional zu $m^a n^b$. Wie groß sind a und b ?*

Banker-Algorithmus:

FOR $k := 1$ TO n DO unmark(k) END;	n Schritte
$d := 0$;	+
REPEAT	n Schleifen
$k := \text{satisfiedProcess}()$;	$n \cdot m$ Schritte
IF $k \neq 0$ THEN $d := d + 1$;	
markProcess(k);	
FOR $s := 1$ TO m DO	m Schritte
$A[s] := A[s] + B[k, s]$	
END;	

```

        END;
UNTIL k=0;
IF d<N THEN Error („Deadlock“) END;

```

Folglich ist die Anzahl der Schritte

$$\begin{array}{ccccccc}
 & & = & n & + & n & (& n \cdot m & + & m &) \\
 & \nwarrow & & \nearrow & & \nwarrow & & \nearrow & & \nwarrow & & \nearrow \\
 & \text{unmark} & & \text{REPEAT} & & \text{satisfied()} & & \text{Available} & & & &
 \end{array}$$

$$= n + n^2 m + nm \cong O(n^2 \cdot m)$$

Aufgabe 2.3-17 Verklemmungen unmöglich machen

In einem Transaktionsystem einer Bank gibt es für jede der vielen parallel stattfindenden Überweisungen von Konto S nach Konto E einen eigenen Prozess. Die Lese/Schreiboperation pro Konto müssen deshalb vor parallelem Zugriff durch einen anderen Prozess geschützt werden. Wird dazu ein Semaphore pro Konto verwendet, so kann es bei Rücküberweisungen leicht zu Verklemmungen kommen, da zuerst S und dann E von einem Prozess und zuerst E und dann S von einem anderen Prozess belegt werden. Entwerfen Sie ein Zugriffsschema, um eine Verklemmung zu vermeiden.

Beachten Sie bitte, dass Lösungen, die zuerst das eine Konto sperren, es verändern und entsperren und dann das andere belegen und verändern, gefährlich sind, da bei Computerstörungen der Überweisungsbetrag verschwinden kann. Diese Art von Lösungen scheiden also aus.

Eine Lösung könnte wie folgt aussehen:

- Jedes Konto hat eine eindeutige Kontonummer.
- Für eine Transaktion (Überweisung) mit einem Konto S und einem Konto E muss immer zunächst das Konto mit der niedrigeren Kontonummer gesperrt werden und dann das andere Konto. Auf diese Weise wird die *circular wait*-Bedingung für einen Deadlock ausgeschlossen.
- Da beide Konten gleichzeitig gesperrt sind, ist eine echte atomare Transaktion möglich, um Computerstörungen zu tolerieren: nach dem Abspeichern der Zustände von Konto S und E wird der Betrag bei S abgebucht und bei E gutgeschrieben; danach werden beide entsperrt und die Transaktion als „abgeschlossen“ vermerkt. Wird der Gesamtvorgang durch eine Störung unterbrochen, so stellt man beim Wiederanlaufen den Transaktions-Zustand von S bzw. E fest und ersetzt beide durch ihre Sicherungskopien.

Aufgabe 2.4-1 Prozesskommunikation

- a) *Beschreiben Sie detailliert, was folgende Kommandozeile in UNIX bewirkt.*

```
grep deb xyz | wc -l
```

Das Programm `grep` durchsucht eine Datei `xyz` nach einer Zeichenkette `deb` und gibt alle Zeilen aus, in denen sich die Zeichenkette befindet.

`wc -l` zählt die Zeilen einer angegebenen Datei, in diesem Fall der Pipe, also der Ausgabe des Programms `grep`.

Die gesamte Kommandozeile zeigt also die Anzahl der Zeilen in der Datei `xyz` an, in denen die Zeichenkette `deb` vorkommt. Mehrfaches Vorkommen von `deb` in einer Zeile wird wie ein einfaches Vorkommen bewertet. Existiert `xyz` nicht, wird 0 ausgegeben.

Der Ablauf der Abarbeitung sieht im Detail wie folgt aus: Nach Eingabe der Kommandozeile `grep deb xyz | wc -l` öffnet die *shell* zunächst eine *pipe*. Dabei werden zwei Filedescriptoren (FDs) angefordert: einer zur Eingabe in die *pipe* und einer zur Ausgabe aus der *pipe*. Für jeden Prozess (`grep` und `wc`) wird dann `fork()` aufgerufen.

Der erste Kindsprozess schließt den FD der Standardausgabe `stdout` und kopiert den FD der *pipe*-Eingabe auf den Dateideskriptor `stdout`. Die beiden FDs der *pipe* werden sodann geschlossen (`stdout` bleibt aber geöffnet) und `grep` wird mit den Parametern `deb` und `xyz` mittels `exec` gestartet und dem Kindsprozess überlagert. So geht die `stdout`-Ausgabe von `grep` in die *pipe*.

Der zweite Kindsprozess verfährt analog mit `stdin`: Der FD der Standardeingabe wird geschlossen, der FD zum Auslesen aus der *pipe* auf `stdin` kopiert, beide FDs der *pipe* geschlossen. Nun wird `wc` mit dem Parameter `-l` über `exec` aufgerufen.

`grep` öffnet die Datei `xyz`, sucht alle Zeilen, die `deb` enthalten, und schreibt diese nach `stdout`, also in die *pipe*. Sobald kein Platz mehr in der *pipe* vorhanden ist, legt sich `grep` schlafen und wartet, bis wieder genug Platz vorhanden ist.

Ähnlich liest `wc` die Zeilen aus der *pipe* und wartet, wenn die *pipe* leer sein sollte, auf den nächsten Schreibvorgang. Sobald die *pipe* von `grep` geschlossen wird und `wc` eine entsprechende Meldung über das Signalsystem erhält, gibt `wc` auf `stdout` die Zeilenzahl aus.

- b) *In UNIX ist die Kommunikation durch Signale auf eine Eltern/Kind-Gruppe beschränkt. Warum könnten die Implementatoren diese Einschränkung getroffen haben?*

Damit sich verschiedene Benutzer nicht über Abbruchsignale (KILL etc.) gegenseitig Prozesse löschen oder durch Signalüberflutung blockieren können (Sicherheitsaspekt). Auch andere Signale (SIGBRK, ...) könnten zu Problemen führen.

- c) *Formulieren Sie den Übergang zwischen Prozesszuständen aus Abschn. 2.1 mit Hilfe von Nachrichten und Mailboxen. Wer schickt an wen die Nachrichten?*

In einem anonymen Prozesssystem kennen die Prozesse nur sich selbst und den Scheduler/Dispatcher, der bei bestimmten Systemaufrufen (Create Process, Terminate usw.) implizit aufgerufen wird. Alle Nachrichten erfolgen also zwischen den Prozessen und dem Scheduler/Dispatcher. Die Art der Nachricht richtet sich dabei nach der involvierten Funktion. Beispielsweise bewirkt dann die Aktion

```
sendMsg(Scheduler, CreateProcess)
```

das Erzeugen eines Prozesses (z. B. als Kopie des sendenden Prozesses) durch den Scheduler sowie seine Nachricht vom Typ *WaitInReadyQueue* an den erzeugten Prozess. Auch das Umhängen von einer Warteschlange in eine andere kann mit Hilfe einer Nachricht, die der betreffende Prozess erhält, gesteuert werden. Zu jedem Zustandsübergang in [Abb. 2.2](#) gibt es also einen Nachrichtentyp und einen Aufruf des Schedulers/Dispatchers.

Aufgabe 2.4-2 Pipes

Betrachten Sie ein Prozesssystem, das nur mit UNIX-pipes kommuniziert.

- a) *Unter welchen Umständen wird ein Prozess gezwungen, zu warten?*

Diese Situation kann mit dem Erzeuger/Verbraucher-Modell beschrieben werden. Ein Prozess muss warten, wenn der erzeugende Prozess die *pipe* nicht gefüllt hat und der Pufferspeicher deshalb leer ist.

- b) *Skizzieren Sie einen geeigneten Mechanismus, um für dieses System Verklemmungen festzustellen oder zu verhindern.*

Eine Verklemmung kann sich ergeben, wenn alle vier Verklemmungsbedingungen gegeben sind. Dies ist der Fall, wenn Zyklen existieren und auf mehrere Eingaben gewartet wird, bevor eine Ausgabe gemacht wird. Die Zyklen können beispielsweise bei der Initialisierung des Prozesssystems vom Elternprozess, der alle Prozesse und die *pipes* aufsetzt, durch einen Test des Prozess-*pipe*-Verbindungsgraphen auf Zyklenfreiheit verhindert werden.

Ist dies nicht möglich, so kann dem Elternprozess ein Signal beim Lesen und eines beim Schreiben zugeschickt werden. Senden nicht alle Prozesse Signale in einem bestimmten Zeitraum, so kann der Elternprozess eine Verklemmung anhand der Signalarten feststellen.

- c) *Gibt es eine Regel in diesem System, um den „Opferprozess“ auszuwählen wenn eine Verklemmung existiert? Wenn ja, begründen Sie die Regel.*

Zweifelsohne ist ein Prozess, der beim Schreiben blockiert wurde, da die *pipe* überläuft, nicht „der Schuldige“. Die Prozesse, die beim Lesen blockierten, kann man aber nicht bewerten, da sie im Zyklus warten könnten.

Eine gute Regel lässt sich in diesem Fall nicht nennen, da die Logik der Verarbeitung alle Prozesse benötigt; der Abbruch nur eines Prozesses lässt die Arbeit der anderen wertlos erscheinen, da sie nicht unabhängig voneinander arbeiten.

9.3 Lösungen zu Kap. 3

Aufgabe 3.1-1 Speicherbelegungsstrategien

Gegeben sei ein Swapping-System, dessen Speicher aus folgenden Löchergrößen in ihrer Speicherreihenfolge besteht: 10 KB, 4 KB, 20 KB, 18 KB, 7 KB, 9 KB, 12 KB und 15 KB.

Welches Loch wird bei der sukzessiven Speicherplatzanforderung von 12 KB, 10 KB, 9 KB mit First-Fit ausgewählt?

Wiederholen Sie die Anforderungen für Best-Fit, Worst-Fit und Next-Fit (Skizze).

Die folgende Tabelle zeigt die Speicherbelegung nach den drei Anforderungen. Dabei ist ein besetzter Speicherbereich grau schraffiert. Der Zeiger für Next-Fit stehe anfangs auf dem 10-KB-Block.

Strategie	Speicherbereiche									
(vorher)	10	4	20		18		7	9	12	15
First-Fit				8KB		9KB				
Best-Fit										
Worst-Fit				8KB		8KB				
Next-Fit				8KB		8KB				

Aufgabe 3.1-2 Speicherreservierung im Buddy-System

Ein Buddy-System verwalte einen Speicher der Größe 256 KiB. Zu Beginn ist lediglich ein Speichersegment von 50 KiB genutzt, was durch das Buddy-System mit einem Segment der Größe 64 KiB (und somit 14 KiB Verschnitt) bedient wurde:

0 KiB		128 KiB		256 KiB
	50+14 KiB	64 KiB frei	128 KiB frei	

Erfüllen Sie nun die folgenden Anfragen in der angegebenen Reihenfolge und zeichnen Sie den Speicher nach jedem Schritt neu:

17 KiB allozieren, 33 KiB allozieren, 50 KiB freigeben, 17 KiB freigeben, 30 KiB allozieren, 48 KiB allozieren, 60 KiB allozieren, 48 KiB freigeben, 33 KiB freigeben, 78 KiB allozieren, 50 KiB allozieren, 32 KiB allozieren.

Falls sich eine Anforderung nicht erfüllen lassen sollte, so machen Sie dies deutlich. In unserem Szenario wollen wir annehmen, dass der anfragende Prozess in diesem Fall eine Fehlermeldung erhalten würde.

Geben Sie den Gesamtverschnitt am Ende aller Anfragen an und zeichnen Sie den dazugehörenden Binärbaum.

17 KiB allozieren. Ergebnis:

50 +14 KiB	17 +15 KiB	32 KiB frei	128 KiB frei	
-------------------	-------------------	-------------	--------------	--

33 KiB allozieren:

50 +14 KiB	17 +15 KiB	32 KiB frei	33 +31 KiB	64 KiB frei
-------------------	-------------------	-------------	-------------------	-------------

50 KiB freigeben:

64 KiB frei	17 +15 KiB	32 KiB frei	33 +31 KiB	64 KiB frei
-------------	-------------------	-------------	-------------------	-------------

17 KiB freigeben:

128 KiB frei		33 +31 KiB	64 KiB frei	
--------------	--	-------------------	-------------	--

30 KiB allozieren:

128 KiB frei		33 +31 KiB	30 +2 KiB	32 KiB frei
--------------	--	-------------------	------------------	-------------

48 KiB allozieren:

48 +16 KiB	64 KiB frei	33 +31 KiB	30 +2 KiB	32 KiB frei
-------------------	-------------	-------------------	------------------	-------------

60 KiB allozieren:

48 +16 KiB	60 +4 KiB	33 +31 KiB	30 +2 KiB	32 KiB frei
-------------------	------------------	-------------------	------------------	-------------

48 KiB freigeben:

64 KiB frei	60 +4 KiB	33 +31 KiB	30 +2 KiB	32 KiB frei
-------------	------------------	-------------------	------------------	-------------

33 KiB freigeben:

64 KiB frei	60 +4 KiB	64 KiB frei	30 +2 KiB	32 KiB frei
-------------	------------------	-------------	------------------	-------------

78 KiB allozieren: Geht nicht, die beiden 64-KiB-Stücke können nicht verschmolzen werden. Keine Speicheränderung.

50 KiB allozieren:

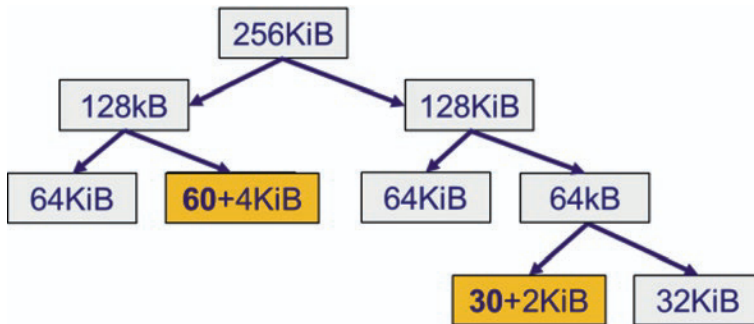
50 +14 KiB	60 +4 KiB	64 KiB frei	30 +2 KiB	32 KiB frei
-------------------	------------------	-------------	------------------	-------------

32 KiB allozieren:

50 +14 KiB	60 +4 KiB	64 KiB frei	30 +2 KiB	32 KiB
-------------------	------------------	-------------	------------------	---------------

Der Gesamtverschnitt ist $(14+4+2+0) = 20$ KiB.

Der Binärbaum faltet sich auseinander und schließt sich wieder zusammen, entsprechend den Anforderungen. Der Zwischenzustand nach der Freigabe von 33 KiB und vor der Allokierung von 78 KiB erklärt, warum diese Anforderung nicht bedienbar ist, obwohl die Summe aller freien Speicherstücke größer ist als 78 KiB.



Jedes Einzelstück ist zu klein, und es gibt keine freien Speicherbuddys, die verschmelzen können.

Aufgabe 3.3-1 Virtuelle Adressierung

Wir haben einen Computer mit virtuellem Speicher.

Folgende Daten sind bekannt:

- 32 virtuelle Seiten
- 8 physische Seiten
- 11 Bit Seitenoffset

Beantworten Sie damit folgende Fragen:

- a) *Wie viele Bits werden für die Adressierung von virtuellen Seiten benötigt?*
Es werden 5 Bits benötigt für die Adressierung von $32 = 2^5$ virtuellen Seiten.
- b) *Wieviel physischen Speicher hat der Computer?*
phys. Seitenzahl $\times$ Seitengröße (=Offsetgröße) = $2^3 \times 2^{11}$ Byte = 16 KiB
- c) *Wie groß ist der virtuelle Adressraum des Computers?*
virt. Seitenzahl $\times$ Seitengröße (=Offsetgröße) = $2^5 \times 2^{11}$ Byte = 64 KiB
- d) *Wie viele Bits werden für die Adressierung der physischen Seiten benötigt?*
Es werden 3 Bits benötigt für die Adressierung von $8 = 2^3$ physischen Seiten.

Aufgabe 3.3-2 Adresstabellen

Eine Maschine hat virtuelle 128-Bit-Adressen und physikalische 32-Bit-Adressen. Die Seiten sind 8 KW groß.

- a) *Wie viele Einträge werden für eine konventionelle bzw. für eine invertierte Seitentabelle benötigt?*

Eine Seite mit $8 \text{ KW} = 8192 \text{ Worten} = 2^{13}$ Adressen wird durch eine 13-Bit-Adresse beschrieben. Die Anzahl der Einträge für die einstufige Seitentabelle ist identisch mit der Anzahl M der möglichen 13-Bit-Seiten im 128 Bit breiten Adressraum, also der Anzahl der Einträge bei $(128 - 13 =) 115$ -Bit-Adressinformation: $M = 2^{115}$.

Tatsächlich aber benötigen wir nur $32 - 13 = 19$ Bit, um eine Seite im 32-Bit-Hauptspeicher zu beschreiben. Die invertierte Seitentabelle benötigt also nur $M = 2^{19} = \frac{1}{2} \cdot 2^{20} = 0,5 \text{ Mio.}$ Einträge – ein Bruchteil der obigen Seitentabelle.

- b) *Wie viele Stufen werden für eine mehrstufige Seitentabelle benötigt, um unter 1 MW (wobei 1 W = 1 Eintrag) Seitentabellenlänge zu bleiben?*

Eine Seitentabelle der Länge $M = 1 \text{ MW} = 2^{20}$ Einträgen benötigt Adressen von 20-Bit-Länge. Die benötigten $128 - 13 = 115$ Adressbits für einen 8 KW Seitenrahmen müssen also auf $\lceil 115/20 \rceil = 6$ Stufen aufgeteilt werden, um höchstens 1 MW Einträge pro Tabelle zu erhalten.

Weitere Stufen wären denkbar, sind aber wegen der Gefahr eines Seitenfehlers beim Zugriff auf die weniger häufig referenzierten Seitentabellen und der erhöhten Zugriffszeit pro Stufe möglichst zu vermeiden.

Es sollte hier aber auch auf die Probleme eines derartigen mehrstufigen Zugriffs aufmerksam gemacht werden: Die sechsstufige Realisierung erfordert 7 Hauptspeicherzugriffe, um das Datum auszulesen: sechsmal in den Seitentabellen lesen und einmal das Datum. Die Prozessorleistung (Operationen pro Sekunde) wird durch den umständlichen Speicherzugriff auf $1/6 \approx 16 \%$ gedrückt.

Besser hätte die Rechnung ausgesehen, wenn wir nur mit einer 48 Bit virtuellen Adresse bei 13 Bit (8 KW) langen Seiten gerechnet hätten. Dann ergibt sich mit den $48 - 13 = 35$ Bits für die Adressinformation der Seitentabelle eine zwei- bis dreistufige Seitentabelle, die in Verbindung mit einem TLB (assoziativem Cache) gute Zugriffsergebnisse auf den Hauptspeicher liefert:

- $2^{48 - 13} = 2^{35} = 2^{15} \text{ MW}$ wäre die Anzahl der Seiteneinträge für eine einstufige Seitentabelle.
- $2^{32 - 13} = 2^{19} = 0,5 \text{ MW}$ Einträge sind dagegen nur nötig für die invertierte Seitentabelle.
- $\left\lceil \frac{35}{12} \right\rceil = 3$ Stufen müsste eine Seitentabelle haben, um in jeder Stufe eine maximale Länge von 12 Bit (4 KW) zu besitzen.
- $\left\lceil \frac{35}{18} \right\rceil = 2$ Stufen müsste eine Seitentabelle haben, um in jeder Stufe eine maximale Länge von 18 Bit (256 KW) zu besitzen.

Aufgabe 3.3-3 Einstufige Adressumrechnung

- a) *Berechnen Sie zu den gegebenen virtuellen Adressen 2204_H und $A226_H$ jeweils die physikalische Adresse. Der Offset beträgt 13 Bit und die Seitennummer hat 3 Bit. Verwenden Sie folgende Seitentabelle:*

Seite	Seitenrahmen
0	7
1	0
2	1
3	6
4	4
5	2
6	3
7	5

Die physikalischen Adressen ermitteln sich wie folgt:

- 1) Ermitteln des Seitenrahmens: Zuerst Umwandeln der virtuellen Adressen in ihre Binärcodierung. Dabei trennt der Doppelpunkt den Tabellenindex vom 13 Bit-Offset. Dann Auslesen des Seitenrahmens aus dem Tabelleneintrag.

$$2204_H = 001 : 0\ 0010\ 0000\ 0100 \Rightarrow \text{Tabellenindex} = 1 \Rightarrow \text{Seitenrahmen} = 0$$

$$A226_H = 101 : 0\ 0010\ 0010\ 0110 \Rightarrow \text{Tabellenindex} = 5 \Rightarrow \text{Seitenrahmen} = 2$$

- 2) Physikalische Adresse = Seitenrahmen konkateniert mit Offset

$$0 : 204_H = 000 : 0\ 0010\ 0000\ 0100 = 0204_H = 516$$

$$2 : 226_H = 010 : 0\ 0010\ 0010\ 0110 = 4226_H = 16934$$

- b) *Wie müsste eine Speicherverwaltung erweitert werden, damit die Seiten ausgelagert werden können?*

Zur Auslagerung müsste jeder Seite eine Bit-Information darüber enthalten, ob die Seite im Speicher ist oder auf Platte. Des Weiteren braucht man eine Ein-/ Auslagerungsstrategie, die die zu ersetzende Seite wählt. Außerdem benötigt man natürlich die Information über die Adressen der Seiten auf dem Auslagerungsmedium.

Aufgabe 3.3-4 Zweistufige Adressumrechnung

Gegeben sei die folgenden 4 virtuellen 16-Bit-Speicheradressen in Hexadezimaldarstellung:

- a. 75B4
- b. 8AC6
- c. 1E9C
- d. 5B3E

Die Adresse ist von links nach rechts kodiert. Das erste Bit wird nicht benutzt. Die nächsten 3 Bits kodieren den jeweiligen Index in der Basis-Seitentabelle, die weiteren 3 Bits den Index in der entsprechenden Tafel. Die übrigen 9 Bits bilden den Offset.

Index 0				Index 1				Offset							
15	14	13	12	11	10	9	8	7	6	5	4	3	2	1	0

Bestimmen Sie anhand der folgenden Abbildung die zu a), b), c) und d) gehörenden physikalischen 32 Bit-Adressen. Geben Sie diese in Hexadezimaldarstellung an.

Die physikalische Speicheradresse der Basis-Seitentabelle ist 9A38B75A.

7	NIL
6	39BE
5	B3C7
4	5B4C
3	NIL
2	1A75
1	87D3
0	A380

B47AB75A

7	B47AB75A
6	NIL
5	5A37B75A
4	A3EBB75A
3	B47AB75A
2	A3EBB75A
1	A3EBB75A
0	5A37B75A

9A38B75A

7	NIL
6	B125
5	4CC1
4	3333
3	B97C
2	NIL
1	5E4A
0	NIL

5A37B75A

7	NIL
6	D2A6
5	8AC6
4	2BE9
3	NIL
2	D17F
1	NIL
0	7A34

A3EBB75A

- a) 75B4 → 0111 0101 1011 0100
 → 7 2 → 1A75 → 0 001.1 010.0 111.0 1011 1011 0100
 Ergänzung für 32 Bit mit führenden Nullen → **0034EBB4**
- b) 8AC6 → 1000 1010 1100 0110
 → 0 5 → 4CC1 → 0 100.1 100.1 100.0 0010 1100 0110
 Ergänzung für 32 Bit mit führenden Nullen → **009982C6**
- c) 1E9C → 0001 1110 1001 1100
 → 1 7 → **Fehler!** Die Adresse existiert nicht
- d) 5B3E → 0101 1011 0011 1110
 → 5 5 → 4CC1 → 0 100.1 100.1 100.0 0011 0011 1110
 Ergänzung für 32 Bit mit führenden Nullen → **0099833E**

Aufgabe 3.3-5 Mehrstufige Seitentabelle

Eine Maschine habe einen virtuellen Adressraum. Die Speicherverwaltung benutzt eine zweistufige Seitentabelle mit einem assoziativen Cache (TLB- Translation Lookaside Buffer) mit einer durchschnittlichen Trefferrate von 90 %. Beachten Sie, dass ein Zugriff zwei „Wege“ benutzen kann: TLB oder über die Seitentabelle.

- a) Wie groß sind die mittleren Zeitkosten für den Zugriff auf den Hauptspeicher, wenn die Zugriffszeit des Speichers 100 ns und die Zugriffszeit des TLB 10 ns beträgt? (Es wird angenommen, dass keine Seitenfehler auftreten!)

Zugriff TLB + HS = 10 ns * 0,9 = 9 ns

Zugriff Seitentabelle + HS = 300 ns * 0,1 = 30 ns

Zugriff gesamt = 129 ns

b) In a) wurden Seitenfehler ausgeschlossen. Wir wollen nun den vereinfachten Fall untersuchen, in dem Seitenfehler nur beim Hauptspeicherzugriff auf das Datum auftreten sollen (vereinfachende Annahme: Page-Tables sind im HS und werden nicht ausgelagert!). Die Häufigkeit von Seitenfehlern sei $1:10^5$. Ein Seitenfehler koste 100 ms.

Wie ist dann die mittlere Zugriffszeit auf den Hauptspeicher? Die mittlere Zugriffszeit sei definiert als Trefferzeit + Fehlzugriffsrate $\times$ Fehlzugriffszeit.

$$\text{Zugriff wie in a)} = \frac{99999}{100000} * 129 \text{ ns} = 128,999 \text{ ns}$$

$$\text{Zugriff mit Pagefault} = 100 \text{ ms} * \frac{1}{100000} = 1000 \text{ ns}$$

$$\text{Zugriff gesamt} = 1128,999 \text{ ns}$$

c) Welche Probleme verursacht Mehrprogrammbetrieb für den TLB und für andere Caches? Denken Sie vor allem an die Identifikation der Blöcke.

Bei einem Prozesswechsel stehen noch die Daten (Adressen) des alten Prozesses im Cache. Verwendet man einen Cache mit physikalischer Adressierung so ist ein Cache-Flush (Cache-Entleerung) nötig. Arbeitet man mit virtuellen Adressen im Cache, so zeigen diese auf falsche physikalische Adressen, da beide Prozesse denselben virtuellen Adressraum haben. Als Lösung kann man eine Prozessidentifikation PID einführen, damit werden die Adressen eindeutig.

Aufgabe 3.3-6 Seitenersetzung

Was sind die grundlegenden Schritte einer Seitenersetzung?

Nach einem *page fault* wird eine Seite gesucht, die ausgelagert werden kann um Platz für eine andere, früher ausgelagerte Seite zu bieten. Dazu benutzt man einen Ersetzungsalgorithmus, der die vorhandenen Seiten überprüft und eine davon auswählt. Dann wird die vorhandene Seite ausgelagert, die Seitentabellen korrigiert, die andere Seite vom Massenspeicher in den Hauptspeicher kopiert und die Seitentabelle dafür ebenfalls aktualisiert. Danach wird das Programm, beginnend mit der Instruktion, die zum *page fault* führte, weiter ausgeführt.

Aufgabe 3.3-7 Referenzketten

Gegeben sei folgende Referenzkette: 0 1 2 3 3 2 3 1 5 2 1 3 2 5 6 7 6 5, und jeder Zugriff auf eine Seite erfolgt in einer Zeiteinheit.

- a) Wie viele Working-Set-Seitenfehler ergeben sich für eine Fenstergröße $h = 3$?
Werden die Seiten nach der FIFO-Strategie ersetzt, dann ergibt sich folgender Ablauf:

Anford.	0 1 2	3 3 2 3 1 5 2 1 3 2 5 6 7 6 5
HS-RAM	0 1 2	3 3 3 3 3 5 5 1 1 2 2 6 7 7 5
	0 1	2 2 2 2 2 3 3 5 5 1 1 2 6 6 7
	0	1 1 1 1 1 2 2 3 3 5 5 1 2 2 6
Platte		0 0 0 0 0 1 1 2 2 3 3 5 1 1 2
		0 0 0 0 0 0 3 5 5 1
		0 3 3 3
		0 0 0
page fault	***	* * * * *

- Ohne die ersten drei initialen Seitenfehler treten insgesamt 7 Seitenfehler auf.
- b) Welches weitere strategische Problem ergibt sich, wenn innerhalb eines Working-Sets eine neue Seite eingelagert werden muss? Überlegen Sie dazu, wann es sinnvoll ist, eine Seite des Working-Sets zu ersetzen, und wann es sinnvoll ist, eine Seite hinzuzufügen und ggf. die Fenstergröße dynamisch zu verändern, falls die maximale Working-Set-Größe noch nicht ausgeschöpft ist!

Es muss entschieden werden, ob eine Seite ausgelagert oder ob eine neue Seite zusätzlich eingelagert werden soll. Erstgenanntes ist sinnvoll, wenn sich das Working-Set verkleinert hat und eine länger unbenutzte Seite (oder mehrere) enthält.

Gehören dagegen alle im Fenster referenzierten Seiten zum Working-Set und werden weitere Seiten darin benötigt, so sollte die Fenstergröße wachsen, d. h. eine neue Seite zum Working-Set hinzugefügt werden. Dabei kann man die Wachstumsgrenze durch einen Wert festlegen.

Aufgabe 3.3-8 Auslagerungsstrategie

Ein Rechner besitzt vier Seitenrahmen. Der Zeitpunkt des Ladens, des letzten Zugriffs und die R und M Bits für jede Seite sind unten angegeben (die Zeiten sind Uhrticks):

Seite	Geladen	Letzte Referenz	R	M
0	126	279	0	0
1	230	260	1	0
2	120	272	1	1
3	160	280	1	1

- a) Welche Seite wird NRU ersetzen?
- Für NRU wird zuerst die Liste der Seiten nach den geringsten Benutzungsanzeichen, also der „lowest numbered pages“ (R, M) der Spalten „M“ und „R“ geordnet. In dieser Kategorie ist als kleinster Wert mit $R = 0$, $M = 0$ nur eine Seite, die Seite 0, vorhanden. Würden mehrere Seiten mit gleichem R und M existieren, so würde eine per Zufall ausgewählt.

b) *Welche Seite wird FIFO ersetzen?*

FIFO ordnet die Liste der Seiten nach der Spalte „Geladen“: Die Seite, die am längsten im Hauptspeicher verweilt, wird ausgelagert. Dies ist Seite 2 (zum Zeitpunkt „120 Ticks“ geladen).

c) *Welche Seite wird von LRU ersetzt?*

Hier wird nach der Spalte „letzte Referenz“ geordnet: Die Seite mit der „ältesten letzten Referenz“, also der ältesten Benutzung, ist Seite 1.

d) *Welche Seite wird die Second-Chance ersetzen?*

Beginnt wie FIFO bei der ältesten Page, also 2. Da das R-Bit = 1 ist, wird das Bit gelöscht und Seite 2 ans Ende der Liste gestellt (Second-Chance). Es wird weitergesucht, bis eine Seite mit R-Bit = 0 gefunden wird: Seite 0.

Aufgabe 3.3-9 Working-Set

a) *Was versteht man unter dem working set eines Programms, wie kann es sich verändern?*

Das Working-Set eines Prozesses ist die Menge von Speicherseiten, die er aktuell adressiert. Aktuell ist dabei als ein Zeitfenster zu betrachten. Das *working set* kann während des Programmablaufs wachsen oder schrumpfen, wenn Speicher angefordert wird oder das Programm verschiedene Programmteile durchläuft.

b) *Welchen Effekt hat eine Erhöhung der Anzahl der Seitenrahmen im Arbeitsspeicher in Bezug auf die Anzahl der Seitenfehler? Welchen Effekt hat das auf die Seitentabellen?*

Wenn die Anzahl der Seitenrahmen erhöht wird ohne den physikalischen Speicher zu vergrößern, wird die Größe einer einzelnen Seite geringer. Damit können mehr und kleinere Speicherbereiche adressiert werden. Dies führt jedoch zu einer Vergrößerung der Seitentabellen und die Zahl der Seitenfehler nimmt zu.

Eine größere Seitengröße hat den Nachteil, dass u. U. wegen ein paar Bytes eine ganze Seite (z. B. 4 KB) ein-/ausgelagert wird, was die Zeitersparnis durch den Cache verringert. Außerdem dauert der Auslagerungsvorgang natürlich bei großen länger als bei kleineren Seitengrößen. Die kleineren Seitenrahmen verringern demnach die obigen Effekte.

Die feinere Granularität des Speichers verringert auch den Effekt der internen Fragmentierung, d. h. der Verschnitt des Speichers wird geringer. Somit kann das *working set* besser angenähert werden

Man muss also versuchen, einen Kompromiss zwischen der Anzahl der Seitenfehler und der „Kosten“ der Seitenfehler sowie Speicherfragmentierung mit Hilfe von Simulationen zu finden.

Aufgabe 3.3-10 Seitenersetzung vs. swapping

Was sind die wichtigsten Eigenschaften von page faults, working set und swapping? Beschreiben Sie deren Vor- und Nachteile.

- a) **page fault:** Der Zugriff auf eine Seite, die nicht im Hauptspeicher vorhanden ist, erzeugt einen Seitenfehler. Bei einem Seitenfehler lagert das Betriebssystem eine wenig benutzte Seite auf die Festplatte aus und lädt die benötigte Seite in den frei gewordenen Seitenrahmen. Vorteilhaft ist hierbei, dass nur bei den benötigten Seiten eingegriffen wird, sonst nicht. Nachteilig wirkt sich aus, dass dies in Echtzeit geschehen muss und nicht bereits vorher im Hintergrund, da die Entscheidung, ob ein Programm bei einer Verzweigung eine neue Seite benötigt, erst bei der Ausführung fällt.
- b) **working set:** Die mittlere (und nicht: minimale) Menge der von einem Prozess zu einem bestimmten Zeitpunkt benötigten Seiten wird *Arbeitsmenge* (Working Set) genannt. Eine Methode, die *working set*-Strategie zu verwenden, ist folgende: Die Prozesse werden anfangs ohne alle benötigten Seiten geladen. Erst bei einem Zugriff auf eine nicht vorhandene, aber benötigte Seite wird ein Seitenfehler erzeugt und die Seite geladen. Die meisten Prozesse neigen dazu, jeweils nur auf einen beschränkten Speicherbereich zuzugreifen und benötigen deshalb jeweils nur eine geringe Menge an Seiten. Ist nicht ausreichend Platz da, so kann in Windows NT durch *Trimmen* ein *working set* von der mittleren auf die nötige Seitenmenge reduziert werden. Der Vorteil eines *working sets* ergibt sich, wenn seine Größe, im laufenden Betrieb ermittelt, weitere Seitenfehler unwahrscheinlich macht. Nachteilig ist dabei, dass dies nicht vorher festgestellt werden kann, sondern erst im laufenden Betrieb.
- c) **swapping:** Alle laufenden Prozesse benötigen oft mehr Speicher als physikalisch vorhanden ist. Deswegen werden alle Prozesse geringer Priorität auf Massenspeicher in einen speziellen *swap*-Bereich ausgelagert und nur in den Arbeitsspeicher geladen, wenn sie benötigt werden. Der Vorteil besteht darin, dass alle Seiten des Prozesses ausgelagert werden und damit viel Platz im Hauptspeicher frei gemacht wird. Der Nachteil ist, dass eine Wiederaufnahme des Prozesses viel Zeit benötigt, um alle benötigten Seiten in den Hauptspeicher zu holen.

Aufgabe 3.3-11 Thrashing

- a) *Was ist die Ursache von thrashing?*

Unter *thrashing* versteht man das ständige Ein-/Auslagern von Seiten. Dies ist häufig der Fall, wenn die *working sets* der abzuarbeitenden Prozesse nicht mehr vollständig in den physikalischen Speicher passen (Hauptspeicher). Wechselt ein Prozess das *working set*, so müssen Seiten eines anderen Prozesses ausgelagert werden. Kommt dieser wieder zum Zug, so muss er erst einmal die ausgelagerten Seiten zum Arbeiten wieder einlagern. Es kann eine Situation entstehen, in der fast die gesamte Zeit mit *swapping* verbracht wird, ohne dass die Prozesse sinnvoll arbeiten.

- b) *Wie kann das BS dies entdecken, und was kann es dagegen tun?*

Das BS kann dies dadurch entdecken, dass die Summe der I/O-Zeiten für Seitenersetzung länger werden als die Zeiten des Abarbeitens der Prozesse.

Es bieten sich drei Lösungsmöglichkeiten:

- Prozesse suspendieren/auslagern: Für einen Prozess ist es besser, eine Zeit lang nichts zu tun (und auf der Platte ausgelagert zu sein), als in einem völlig überlasteten System zu arbeiten. Durch das Suspendieren wird Hauptspeicherplatz frei, damit können die laufenden Prozesse ihr *working set* einlagern und das *thrashing* vermindert sich (hoffentlich!).
- Alternativ könnte man eine Prüfung auf *thrashing* einführen.
- Der Speicher, den ein Prozess im HS belegen darf, könnte beschränkt werden.

Aufgabe 3.3-12 Thrashing selbst erleben

Erzeugen Sie nun selbst eine Thrashing-Situation. Besorgen Sie sich dafür eine möglichst große Bilddatei (am besten mehr als 1 GB) und öffnen sie diese mehrfach als Kopie, bis ihr Arbeitsspeicher voll ist.

- a) *Beobachten und dokumentieren sie dabei jeweils, wie sich ihr PC im Laufe des Experiments verhält bezüglich der CPU – Auslastung, des freien und virtuellen Speichers und der I/O-Aktivität.*

Der Hauptspeicher hat nicht genug Kapazität, um die Aufgabe schnell und effizient zu lösen:

- CPU – Auslastung steigt je nach CPU des Benutzers Auslastungen des freien und virtuellen Speichers steigen
- I/O-Aktivität steigt zuerst linear, dann stärker als linear

- b) *Nachdem der Hauptspeicher voll ist, öffnen Sie weiterhin Bilder (ohne die alten zu schließen), bis Sie fast nicht mehr arbeiten können. Dokumentieren Sie auch hier die Leistungsdaten.*

Nun steigen die Leistungsdaten (Antwortzeiten, I/O-Aktivität) extrem an, während die CPU- und Speicherauslastung konstant bleiben.

- c) *Schließen Sie danach alle Bilder wieder und dokumentieren die Veränderungen (Prozesse, CPU-Auslastung etc ...), die Sie erkennen, im Vergleich zu dem Stand, bevor sie mit Thrashing angefangen haben. Sollten sich Bilder nicht mehr öffnen lassen, nehmen sie ein anderes Anzeigeprogramm.*

Beim Schließen der Bilder sollten nicht alle direkt geschlossen sein, sondern es dauert eine Zeit, bis jedes Bild geschlossen wird. Außerdem sinkt die maximale Auslastung, die man hat, nicht direkt, sondern fällt erst nach einer gewissen Zeit.

- d) *Nennen Sie Lösungswege, wie man die Probleme des beobachteten Thrashing beheben kann.*

Lösungswege:

- RAM-Speicher des Rechners erhöhen.
- Anzahl der parallel arbeitenden Programme verringern bzw. begrenzen, indem unnötige Prozesse stillgelegt oder erst verzögert gestartet werden.

- Swap-Bereich auf verschiedene Platten verteilen. Dies ist allerdings nur dann erfolgreich, wenn das Betriebssystem die verschiedenen Swap-Bereiche auch parallel nutzt und nicht zuerst den einen Bereich füllt und dann den anderen. Die Methode, nur lokalen Programmcode zu erstellen (inline-code) greift hier nicht, da man weder die vorhandenen Programme neu schreiben kann, noch die Bild-dateien dadurch kleiner werden. Auch die Bild-Dateigrößen kann man nicht ändern, da sie eine Anforderung darstellen, die vorgegeben ist.

Aufgabe 3.4-1 Segmenttabellen

Es sei folgende Segment-Tabelle gegeben:

Segment	Segmentanfangsadresse	Segmentlänge
0	219	600
1	2300	14
2	90	100
3	1327	580
4	1952	96
5	900	30

Finden Sie heraus, ob es bei folgenden Adressen Speicherzugriffsfehler gibt oder nicht.

- 649
- 2310
- 195
- 1727

Bei dieser Aufgabe geht man am besten so vor:

Wir wissen, dass die physikalische Adresse gleich der Segmentsanfangsadresse + Offset ist. Die physikalische Adresse muss also größer als die Segmentsanfangsadresse und kleiner als die Segmentsanfangsadresse + Segmentlänge sein.

- $219 < \mathbf{649} < 819 = 219 + 600$, also OK
- $2300 < \mathbf{2310} < 2314 = 2300 + 14$, also OK
- $90 < 90 + 100 < \mathbf{195} \Rightarrow$ Speicherzugriffsfehler !
- $1327 < \mathbf{1727} < 1907 = 1327 + 580$, also OK

Aufgabe 3.5-1 Cache

Der dem Hauptspeicher vorgeschaltete Cache hat eine Zugriffszeit von 50 ns. In dieser Zeit sei die Hit/Miss-Entscheidung enthalten. Der Prozessor soll auf den Cache ohne Wartezyklen und auf den Hauptspeicher mit drei Wartezyklen zugreifen können. Die Trefferrate der Cachezugriffe betrage 80 %. Die Buszykluszeit des Prozessors ist 50 ns. Berechnen Sie

- a) die durchschnittliche Zugriffszeit auf ein Datum sowie
 b) die durchschnittliche Anzahl der benötigten Wartezyklen.
- a) Mit der Cachezeit $T_c = 50$ ns und der Hauptspeicherzeit $T_h =$ Wartezeit von $3 \cdot 50 = 150$ ns + Zugriffszeit 50 ns = 200 ns errechnet sich die durchschnittliche Zugriffszeit T auf ein Datum zu $T = P(T_c) \cdot T_c + P(T_h) \cdot T_h = 0,8 \cdot 50 + 0,2 \cdot 200 = 40 + 40 = 80$ ns.
- b) Die mittlere Anzahl der Wartezyklen ist $N = P(N_c) \cdot N_c + P(N_h) \cdot N_h = 0,8 \cdot 0 + 0,2 \cdot 3 = 0,6$, also im Mittel weniger als ein Wartezyklus pro Zugriff.

9.4 Lösungen zu Kap. 4

Aufgabe 4.2-1 Isoliertes Verzeichnis

Angenommen, ein Verzeichnis „Test“ sei über/root/ ... zu erreichen. Legen Sie ein Unterverzeichnis darin an. Legen Sie einen *hard link* auf das übergeordnete Verzeichnis an. Es existiert nun ein zirkulärer link. Versuchen Sie jetzt, das Verzeichnis Test aus dem root-Pfad zu löschen. Was erreichen Sie?

Angenommen, man könnte einen *hard link* auf Test legen, dann könnte man auch einen zirkulären *link* erzeugen. Im Unterschied zu früheren Versionen ist es aber in der aktuellen UNIX-Version unmöglich, einen *hard link* auf ein *directory* zu legen. Dies hat zwei Gründe:

- Würde man den *link* zu Test vom übergeordneten Verzeichnis aus löschen, so wäre Test isoliert: Man könnte weder von außen darauf zugreifen noch es löschen. Mit anderen Worten: Man hätte eine Dateisystemleiche!
- Dieser Fall würde unter UNIX jedoch nicht auftreten, da UNIX beim Löschen eines Verzeichnisses testet, ob es leer ist. Da Test einen *link*-Eintrag nach oben enthält, kann es folglich von oben nicht gelöscht werden. Versucht man nun aber innerhalb von Test, den *link* nach oben zu löschen, so passiert dasselbe: Das darüber liegende Verzeichnis und damit der *link* kann nicht gelöscht werden, da es eine Datei Test enthält und deshalb nicht leer ist. In diesem Fall hätte man also ein nicht löschesbares Verzeichnis.

Aufgabe 4.2-2 Namenskonversion

Im vorangehenden Beispiel zur Namenskonversion wurde der Algorithmus für Windows NT beschrieben, um einen langen Namen in einen kurzen, eindeutigen Dateinamen zu konvertieren.

- a) Wie viele lange Dateinamen, die in den ersten 6 Buchstaben übereinstimmen, lassen sich so eindeutig mit nur 8 Buchstaben kodieren?

Die 6 Buchstaben sowie die Zeichen „~n“ mit $n = 1..9$ sind bereits 8 Buchstaben. Es lassen sich somit 9 Dateien (und für die Null eine weitere) mit langem Dateinamen, die in den ersten 6 Buchstaben gleich sind, mit diesem Schema kodieren.

- b) *Wie könnte man das Schema abändern, um mit 8 ASCII-Buchstaben eine größere Anzahl langer Dateinamen eindeutig zu kodieren? Warum, meinen Sie, haben die Implementierer diese Möglichkeit nicht gewählt?*

Das Schema lässt sich erweitern, indem man auch Buchstaben anstelle der Zahlen zulässt. Vermutlich wird dies aber nicht so durchgeführt, weil dann die Dateinamen nicht mehr sofort durch die angehängte Zeichenkette „~n“ als „konvertierte lange Namen“ kenntlich sind.

Statt dessen wird in der aktuellen Implementierung bei Namensgleichheit nach 9 Dateien der lange Dateiname bereits nach dem fünften Buchstaben abgeschnitten und die generische Zeichenkette „~10“ angehängt, die dann wieder bis „~99“ hochgezählt werden kann.

Aufgabe 4.2-3 Extension und Inhalt

ASCII – Dateien haben die gemeinsame Eigenschaft, dass sie lesbaren Text enthalten. Warum kann bei diesen Dateien die Dateinamenserweiterung (Extension) wichtig sein? Wann kann bei Dateien darauf verzichtet werden?

Eine Anwendung kann aus dem Inhalt einer Textdatei im Allgemeinen nicht entscheiden, wie die Datei verwendet werden soll. So haben z. B. C-Quelltexte und C-Header-Dateien die gleiche Syntax, vom *Make*-Befehl oder dem Compiler müssen sie aber unterschiedlich behandelt werden, da sie unterschiedliche Informationen beinhalten. Aus diesem Grund werden die Dateien über verschiedene Dateinamenserweiterungen „.c“ und „.h“ unterschieden. Ähnlich verhält es sich auch bei HTML- und XML-Dateien.

Bei Dateien kann auf die Erweiterung verzichtet werden, wenn die Datei in einer globalen Datenbank (z. B. dem Dateisystem) extra mit ihrer Bedeutung gekennzeichnet werden. Im MAC OS werden dazu nicht sichtbare Dateien mit gleichem Namen erstellt.

Ein anderer Weg besteht darin, die Bedeutung direkt aus dem Inhalt der Datei ermittelt werden kann, z. B. durch einen eindeutigen Bezeichner zu Beginn der Datei. So beginnen z. B. PDF-Dateien der Version 1.5 immer mit dem Texteintrag

%PDF-1.5, ...,

Binäre Dateien haben in Unix eine *magic number*, eine spezielle Zahl, am Anfang der Datei:

47 49 46 38 = GIF

ff d8 ff e0 = JPG

4d 5a = ausführbare Datei unter DOS/Windows

7f 45 4c 46 = ausführbare Datei unter UNIX

Aufgabe 4.2-4 Pfadnamen

- a) Was sind die Vor- und Nachteile von relativen und absoluten Pfadnamen? Wo/wann sollte man welchen Typ einsetzen? Vollziehen Sie dies anhand z. B. eines Compilersystems nach!

	Vorteile	Nachteile
relative	– kürzerer Pfadname – leichte Verschiebbarkeit ganzer Verzeichnisbäume – weniger Tipparbeit – es werden weniger Daten zum Speichern des Pfades gebraucht – lassen sich schneller durchlaufen, als absolute Pfadnamen, da nicht immer wieder Rechte bis zum Startverzeichnis neu überprüft werden müssen	– können bei großen Entfernungen stark anwachsen
absolute	– kurze Pfadangabe bei Positionen dicht am Wurzelverzeichnis – als eindeutige Identifikation bei Programmen mit gleichem Namen, aber verschiedenen Versionsnummern	– statisch und schwer an Strukturen anzupassen – Verzeichnisse lassen sich nicht einfach verschieben und danach ohne Änderung weiterbenutzen (z. B. Compilerverzeichnis mit seinen Unterverzeichnissen, wenn auf diese mit absoluten Pfadnamen zugegriffen wird)

- b) Wie viele Platten-Leseoperationen werden benötigt, um das Änderungsdatum von einer Datei /home/os/source/aufgabe6/musterlösung.pdf auszulesen?

Nehmen sie an, dass der Indexknoten für das Wurzelverzeichnis im Speicher liegt, sich jedoch keine weiteren Elemente entlang des Pfades im Speicher befinden. Nehmen sie außerdem an, dass alle Verzeichnisse jeweils in gesonderten Plattenblöcken gespeichert sind.

Die folgenden Leseoperationen sind erforderlich:

- Ordnerinhalt von/
- Index für Datei/home
- Ordnerinhalt von/home
- Index für Datei/home/os
- Ordnerinhalt von/home/os
- Index für Datei/home/os/source
- Ordnerinhalt von/home/os/source
- Index für Datei/home/os/source/aufgabe6
- Ordnerinhalt von/home/os/source/aufgabe6

- Index für Datei/home/os/source/aufgabe6/musterlösung.pdf

Daraus folgt, dass man insgesamt 10 Lesevorgänge braucht, um die Dateiinformationen zu erfassen und danach den Inhalt zu lesen.

Aufgabe 4.3-1 Objektorientierte Dateiverwaltung

Angenommen, man wollte die Dateiverwaltung in UNIX objektorientiert gestalten. Welche Attribute und Methoden sind für ein UNIX-Verzeichnisobjekt nötig? Welche für ein Dateiobjekt?

Die Zugriffsrechte aus [Abschn. 4.3.1](#) müssen als Attribute vermerkt werden, die mit den entsprechenden Methoden (`chmod()`-Systemaufrufe) verwaltet werden. Bei Dateiobjekten kommen noch zusätzlich die Statusinformationen als Attribute mit ihren Methoden wie `seek()` und `flush()` sowie die Methoden (Datei-Standardfunktionen) wie `ReadFile()` und `WriteFile()` aus [Abschn. 4.4](#) dazu.

Aufgabe 4.3-2 Access Control List

Was sind die Vor- und Nachteile einer Access Control List, die jeweils beim Benutzer für alle Dateien angelegt wird, gegenüber einer, die jeweils bei der Datei für alle Benutzer abgespeichert wird?

	Vorteile	Nachteile
Benutzer	– verbotene Zugriffe können vor dem Zugriff erkannt werden – es können nur solche Objekte erkannt werden, die in der Liste der Zugriffsrechte stehen, d. h. alle unregulierten Dateien sind tabu. Dies vereinfacht die Verwaltung, wenn die meisten Dateien für Benutzer (z. B. <i>guest</i> oder <i>anyone</i>) gesperrt sein sollen.	– höherer Speicherbedarf. Im Extremfall muss bei jedem Benutzer der gesamte Dateibaum mit allen Rechten gespeichert werden. – Suchvorgänge durchlaufen im <i>worst case</i> den ganzen Baum – das Ändern allgemeiner Zugriffsrechte einer Datei hat das Ändern vieler Tabellen von verschiedenen Benutzern zur Folge – die Zugriffspfade zu den Dateien sind u. U. sehr lang, es muss also dafür Platz reserviert werden – wird eine Datei verschoben, so müssen alle Listen aller Benutzer angepasst werden
Datei	– leichtes Ändern – einfache Verwaltung – nur wenige Benutzereinträge müssen durchsucht werden (Berechtigungsprüfung) – beim Verschieben einer Datei müssen alle <i>Access Control Lists</i> nicht neu gesetzt werden	– erst beim Dateizugriff (Öffnen) kann man die Berechtigung feststellen, da die Access Control List dort zu finden ist. – aufwendigeres Ändern eines Benutzers

Aufgabe 4.4-1 Open File

Ein Betriebssystem kann Dateioperationen auf zwei verschiedene Arten durchführen: Entweder muss der Benutzer zum Dateizugriff die Datei vorher öffnen (Normalfall), oder dies geschieht automatisch beim ersten Zugriff auf die Datei.

Im ersten Fall sieht ein Dateizugriff dann z. B. wie folgt aus:

`Open () ... read () ... weitere beliebige I/O-Ops ... Close ()`

Im anderen Fall hätte man nur die Dateizugriffe (I/O-Ops); die Datei würde dann bei Programmende geschlossen. Welche Vor- und Nachteile besitzen die beiden Varianten?

	Vorteile	Nachteile
Normalfall	– Datei kann vor Programmende geschlossen werden, d. h. es sind weniger Dateien gleichzeitig geöffnet, also bessere Ressourcenkontrolle – Es muss nur einmal die Zugriffsberechtigung überprüft werden – file locking erst, wenn es erforderlich ist – höhere Flexibilität	– mehr Aufwand für den Benutzer/Programmierer – fehleranfälliger, da die Reihenfolge <code>open .. close</code> nicht beliebig ist
Automatisch	– dem Benutzer wird Arbeit abgenommen, da <code>open ()</code> und <code>close ()</code> wegfallen – Fehlermöglichkeiten werden reduziert (falsches <i>file handle</i>) – man muss sich nicht darum kümmern, ob die Datei zu öffnen ist und wie.	– (zu) viele Dateien gleichzeitig geöffnet und zu lange: Jeder Kernel besitzt eine Grenze für die maximal geöffneten Dateien. Diese Grenze wird mit dieser Lösung schneller erreicht als mit dem Normalfall. – Blockieren von anderen Prozessen, falls auch diese auf die Datei schreibend zugreifen wollen. – Jeder Dateizugriff muss über einen eindeutigen Namen geschehen. Dadurch muss jedes Mal wieder eine Rechteprüfung oder ein <code>Lookup-File-Name</code> gemacht werden.

Aufgabe 4.4-2 write-Befehl

Es gibt den Befehl `write`, um in Dateien zu schreiben. Gibt es solch einen Befehl auch für Verzeichnisse? Wenn ja, welchen, wenn nein, warum nicht?

Der Inhalt eines Verzeichnisses kann zwar durch Benutzerprogramme gelesen werden, Änderungen erfolgen allerdings nur indirekt durch Aufträge an das Betriebssystem für das Anlegen oder Löschen von Dateien und Verzeichnissen.

Direktes Beschreiben eines Verzeichnisses durch ein Benutzerprogramm ist nicht unbedingt notwendig, kann aber leicht durch Formatverletzungen die Integrität des Dateisystems gefährden.

Aufgabe 4.4-3 copy-Befehl

Jedes Betriebssystem kennt Befehle zur Verwaltung von Dateien. Sie sollen sich in die Lage eines Betriebssystemarchitekten versetzen, der einen `copy`-Befehl schreibt.

- Implementieren Sie den `copy`-Befehl zum Kopieren eines Files des Betriebssystems mit den Bibliotheksfunktionen zur Dateiverwaltung (`read`, `write`, ...).*
- Welche Änderungen müssten für den `move`-Befehl vorgenommen werden? Reicht es, erst zu kopieren und dann die Datei im Quellverzeichnis zu löschen? (Denken Sie an Schutzmechanismen!)*
- Bonus: Wie müsste der `copy`-Befehl aussehen, um ganze Verzeichnisse mit ihren Unterverzeichnissen zu kopieren? Implementieren Sie ihn.*

Lösung zu a) und c) in UNIX in C-Code:

```
#include <sys/types.h>
#include <sys/stat.h>
#include <dirent.h>
#include <fcntl.h>
#include <unistd.h>
#include <limits.h>
#include „ourhdr.h“

#define FMODE      0644      /* rwxr--r-- */
#define DMODE      0755      /* rwxr-xr-x */
#define LBUFSIZE   8192

/* Copy regular files */
void cpreg(char *src, char *dest)
{
    int          fdsrc, fddest;
    long         BUFSIZE;
    struct stat   sbuf;
    char         *buf;
    int          n;

    /* Open source file */
    if ((fdsrc = open(src, O_RDONLY)) == -1) {
        printf(„Fehler beim Lesen der Quelldatei\n“);
        exit(0);
    }

    /* Optimal buffer size */
    if (!stat(src, &sbuf))
        BUFSIZE = sbuf.st_blksize;
    else
        BUFSIZE = LBUFSIZE;

    buf = (char *) calloc(BUFSIZE, sizeof(char));
```

```

/* Create destination file */
if ((fddest = creat(dest, FMODE)) == -1) {
    printf("Fehler beim Erzeugen der Zieldatei\n");
    exit(0);
}

/* Copy loop */
while (n = read(fdsrc, buf, BUFFSIZE))
    write(fddest, buf, n);
}

/* Copy directories */
void cpdir(char *source, char *dest)
{
    DIR          *dp; /* pointer to directory */
    struct dirent *dirp; /* pointer to directory entry */
    char          src_path[256], dest_path[256];

    /* Open directory */
    if ((dp = opendir(source)) == NULL) {
        printf("Verzeichnis nicht lesbar");
        exit(0);
    }

    /* Copy content of directory (without ./ and ../) */
    while ((dirp = readdir(dp)) != NULL) {
        if (strcmp(dirp->d_name, ".")==0
            || strcmp(dirp->d_name, "..")==0)
            continue;

        strcat(strcat(strcpy(src_path, source), "/"), dirp->d_name);
        strcat(strcat(strcpy(dest_path, dest), "/"), dirp->d_name);

        cprec(src_path, dest_path);
    }
}

/* Copy recursive directories and files */
int cprec(char *source, char *dest)
{
    struct stat sbuf;

    if (stat(source, &sbuf))
        exit(0);

    /* if source is directory */
    if (S_ISDIR(sbuf.st_mode))
    {
        if (mkdir(dest, (sbuf.st_mode | DMODE))) {
            printf("Zielverzeichnis konnte nicht
                erstellt werden ");
            exit(0);
        }
    }
}

```

```

/* Copy directory content from source to dest */
cpdir(source, dest);
}

/* if source is regular file */
else if (S_ISREG(sbuf.st_mode))
    cprec(source, dest);

/* other file types (char,block,pipes&fifos,sockets) */
else printf("anderer Typ \n");
}

main(int argc, char *argv[])
{
    if (argc !=3) {
        printf("usage: my_cp <src-file> <dest-file>\n");
        exit(0);
    }
    cprec(argv[1], argv[2]);
}

```

- c) Bei einem `move` muss beachtet werden, dass man über Schreibrechte des Verzeichnisses verfügt, aus dem die Datei herausbewegt (später also gelöscht) werden soll.

Aufgabe 4.4-4 Indexbaum

Gegeben sei eine hierarchisch verzweigte Indexliste, die als mehrstufige Übersetzung von logischer zu physikalischer Blocknummer dient. Sei m die Anzahl der Verzweigungen (Adressen) eines jeden Blocks einer n -stufigen Übersetzung. Wir haben D adressierbare Speichereinheiten auf der n -ten Stufe der Indexliste. Man gehe davon aus, dass im Mittel $m/2$ Einträge pro Block durchsucht werden müssen, wobei m eine reelle Zahl sei.

Berechnen Sie das optimale m , bei dem die mittlere Suchzeit t minimal ist. Auf jeder der n Stufen gibt es m mögliche Einträge im Container. Also stehen den D Speicherblöcken $m \cdot m \cdot \dots \cdot m = m^n$ Einträge auf der n -ten Stufe zur Verfügung. Also gilt $D = m^n$ oder $n = \log_m(D)$. Da pro Stufe $m/2$ Einträge durchsucht werden müssen, ist $t = n \cdot m/2$. Setzen wir für n die obige Beziehung ein, so gilt

$$t(m) = \log_m(D) \frac{m}{2} = \frac{\ln(D)}{\ln(m)} \frac{m}{2} = \frac{\ln(D)}{2} \frac{m}{\ln(m)}$$

Bedingung für ein Extremum ist, dass an dieser Stelle die erste Ableitung gleich null ist. Mit der Quotientenregel erhalten wir für die Ableitung

$$t'(m) = \frac{\ln(D)}{2} \frac{\ln(m) - m^{-\frac{1}{m}}}{\ln^2(m)} = \frac{\ln(D)}{2} \frac{\ln(m) - 1}{\ln^2(m)}$$

Dies ist genau dann null bei $m \neq 0$, wenn $\ln(m) = 1$ oder $m = e$ ist. Nachprüfen zeigt, dass die zweite Ableitung an dieser Stelle positiv ist, so dass hier ein Minimum vorliegt. Also ist für die geringste Zugriffszeit $t = n \cdot m/2 = n \cdot e/2 = \ln(D) \cdot e/2$: der Zugriff ist logarithmisch in der Zahl der Speichereinheiten bei $e/2$ Zugriffen pro Container.

Aufgabe 4.4-5 B*-Bäume

Zeigen Sie: Für die Aufteilung von $2m - 2$ Schlüsseln in drei Teile gilt

$2m - 2 = \lfloor (2m - 2)/3 \rfloor + \lfloor (2m - 1)/3 \rfloor + \lfloor (2m)/3 \rfloor$. Untersuchen Sie dafür die drei Fälle, wenn jeweils $2m - 2$, $2m - 1$ oder $2m$ glatt durch 3 teilbar ist.

Fall 1: $2m - 2$ ist ohne Rest durch 3 teilbar.

$$\begin{aligned} \text{Dann ist } & \lfloor (2m - 2)/3 \rfloor + \lfloor (2m - 1)/3 \rfloor + \lfloor (2m)/3 \rfloor \\ &= (2m - 2)/3 + \lfloor (2m - 1)/3 - 1/3 + 1/3 \rfloor + \lfloor (2m)/3 - 2/3 + 2/3 \rfloor \\ &= (2m - 2)/3 + \lfloor (2m - 2)/3 + 1/3 \rfloor + \lfloor (2m - 2)/3 + 2/3 \rfloor \end{aligned}$$

Da die Brüche $1/3$ und $2/3$ durch die Abrundung wegfallen, ergibt sich

$$= (2m - 2)/3 + (2m - 2)/3 + (2m - 2)/3 = 2m - 2$$

Fall 2: $2m - 1$ ist ohne Rest durch 3 teilbar.

$$\begin{aligned} \text{Dann ist } & \lfloor (2m - 2)/3 \rfloor + \lfloor (2m - 1)/3 \rfloor + \lfloor (2m)/3 \rfloor \\ &= \lfloor (2m - 2)/3 + 1/3 - 1/3 \rfloor + (2m - 1)/3 + \lfloor (2m)/3 - 1/3 + 1/3 \rfloor \\ &= \lfloor (2m - 1)/3 - 1/3 \rfloor + (2m - 1)/3 + \lfloor (2m - 1)/3 + 1/3 \rfloor \end{aligned}$$

Beim ersten Term verringert sich durch die Abrundung die Zahl um 1, beim letzten Term fällt $1/3$ weg und es ergibt sich

$$= (2m - 1)/3 - 1 + (2m - 1)/3 + (2m - 1)/3 = (2m - 1) - 1 = 2m - 2$$

Fall 3: $2m$ ist ohne Rest durch 3 teilbar.

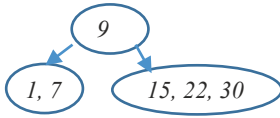
$$\begin{aligned} \text{Dann ist } & \lfloor (2m - 2)/3 \rfloor + \lfloor (2m - 1)/3 \rfloor + \lfloor (2m)/3 \rfloor \\ &= \lfloor (2m)/3 - 2/3 \rfloor + \lfloor (2m)/3 - 1/3 \rfloor + (2m)/3 \\ &= (2m)/3 - 1 + (2m)/3 - 1 + (2m)/3 = (2m) - 2, \end{aligned}$$

was zu beweisen war.

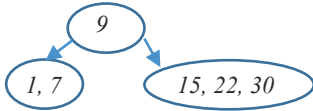
Aufgabe 4.4-6 B-Bäume vs. B*-Bäume

1. Zeichnen Sie den Baum, der sich für $m = 4$ (4 Verzweigungen pro Container) nach Einfügen der Schlüssel 7, 22, 9, 30 ergibt
 - a) in die leere Wurzel eines B-Baumes
 - b) in die leere Wurzel eines B*-Baumes
2. Zeichnen Sie den Baum, der sich für $m = 4$ nach Einfügen der Schlüssel 16 und 3 ergibt

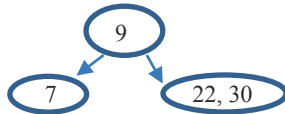
a) in folgenden B-Baum



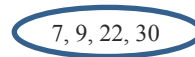
b) in folgenden B*-Baum



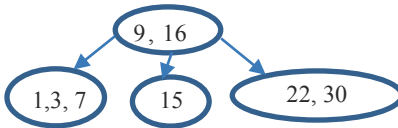
1a)



b)

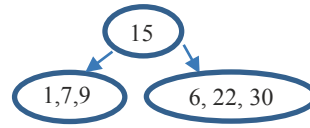


2a) B-Baum: Container zerteilen

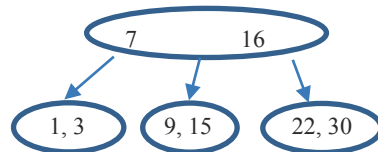


und mit 3 auffüllen.

b) B*-Baum: mit Schlüssel 16 überfließen



und beim Einfügen von Schlüssel 3
in B* -Baum: Container zusammen -
fassen und in drei zerteilen



Aufgabe 4.4-7 I/O

Warum wird in UNIX zwischen der Standardausgabe und der Standardfehlerausgabe unterschieden, wenn doch beide per default auf den Bildschirm gehen?

Unter UNIX ist es üblich, die Standardausgabe (`stdout`) eines Programms mit der Standardeingabe (`stdin`) über eine *pipe* zu verknüpfen. Somit ist es möglich, Daten zwischen zwei Programmen fließen zu lassen, ohne auf temporäre Dateien zugreifen zu müssen. Das erste Programm schreibt seine Daten also in ein `stdout`, das zweite liest diese Daten von seinem `stdin`.

Man möchte aber trotzdem dabei die Möglichkeit haben, Fehlermeldungen und ähnliches auf das aktuelle Terminal und nicht auf die *pipe* auszugeben oder diese, falls sie visuell nicht erwünscht sind, in eine gesonderte Datei umzuleiten. Deshalb existiert als dritter Datenstrom die Standardfehlerausgabe (`stderr`).

Aufgabe 4.4-8 `echo` auf *special devices*

Was passiert, wenn Sie einen Text mit dem Programm `echo` auf `/dev/tty` oder `/dev/null` ausgeben?

- a) `echo „Hallo“ >/dev/null` führt zu einer Ausgabe von „Hallo“ in den imaginären Mülleimer von UNIX. Eine Ausgabeumleitung in diesen ist immer dann sinnvoll, wenn die Ausgabe nicht weiter benutzt wird und auch nicht auf dem Bildschirm ausgegeben werden soll.
- b) `echo „Hallo“ >/dev/tty` gibt den Text auf dem aktuellen Terminal (xterm) aus.

Aufgabe 4.5-1 i-Nodes und Bitmaps

- a) Zwei Informatikstudentinnen, Caroline und Leonore, diskutieren über Indexknoten (I-nodes). Caroline behauptet, dass Speicher so schnell und billig geworden ist, dass es sinnvoll ist, eine Kopie der I-node in den Cache des Prozesses zu laden und dort direkt damit zu arbeiten, und nicht bei jedem Zugriff auf die Datei umständlich das Original auf der Platte zu benutzen.

Leonore ist anderer Meinung.

Wer hat Recht?

Leonore hat Recht. Zwei Versionen einer I-node, eine in dem Cache und eine auf der Platte zur gleichen Zeit zu haben, führt zu Inkonsistenzen, außer beide sind schreibgeschützt. Im schlimmsten Fall werden beide gleichzeitig von unterschiedlichen Prozessen unterschiedlich aktualisiert, was in zwei inkonsistenten I-nodes resultiert. Wenn dann die I-node zurück auf die Platte geschrieben wird, werden die Änderungen des anderen Prozesses überschrieben und die Datenstruktur auf der Festplatte inkonsistent.

- b) Das UNIX – Dateisystem hat 1 KiB – Blöcke und 4 Byte Plattenadressen. Was ist die maximale Dateigröße, falls die I-nodes zehn direkte Einträge und jeweils einen einfach-, einen doppelt und einen dreifach-indirekten Eintrag besitzen?

Die I-node speichert 10 Zeiger zu 10 Blöcken. Ein einzelner, indirekter Block speichert 256 Zeiger. Der doppelt-indirekte Block schafft schon $256^2 = 65.536$ Zeiger und der dreifach-indirekte Block speichert dann $256^3 = 16.777.216$ Zeiger. Das alles aufaddiert ergibt eine Maximalgröße von $10 + 256 + 65.536 + 16.777.216 = 16.843.018$ Blöcken je 1 KiB, was ungefähr 16,06 GiB entspricht.

- c) Nachdem eine Plattenpartition formatiert worden ist, sieht der Anfang einer Bitmap zur Verwaltung der freien Speicherblöcke so aus: 1000 0000 0000 0000, wobei der

erste Block durch das Wurzelverzeichnis belegt sei. Das System sucht nach freien Blöcken, indem es immer mit der niedrigsten Blocknummer beginnt.

Nachdem die Datei A geschrieben worden ist und nun sechs Blöcke belegt, sieht die Bitmap folglich so aus: 1111 1110 0000 0000. Stellen Sie die Bitmap nach jeder der folgenden weiteren Aktionen dar:

- Die Datei B wird geschrieben, B benötigt fünf Blöcke.
- Die Datei A wird gelöscht.
- Die Datei C wird geschrieben, C benötigt acht Blöcke.
- Die Datei B wird gelöscht

Der Anfang der Bitmap schaut folgendermaßen aus:

- nach dem Schreiben der Datei B: 1111 1111 1111 0000
- nach Löschen der Datei A: 1000 0001 1111 0000
- nach dem Schreiben der Datei C: 1111 1111 1111 1100
- nach Löschen der Datei B: 1111 1110 0000 1100

Aufgabe 4.5-2 Speicherverwaltung von Dateien

Die zusammenhängende Speicherung von Dateien führt zu Plattenfragmentierung. Handelt es sich dabei um interne oder externe Fragmentierung?

Es handelt sich um externe Fragmentierung, weil Blöcke unterschiedlicher Größe zwischen den verschiedenen Dateien und nicht innerhalb der Dateien übrigbleiben.

9.5 Lösungen zu Kap. 5

Aufgabe 5.2-1 Gerätemodellierung

- a) Was ist I/O-memory mapping und wozu dient es?

Beim I/O-memory mapping werden die Gerätereister wie Speicher an bestimmten Adressen im Adressraum angesprochen. Damit wird die Benutzung von speziellen, hardware-abhängigen I/O-Maschinenbefehlen vermieden.

- b) Informieren Sie sich über Systemuhr (Real Time Clock) und finden Sie durch geeignete Befehle auf Windows und Linux die I/O-Adressen und den dazugehörigen Interruptvektor heraus.

Nachsehen unter Windows beim Menü Geräte Manager -> Ansicht -> Ressourcen nach Verbindung

Unter Linux:

```
cat /proc/ioports | head -n 8
```

```
0000-0cf7 : PCI Bus 0000:00
```

```
0000-001f : dma1
```

0020–0021 : pic1
 0040–0043 : timer0
 0050–0053 : timer1
 0060–0060 : keyboard
 0064–0064 : keyboard
 0070–0077 : rtc0

Mit dem Befehl `head` wurden nur die relevanten Zeilen am Anfang ausgegeben, damit die Ausgabe nicht zu lang wird:

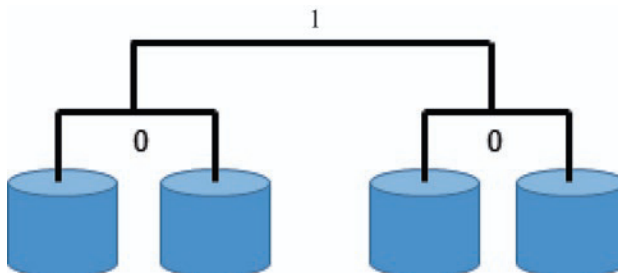
```
cat /proc/interrupts | head -n 4
```

	CPU0	CPU1	CPU2	CPU3			
0:	19	0	0	0	IO-APIC	2-edge	timer
1:	2	0	0	0	IO-APIC	1-edge	i8042
8:	0	0	0	0	IO-APIC	8-edge	rtc0

Aufgabe 5.3-1 RAID-Ausfall

Die Wahrscheinlichkeit, dass eine Festplatte in einem gegebenen Zeitraum einen Fehler (Head-Crash) aufweist betrage p .

- Bestimmen Sie die Wahrscheinlichkeit, dass im Beobachtungszeitraum Daten verloren gehen, für eine Konfiguration mit zwei gleichen Platten für ein RAID-0-System.
- Bestimmen Sie die Wahrscheinlichkeit, dass im Beobachtungszeitraum Daten verloren gehen, für eine Konfiguration mit vier gleichen Platten für ein RAID-01-System, wobei jeweils zwei RAID-0 Platten mittels striping zu einem Subsystem verbunden sind und beide Subsysteme mittels RAID 1 gespiegelt werden.



- Bei 2 Platten D1 und D2 gibt es im RAID0-Modus keine Fehlertoleranz. Das Gesamtsystem fällt also aus, wenn eine der Platten oder beide defekt sind. Die Wahrscheinlichkeit dafür kann auf zwei Arten berechnet werden:

- Zum einen direkt über die Summe aller Ausfallmöglichkeiten, also $P(\text{Ausfall}) = P(\text{Ausfall D1}) + P(\text{Ausfall D2}) + P(\text{Ausfall beider}) = 2(1 - p)p + p^2 = 2p - 2p^2 + p^2 = 2p - p^2$.
- Zum anderen über die Gegenwahrscheinlichkeit $P(\text{ok})$ mit $P(\text{Ausfall}) = 1 - P(\text{ok})$. Da bei Raid0 das System nur dann ok ist, wenn beide Platten ok sind, ist bei unabhängigen Platten die Auftretswahrscheinlichkeit die Produktwahrscheinlichkeit $P(\text{ok}) = (1 - p)(1 - p)$ und damit

$$P(\text{Ausfall}) = 1 - (1 - p)(1 - p) = 1 - [1 - 2p + p^2] = 2p - p^2.$$

b) Bei vier Platten ist das schwieriger direkt hinzuschreiben. Das Problem lässt sich verschieden angehen.

- Die eine Idee besteht darin, beide RAID0-Kombinationen als Subsysteme aus je einem System wie in Aufgabe a) aufzufassen und weiterhin die Information zu verwenden, dass RAID1-Systeme genau dann ausfallen, wenn beide Subsysteme ausgefallen sind. Sind beide Subsysteme unabhängig voneinander, so ist die Auftretswahrscheinlichkeit die Produktwahrscheinlichkeit, also $P(\text{Ausfall}) = P(\text{Ausfall Subsystem A}) P(\text{Ausfall Subsystem B})$. Einsetzen der Ergebnisse von a) ergibt

$$P(\text{Ausfall}) = (2p - p^2)(2p - p^2) = p^4 - 4p^3 + 4p^2$$

- Eine andere Idee ist, wieder die Gegenwahrscheinlichkeit $P(\text{ok})$ einzusetzen. In diesem Fall ist $P(\text{Ausfall}) = 1 - P(\text{ok})$ und $P(\text{ok})$ ist die Summe aller Fälle, in denen das System funktioniert. Dies ist der Fall, wenn kein oder nur ein Subsystem ausgefallen ist:

$$\begin{aligned} P(\text{ok}) &= P(\text{beide Subsysteme ok}) + P(\text{Subsystem 1 ok}) + P(\text{Subsystem 2 ok}) \\ &= (1 - p)^2(1 - p)^2 + (1 - p)^2 [1 - (1 - p)^2] + [1 - (1 - p)^2](1 - p)^2 \\ &= (1 - 2p + p^2)(1 - 2p + p^2) + (1 - 2p + p^2)(2p - p^2) + (2p - p^2)(1 - 2p + p^2) \\ &= 1 - 4p^2 + 4p^3 - p^4 \end{aligned}$$

- Damit ist der Ausfall als Gegenwahrscheinlichkeit

$$\begin{aligned} P(\text{Ausfall}) &= 1 - P(\text{ok}) \\ &= 1 - [(1 - p)^2(1 - p)^2 + (1 - p)^2 [1 - (1 - p)^2] + [1 - (1 - p)^2](1 - p)^2] \\ &= 1 - (1 - 4p^2 + 4p^3 - p^4) = 4p^2 - 4p^3 + p^4 \end{aligned}$$

- Eine sichere, aber umfangreichere Lösung besteht darin, indem man eine Tabelle aller möglichen Zustände der vier Festplatten D1, D2, D3 und D4 aufstellt und die Wahrscheinlichkeiten aller Zustände aufsummiert, die zu Datenverlusten führen.

Dabei berücksichtigen wir die drei Fälle, die zum Ausfall führen: den Ausfall von 4 Einheiten, den Ausfall von 3 Einheiten, und den Ausfall von 2 Einheiten, wenn sie in unterschiedlichen Subsystemen sind.

D1	D2	D3	D4	Ausfall prob.	Gesamtausfall
0	0	0	0	p^4	true
0	0	0	1	$p^3(1-p)$	true
0	0	1	0	$p^3(1-p)$	true
0	0	1	1	$p^2(1-p)^2$	false
0	1	0	0	$p^3(1-p)$	true
0	1	0	1	$p^2(1-p)^2$	true
0	1	1	0	$p^2(1-p)^2$	true
0	1	1	1	$p(1-p)^3$	false
1	0	0	0	$p^3(1-p)$	true
1	0	0	1	$p^2(1-p)^2$	true
1	0	1	0	$p^2(1-p)^2$	true
1	0	1	1	$p(1-p)^3$	false
1	1	0	0	$p^2(1-p)^2$	false
1	1	0	1	$p(1-p)^3$	false
1	1	1	0	$p(1-p)^3$	false
1	1	1	1	$(1-p)^4$	false

Damit ergibt sich die Ausfallwahrscheinlichkeit als Summe von 9 Fällen aus der Tabelle mit „true“ zu

$$P(\text{Ausfall}) = p^4 + 4p^3(1-p) + 4p^2(1-p)^2 = p^4 + 4p^3 - 4p^4 + 4p^2(1-2p+p^2) \\ = p^4 + 4p^3 - 4p^4 + 4p^2 - 8p^3 + 4p^4 = p^4 - 4p^3 + 4p^2$$

- Wir können natürlich auch in diesem Fall wieder die Gegenwahrscheinlichkeit einsetzen. Es ist $P(\text{ok})$ die Summe aller Fälle, bei denen das Gesamtsystem ok ist. Dies sind in der Tabelle alle 7 Fälle mit „false“, so dass sich ergibt

$$P(\text{ok}) = p^2(1-p)^2 + p(1-p)^3 + p(1-p)^3 + p^2(1-p)^2 + p(1-p)^3 + p(1-p)^3 + (1-p)^4 \\ = 2p^2(1-p)^2 + 4p(1-p)^3 + (1-p)^4 \\ = 2p^2(1-2p+p^2) + 4p(1-p)(1-2p+p^2) + (1-2p+p^2)(1-2p+p^2) \\ = (2p^2 - 4p^3 + 2p^4) + (-12p^2 + 12p^3 - 4p^4) + (1 + 6p^2 - 4p^3 + p^4) \\ = (1 - 4p^2 + 4p^3 - p^4)$$

Damit ist der Ausfall als Gegenwahrscheinlichkeit

$$P(\text{Ausfall}) = 1 - P(\text{ok}) \\ = 1 - (1 - 4p^2 + 4p^3 - p^4) = 4p^2 - 4p^3 + p^4$$

Aufgabe 5.3-2 RAID und NON-RAID

Angenommen, Sie haben ein System S1 aus 5 Laufwerken, die jeweils eine Kapazität von 1 TB haben und mit der Wahrscheinlichkeit $p = 0,1\%$ ausfallen. Sie sind als RAID 5 zusammengeschaltet.

Außerdem haben Sie noch ein einzelnes Laufwerk S_2 von 4 TB, das mit $p_2 = 0,001\%$ ausfällt.

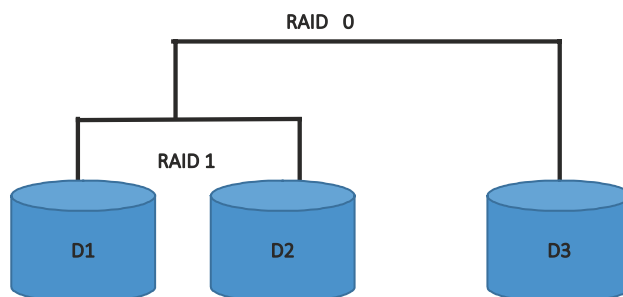
- Welches System hat mehr Speicherkapazität für Anwenderdaten und warum?
 - Welches System ist zuverlässiger als das andere und warum?
 - Wenn beide Systeme gleich viel kosten, welches von beiden sollte man kaufen, und warum?
- Ein RAID5-System hat jeweils eine Einheit (Platte) als Aufnahmegerät für die Paritätsinformation. Zwar sind die Daten in *striping*-Manier verteilt, aber die Speicherkapazität ändert sich nicht, so dass 4 TB für Nutzerdaten zur Verfügung stehen. Im Vergleich zu dem anderen System gibt es für die Speicherkapazität also weder Vorteil noch Nachteil.
 - Die Ausfallwahrscheinlich P ist $P(S_1) = 1 - P(\text{ok}) = 1 - [P(\text{alle 5 Einheiten ok}) + P(\text{nur eine der Platten ist ausgefallen})] = 1 - [(1 - p_1)^5 + 5(1 - p_1)^4 p_1] = 1 - 0,999^5 - 0,005 \times 0,999^4 = 0,998 \cdot 10^{-5}$. Im Vergleich zu S_2 mit $p_2 = 0,001\% = 10^{-5}$ ist das RAID-System also etwas zuverlässiger, aber nicht viel.
 - Entscheidend sind hier die Reparaturkosten: Fallen zwei Platten im RAID-System (und damit das ganze System) aus, so kostet es nur zwei Fünftel des zweiten Systems, um es wieder in einen sicheren Zustand zu versetzen.

Aufgabe 5.3-3 RAID-Ausfallzeit

Gegeben sei folgendes System aus drei Festplatten:

Platte	Kapazität	MTBF	Aktuelles Alter
D1	4TB	630.720 h	3 Jahre
D2	6TB	858.480 h	2 Jahre
D3	8TB	1.156.320 h	1 Jahr

mit folgendem Aufbau



- a) *Geben Sie an, welche Datenmenge darin abgespeichert werden kann, und erklären Sie, wie diese Zahl zustande kommt.*

Es können insgesamt 12 TB abgespeichert werden. Das RAID 1 (*Spiegelplatten*) hat die Kapazität der kleinsten Festplatte von beiden (4 TB), während das RAID 0 (*striping*) die gesamte Kapazität aus beiden Subsystemen (4 + 8 TB) nutzt und Sie nur in Streifen neu organisiert.

- b) *Das genannte System wird von jetzt an 3 Jahre lang genutzt. Berechnen Sie die Wahrscheinlichkeit, dass es während dieser Zeit zu einem Datenverlust kommt.*

Es gelten folgende Annahmen:

- *Beim Auftreten von Fehlern gelten keinerlei Abhängigkeiten zwischen den Festplatten.*
- *Während der bereits erfolgten Betriebszeit traten keinerlei Fehler auf.*

Als gemeinsame Zeiteinheit wählen wir Jahre. Dazu wird MTBF in Jahre umgerechnet: MTBF1 = 72 Jahre, MTBF2 = 98 Jahre, MTBF3 = 132 Jahre

Nun berechnen wir die Ausfallwahrscheinlichkeiten der einzelnen Platten. Die Formel bei einer konstanten Ausfallrate und exponentieller Verteilung war

$$p(T) = 1 - \exp(-T/\text{MTBF})$$

Damit ergibt sich bei $T = 3$ für die Ausfallwahrscheinlichkeiten P_i der drei Platten D_i

$$P_1 = 0,040810543 = 4,08 \%$$

$$P_2 = 0,030148435 = 3,01 \%$$

$$P_3 = 0,022470954 = 2,25 \%$$

Darauf können wir die Ausfallwahrscheinlichkeit von RAID 1 mittels der Tabelle aller möglichen Zustände der Festplatten D_i (0 = defekt, 1 = intakt) berechnen:

Zustand D1	Zustand D2	Ausfall- Wahrscheinlichkeit
0	0	$P_1 * P_2 = 0,001230374$
0	1	$P_1 * (1 - P_2)$
1	0	$(1 - P_1) * P_2$
1	1	$(1 - P_1) * (1 - P_2)$

Das System fällt nur dann aus, wenn beide Spiegelplatten gleichzeitig ausfallen, also

$$D_1 = 0, D_2 = 0 \text{ und } \text{Prob}(\text{RAID1} = 0) = R_1 = 0,001230374 = 1,23 \%$$

Nun können wir damit die Ausfallwahrscheinlichkeit des Gesamtsystems RAID 0 berechnen, bestehend aus den Systemen RAID1 und D3:

Zustand RAID1	Zustand D3	Ausfall- Wahrscheinlichkeit
0	0	$R1 \cdot P3$
0	1	$R1 \cdot (1 - P3)$
1	0	$(1 - R1) \cdot P3$
1	1	$(1 - R1) \cdot (1 - P3) =$ 0,97632632

RAID0 fällt immer dann aus, wenn nur eine Komponente im Gesamtsystem ausfällt. Die Ausfallwahrscheinlichkeit von RAID0 ist also $\text{Prob}(\text{RAID0} = 0) = 1 - P(1,1) = 1 - 0,97632632 = 0,02367368 = 2,4 \%$. Dies kann auch als Summe aller Wahrscheinlichkeiten der Zustände errechnet werden, bei denen mindestens ein Subsystem defekt ist.

Aufgabe 5.3-4 RAID-Folgefehler

Leider trifft eine wichtige Voraussetzung für die Berechnung der Ausfälle meist nicht zu: Die einzelnen Platten fallen nicht unabhängig voneinander aus.

Angenommen, Sie haben einen RAID5-Verbund von drei Platten D1, D2 und D3, wobei D1 und D2 im selben Schacht stecken. Dies bedeutet, dass neben den Einzelausfallwahrscheinlichkeiten P1, P2 und P3 auch eine bedingte Wahrscheinlichkeit Pa existiert, mit der nach dem Ausfall einer Platte kurz darauf die andere Platte auch ausfällt, etwa durch Überhitzung.

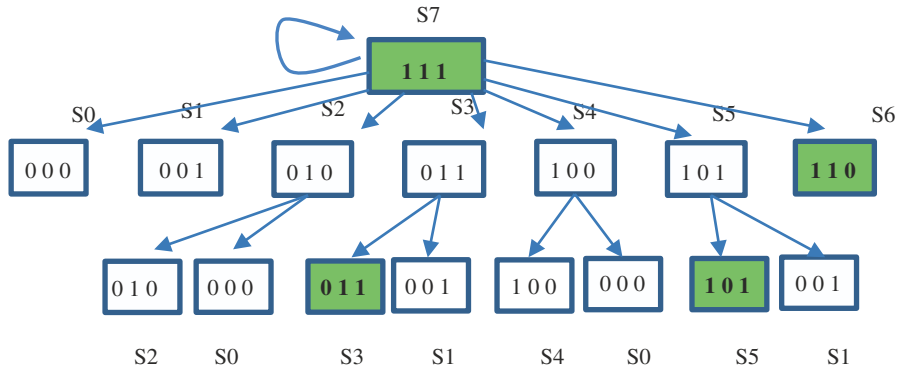
Wie hoch ist die Ausfallwahrscheinlichkeit P des Gesamtsystems als Funktion von P1, P2, P3 und Pa?

Bei dieser Aufgabe stellen wir zuerst die Tabelle auf mit allen Ausfallwahrscheinlichkeiten, notiert mit (0 = defekt, 1 = intakt)

Zustand	D1 D2 D3	Unabhäng. Auftrittswahrsch.
S0	0 0 0	$P1 \cdot P2 \cdot P3$
S1	0 0 1	$P1 \cdot P2 \cdot (1 - P3)$
S2	0 1 0	$P1 \cdot (1 - P2) \cdot P3$
S3	0 1 1	$P1 \cdot (1 - P2) \cdot (1 - P3)$
S4	1 0 0	$(1 - P1) \cdot P2 \cdot P3$
S5	1 0 1	$(1 - P1) \cdot P2 \cdot (1 - P3)$
S6	1 1 0	$(1 - P1) \cdot (1 - P2) \cdot P3$
S7	1 1 1	$(1 - P1) \cdot (1 - P2) \cdot (1 - P3)$

Ein RAID5-System verträgt nur den Ausfall von einer Platte. Fallen zwei davon aus, so ist das Gesamtsystem defekt. Also ist die Ausfallwahrscheinlichkeit $P(\text{Ausfall}) = P(S0 \text{ oder } S1 \text{ oder } S2 \text{ oder } S4)$.

Die Auftrittswahrscheinlichkeiten der Zustände in der Tabelle sind allerdings nur die initialen Ausfallwahrscheinlichkeiten. Dazu kommen noch die bedingten Ausfälle mit der Wahrscheinlichkeit P_a , die das System in einen anderen Zustand überführen. Dies können wir uns mit Hilfe eines Zustandsübergangsgraphen verdeutlichen, der eine Baumstruktur hat.



Dabei kann Zustand S7 zuerst in jeden anderen Zustand übergehen, und dann für die Einheiten im gemeinsamen Schacht in einem zweiten Schritt in einen weiteren Zustand übergehen. Am Ende landen alle Pfade wieder in einem bekannten Zustand. Alle intakten Endzustände sind in der Abbildung grün eingefärbt. Also gilt für die wenigen intakten Zustände

$$P(\text{Ausfall}) = 1 - P(\text{OK})$$

$$\begin{aligned} \text{mit } P(\text{OK}) &= P(\text{keine Einheit ausgefallen}) + P(\text{eine Einheit ausgefallen}) \\ &= P(S7) + P(S3)(1 - P_a) + P(S5)(1 - P_a) + P(S6) \end{aligned}$$

$$\text{Also ist } P(\text{Ausfall}) = 1 - (1 - P1)(1 - P2)(1 - P3) - P1(1 - P2)(1 - P3)(1 - P_a) - (1 - P1)P2(1 - P3)(1 - P_a) - (1 - P1)(1 - P2)P3$$

Aufgabe 5.4-1 Treiber

- a) Was ist ein Treiber und was der Unterschied zwischen einer abstrakten Maschine, einem Treiber und einem Gerätetreiber?

Ein Treiber ist eine Zwischenschicht im Betriebssystem zwischen den Anforderungen und der Ein- und Ausgabe. Dies entspricht einer virtuellen Maschine und hat 2 Schnittstellen. Im Gegensatz dazu betrachtet eine abstrakte Maschine ein Gerät von außen und hat deshalb nur eine Schnittstelle. Ein Beispiel dafür ist eine logische Einheit. Ein Gerätetreiber enthält gerätetypische Merkmale und Dienste, im Unterschied etwa zu einem Dateitreiber, der nicht an ein bestimmtes Gerät gebunden ist.

- b) In welchem Systemmodus laufen Treiber? Warum?

Im *kernel mode*, um möglichst rasch und privilegiert auf alle Hardwareadressen und Daten zugreifen zu können.

c) *Recherchieren Sie: Was ist ein FUSE-Treiber?*

FUSE erlaubt es, Dateisystem-Treiber vom *kernel* in den *user mode* zu verlagern. Ein Beispiel dafür wäre *mount*. Zwar benötigt FUSE auf jeden Fall Administrator-Rechte, jedoch benötigen die Geräte, die mit *mount* eingebunden werden, wegen FUSE diese Rechte nicht. Eine Abbildung dazu ist in [https://msdn.microsoft.com/en-us/library/windows/hardware/ff554836\(v6=6 vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/hardware/ff554836(v6=6%20vs.85).aspx) zu finden.

d) *Wie wird der dazu gehörende Treiber bei dem Interrupt eines Geräts ausgewählt?*

Der Interrupt liefert durch Hardware die Interruptnummer (Interruptvektor), der einem Index in einer Interrupt-Tabelle entspricht. In diesem Tabelleneintrag ist die Treiberadresse gespeichert, die dann benutzt wird, um den Treiber anzuspringen.

Aufgabe 5.5-1 I/O-Dienste

Welche Dienste bietet der BS-Kern einem I/O-Subsystem bzw. Geräten? Nennen Sie mindestens zwei.

- **Spooling:** Geräte-Aufträge werden zwischengespeichert, wenn sie nacheinander abgearbeitet werden (z. B. beim Drucker).
- **Buffering:** Daten werden abgespeichert, die zwischen Gerät und Programm übertragen/transferiert werden, um z. B. Geschwindigkeitsunterschiede von Geräte-I/O und CPU auszugleichen. Heutzutage gibt es z. B. im Grafikbereich *double buffer*, die abwechselnd durch das Programm beschrieben und zur Darstellung ausgelesen werden, um eine flackerfreie Darstellung zu gewährleisten.
- **I/O Scheduling:** Das Scheduling optimiert die Abarbeitungsreihenfolge von I/O Anforderungen mit dem Ziel, den Durchsatz zu erhöhen und die durchschnittliche Wartezeit zu verkürzen. Es wird hierzu eine Warteschlange pro Gerät verwendet; die Reihenfolge innerhalb der Warteschlange kann dabei durch den Scheduler geändert werden. Zusätzlich wird eine *device status table* verwaltet, die einen Eintrag pro Gerät enthält und zu der die Warteschlangen verlinkt sind.

Aufgabe 5.5-2 Pufferung

Betrachten Sie das folgende Schichtenmodell:

Dateisystem
Multi-device-Treiber
Gerätetreiber
Controller
Gerät

Welche Vor- und Nachteile ergeben sich, falls Pufferung in eine der Schichten eingeführt wird? Unterscheiden Sie dazu die vier Fälle beim Übergang von einer Schicht zur anderen. Achten Sie insbesondere auf Dateninkonsistenzen bei parallelen Aktionen.

Schichtung	Vorteile	Nachteile
Dateisystem Pufferung Multi-device-Treiber	schneller Zugriff auf Files	Dateninkonsistenzen, wenn zwei Prozesse schreibend/lesend auf dasselbe File zugreifen
Multi device Treiber Pufferung Gerätetreiber	schneller Zugriff auf Blöcke	dito
Gerätetreiber Pufferung Controller	Pufferung von ganzen Spuren ermöglicht schnelleren Zugriff	Dateninkonsistenzen, wenn bei Systemausfall die Puffer nicht zurückgeschrieben wurden
Controller Pufferung Gerät	Transfer über den Sys- tembus wird erhöht (notwendig)	dito und Interleaving
Allgemein	schneller Zugriff auf oft benutzte Objekte (Files, Spuren, Blöcke, Sektoren)	höherer Aufwand bei Verwal- tung, falls mehrere Prozesse auf Objekte zugreifen (s. Cache) → Synchronisierung, Probleme bei Systemabsturz

9.6 Lösungen zu Kap. 6

Aufgabe 6.2-1 Schichtenprotokolle

In vielen Schichtenprotokollen erwartet jede Schicht ihr eigenes Kopfformat am Anfang einer Nachricht. In jeder Schicht wird beim Durchlaufen die Nachricht als Datenpaket betrachtet, das mit einem eigenen Format und einem eigenen Nachrichtenkopf versehen an die nächste Schicht nach unten weitergereicht wird.

Es wäre sicherlich effizienter, wenn es einen einzigen Kopf am Anfang einer Nachricht gäbe, der die gesamte Kontrollinformation enthält, als mehrere Köpfe. Warum wird dies nicht gemacht?

Das Zusammenlegen der Kontrollinformation in einem einzigen Header würde die Unabhängigkeit der Implementierung der einzelnen Schichten im Schichtenmodell untergraben. Welche Schicht soll den Header erzeugen?

Wird der Header auf oberster Schichtebene generiert, so müsste die Applikation wissen, welche Schichten unter ihr existieren, damit sie entsprechend „Platz“ im Header lassen kann. Effektiv existiert somit nur noch eine einzige „Schicht“.

Wird der Header auf unterster Ebene erzeugt, so müssen alle darüber liegenden Schichten ihre Kontrollinformationen an die unterste Schicht weiterleiten, damit diese sie in den Header einbauen kann. Auf der Empfängerseite haben wir dann das Problem,

die jeweiligen Kontrollinformationen wieder den einzelnen Schichten zuzuordnen, wir müssten den Header also segmentieren, für jede Schicht ein Segment. Das bedeutet aber wieder fast dasselbe, wie jede Schicht ihren eigenen Header schreiben zu lassen, nur mit mehr Aufwand.

Aufgabe 6.2-2 Endian

Ein SPARC-Prozessor benutzt 32-Bit-Worte im Format big endian. Falls nun ein SPARC-Prozessor die Ganzzahl 2 an einen 386-Prozessor sendet, der das Format little endian verwendet, welchen Wert sieht der 386-Prozessor (unter Vernachlässigung der Transportschicht)?

$2 = 2^1$ entspricht in binärer Darstellung $0 \dots 0 \mid 0000\ 0010$ (big endian), wobei der Strich die Bytegrenze markiert.

Folglich ergibt sich für little endian : $00000010 \mid 0 \dots 0 = 2^{25}$

Aufgabe 6.2-3 Internetadressierung

Gegeben ist die logische Internetadresse 134.106.21.30

- a) Welche 32-Bit-Zahl ist dies, wenn alle Zahlen innerhalb der logischen Internetadresse die gleiche Anzahl Bits benötigen? Welcher Dezimalzahl entspricht sie?

Bei 8 Bits pro Zahl ergibt sich als Binärzahl

134	.	106	.	21	.	30
1000 0110	.	0110 1010	.	0001 0101	.	0001 1110

wobei die Zahlengrenze jeweils durch einen Punkt markiert wird, der hier mit der Bytegrenze identisch ist. Als Hexzahl ergibt sich 86.6A.15.1E, was der Dezimalzahl 2255099166 entspricht. Selbstverständlich lassen sich hier keine Zahlengrenzen und damit keine Punkte mehr angeben.

- b) Finden Sie heraus, welcher Rechner unter dieser Adresse zu erreichen ist.
Es ist der Rechner karneol.Informatik.Uni-Oldenburg.de
- c) Angenommen, die angegebene 32-Bit-Zahl sei im big endian-Format dargestellt. Welche Dezimalzahl ergibt sich, wenn das little endian-Format benutzt wird?
 $30.21.106.134 = 504720006$
- d) Welche Portnummern verwenden folgende Servicedienste ftp, telnet und talk auf Ihren Bereichsrechnern? (Schauen Sie dazu in der Datei/etc/services nach!)
ftp: 21, telnet: 23, talk: 517

Aufgabe 6.2-4 Server und Adressierung im Netz

Recherchieren Sie: Was ist ein DNS-Server und was im Gegensatz dazu ein DHCP-Server, und wofür werden sie benötigt?

Ein DNS-Server (*Domain Name System*) wird benötigt, um die eingegebenen URLs den IP-Adressen zuzuordnen, über den die jeweilige Seite zu erreichen ist. Gibt man z. B. „siemens.com.uk“ in die URL-Leiste des Browsers ein, so überprüft der zuständige, im Netzwerk bekannte DNS-Server zunächst, ob er die zur URL gehörende IP-Adresse bereits im Cache hat. Falls dies nicht ist, löst er die URL von rechts nach links auf. Dazu schickt er die URL-Anfrage zuerst zum Name Server des Root Server „uk“, dann zu „com“, usw., bis die korrekte IP-Adresse ermittelt ist.

Ein DHCP-Server (*Dynamic Host Configuration Protocol*) dagegen wickelt ein Protokoll ab, bei dem einem neuen Gerät in einem lokalen Netzwerk automatisch eine IP-Adresse im lokalen Netzwerk zugewiesen wird, wobei diese Adresse nicht im Konflikt zu bereits existierenden stehen darf. Wird dieses Protokoll bei einem neuen Gerät ausgeschaltet, so muss man stattdessen eine feste IP-Adresse einstellen, die sich von allen anderen unterscheidet. Ist dies nicht der Falle, so können beide Geräte entweder gar nicht oder nur fehlerhaft arbeiten.

Aufgabe 6.2-5 Remote Procedure Calls

- a) *Erklären Sie: Was ist ein RPC? Wann ist die Verwendung von RPCs sinnvoll bzw. wann ist es sinnvoller, auf sie zu verzichten?*

Ein RPC (*Remote Procedure Call*) ermöglicht es einem Client, Funktionen auf einem Server aufzurufen und das Ergebnis abzufragen. Für den laufenden Prozess ist die externe Verarbeitung hierbei nicht sichtbar. Ein wichtiges Problem für RPCs ist die benötigte Zeit für die Kommunikation über das Netzwerk. Deshalb sollte man RPCs nur verwenden, wenn die benötigte Zeit für das Verschicken, Berechnen auf dem Server und Empfangen des Ergebnisses kürzer ist, als die lokale Berechnung mit dem Client.

Ausnahmen, die für die Nutzung für RPCs sprechen:

- der lokale Client kann das Ergebnis nicht berechnen, z. B. Wettervorhersage.
- wenn auf eine zentral gehaltene Datenbank zugegriffen werden muss.

- b) *Wie werden bei RPCs Parameter und/oder Datenstrukturen übertragen? Wie wird mit diesem Problem in Java umgegangen?*

Mittels *Data Marshalling* werden die Argumente für die Verwendung von RPCs beim Ein-/Auspacken jeweils vorher in das benötigte Format umgewandelt, bevor es verschickt oder benutzt wird.

In Java wird dazu eine sogenannte *Serialisierung* betrieben, also eine Umwandlung in ein neutrales Darstellungsformat.

- c) *In Abschn. 6.2 wurde die Implementierung von RPCs behandelt. Was muss unternommen werden, um „normale“ (UNIX)-RPCs durch Stubprozeduren bzw. Stubprozeduren durch „normale“ RPCs zu ersetzen?*

Bei Stubprozeduren gibt man immer einen gewöhnlichen Funktionsaufruf wie z. B. `write()` an. Der Rechner stellt dann innerhalb der Prozedur fest, ob dies ein

lokaler oder Netzwerkaufruf (RPC) ist. Im ersten Fall wird er sofort ausgeführt, im letzteren Fall müssen die Argumente gepackt werden und der Prozeduraufruf zum entsprechenden Server geschickt werden.

Falls man nun eine Stubprozedur durch einen normalen RPC mittels `callrpc()` ersetzen will, muss man mit dem Prozedurnamen als Argument für den Prozeduraufruf in einer Systemtabelle nachschauen, ob es sich hierbei um eine lokale oder RPC-Prozedur handelt, und ggf. explizit im Programm den entsprechenden Serviceaufruf mit den Argumenten absetzen. Hierbei wird also das Originalprogramm verändert.

Andersherum: Soll ein RPC in einem Programm durch eine Stubprozedur ersetzt werden, so muss an dieser Programmstelle der Wert des Arguments bekannt sein, also der Prozedurname, und dieser Name als Zeichenkette in das Programm geschrieben werden.

Aufgabe 6.3-1 Zugriffssemantiken

- a) *Wollen mehrere Personen gleichzeitig sowohl lesend als auch schreibend auf eine Datei zugreifen, so kann diese je nach verwendeter Zugriffssemantik nach Abschluss der Änderungen unterschiedliche Zustände aufweisen.*

Beschreiben Sie den Zustand einer Datei davon ausgehend, dass zwei Personen diese Datei öffnen, Änderungen vornehmen und die Datei dann schließen, für folgende Zugriffssemantiken:

- i) *die Operationssemantik*
 - ii) *die Sitzungsemantik*
 - iii) *die Transaktionssemantik*
- i) Nach jeder Aktion von A bzw. B wird die Datei sofort verändert. Nach Schließen der Datei ist sie in dem Zustand, der einer bunten Mischung aller Aktionen entspricht, wobei die Reihenfolge stark zufallsabhängig ist.
 - ii) Je nachdem, ob A oder B als letzter abschließt, sind ausschließlich nur die Änderungen entweder von A oder von B enthalten.
 - iii) Je nachdem, ob A oder B zuerst Zugriff hatte, sind ausschließlich nur die Änderungen entweder von A oder von B enthalten. Allerdings hatte im Unterschied zu (ii) nur einer von beiden Zugriff, so dass der andere bemerkt, dass seine Aktionen obsolet sind.
- b) *Microsoft Office Online erlaubt das gemeinsame Arbeiten an Dokumenten. Welcher der bisher kennengelernten Zugriffssemantiken entspricht die Funktionsweise dieser Anwendung? Begründen Sie Ihre Lösung.*

Office 365 erlaubt es, dass Person A in Absatz 1 schreibt, während Person B in Absatz 2 schreibt. Beide Änderungen werden hierbei gemeinsam übernommen. Schreiben beide im gleichen Absatz, wird nur die letzte Änderung übernommen. Damit entspricht es der Operationssemantik.

Aufgabe 6.3-2 Serverzustände

a) *Man unterscheidet zwischen zustandsbehafteten und zustandslosen Servern. Was sind die Vor- und Nachteile dieser beiden Konzepte?*

Zustandsbehaftete Server haben folgende **Vorteile**:

- schneller Datenzugriff nach Initialisierung der Kommunikation durch Bereitstellung von Dateideskriptoren und Puffern.
- Auftragskopien können eliminiert werden, da die Reihenfolge (Nummerierung) der Nachrichten überprüft werden kann
- Sperren von Dateien für die Transaktionssemantik ist möglich.

Nachteile sind:

- Wenn ein Client „abstürzt“ erfährt der Server dies nicht und reserviert Dateideskriptoren und Puffer unnötigerweise (Gefahr des Daten-overflow).
- Versagt der Server selbst, so bemerkt der Client dies nicht und reserviert ebenfalls unnötigerweise Platz und Systemtafeln.
- Es gibt eine maximale Zahl von gleichzeitigen Nutzern

Prinzipiell sind die Vorteile der zustandslosen Server die Nachteile der zustandsbehafteten Server und umgekehrt: der zustandslose Server spart sich die Initialisierung, aber benötigt für die einzelnen Transaktionen länger. Auftragskopien können unnötigerweise erledigt werden und der Dateizugriff ist nicht reglementiert. Dafür bedeutet das Versagen des Servers oder Clients keine Probleme auf der anderen Seite: die Kommunikation vom Client wird solange wiederholt, bis sie erfolgreich war.

b) *Können bei NFS-Servern Verklemmungen auftreten? Begründen Sie Ihre Antwort.*

Ja, Verklemmungen sind möglich. NFS-Server sind zwar fehlertolerant, einen Schutz vor Verklemmungen bietet das jedoch nicht.

Beispiel:

- Prozess 1: Lockt Datei A
- Prozess 2: Lockt Datei B
- Prozess 1: Möchte Datei B – Die ist aber bereits gelockt
- Prozess 2: Möchte Datei A – Die ist aber bereits gelockt

NFS-Protokolle Versionen 2 und 3 unterstützen kein *File Locking*, dafür wird dann NLM (*Network Lock Manager*) als Hilfsprotokoll verwendet: soll eine

Datei gesperrt werden, wird statt eines NFS-RPC einfach ein NLM-RPC dafür abgesetzt.

Quelle: http://docstore.mik.ua/orelly/networking_2ndEd/nfs/ch11_02.htm

NFSv4 dagegen unterstützt das Sperren einer Datei direkt: *Unlike earlier versions, the NFS version 4 protocol supports traditional file access while integrating support for file locking and the MOUNT protocol.*

Quelle: IETF RFC 7530, <https://tools.ietf.org/html/rfc7530>

Aufgabe 6.5-1 Offlinedateien

- a) Geben Sie einen Ordner auf einem beliebigen Gerät via SMB-Protokoll als Netzlaufwerk frei (z. B. als Windows-Dateifreigabe oder unter Linux mit Samba) und speichern Sie eine Textdatei in diesem. Verwenden Sie nun ein anderes Gerät. Dieses sollte Windows als Betriebssystem verwenden und Schreibrechte für das Netzlaufwerk haben. Machen Sie den Ordner des Netzlaufwerkes auf diesem Gerät als Offlinedatei verfügbar. Trennen Sie anschließend die Verbindung zum Netzwerk, ändern Sie den Inhalt der Datei und speichern Sie diese. Stellen Sie anschließend die Netzwerkverbindung wieder her und starten Sie Ihren PC neu.

Wie hat sich der Inhalt der Datei auf dem Netzlaufwerk verändert?

Hinweise:

- Die Verwaltung der Offlinedateien finden Sie unter Systemsteuerung/Alle Systemsteuerungselemente/Synchronisierungszentrum/Offlinedateien verwalten
- Sollte Ihnen nur ein einziger PC zur Verfügung stehen, so können Sie mit Virtual-Box einen zweiten PC simulieren. Hierzu müssen Sie den virtuellen PC mit einer Netzwerkbrücke verbinden. Die zugehörige Option finden Sie unter Ändern/Netzwerk/Angeschlossen an.

Der Inhalt der Datei auf dem Netzlaufwerk sollte nun der zuvor offline erfolgten Änderung entsprechen. Bei jedem Login/Logout (oder auf Wunsch auch manuell) wird der Inhalt der Offlinedateien mit ihrem „Original“ auf dem Server synchronisiert.

- b) Welchem aus diesem Kapitel bekannten Prinzip entspricht das in a) dokumentierte Verhalten bzw. die Funktionsweise von Offlinedateien unter Windows? Erklären Sie die Funktionsweise dieses Prinzips!

Die Funktionsweise von Offlinedateien unter Windows entspricht der eines Schattenservers. Auf dem Client eines Schattenservers werden nur die Dateien und Programme gehalten, die vom Nutzer dort zuletzt benötigt wurden. Wird mehr benötigt, so wird es bei Netzwerkverbindung vom Server geladen. Ein Schattenserver spiegelt also immer nur Teile eines Client wieder, die beim jeweiligen Client relevant sind; der Client hat die Rolle eines Puffers bzw. Cache. Damit lässt sich eine automatische, zentrale Datenhaltung und Wartung (Backup und Versionskontrolle) etablieren.

Aufgabe 6.5-2 Schattenserver und Netzwerkcomputer

- a) Wie funktioniert ein Netzwerk mit Schattenserver (SS), und wie ein Netzwerk mit Netzwerk-Computern NC?

- b) Was unterscheidet einen Client im SS-Netz von einem Client im NC-Netz funktionell, hardware- und softwaremäßig?
- c) Welche Vorteile bietet die jeweilige Konfiguration?
- a) In einem Netzwerk mit Schattenserver enthält der Server alle Dateien, die auch auf den Client-Rechnern sind und bildet damit einen *back-up* aller im Netz befindlichen Systeme. Alle Client-Rechner sind aber autonom und können auch ohne Netzwerk arbeiten.
- Im NC-Netzwerk laden alle NC-Computer Programme und Daten direkt vom Server; ohne Netzwerk können sie nicht funktionieren.
- b) Ein Client-Rechner im SS-Netz ist autonom und ist sowohl hardwaremäßig, programmäßig als auch datenmäßig vollständig ausgestattet. Das Netzwerk dient allein der Aktualisierung der Software und der Daten. Im Unterschied dazu haben die NC-Rechner keine Festplatten etc., da alle Daten und Programme nur auf dem Server gespeichert sind.
- c) Die Vorteile des NC sind leichte Skalierbarkeit, Datenkonsistenz, leichtere Wartbarkeit, Energieeinsparung und preiswerte Clients. Die Vorteile des SS sind auch leichte Skalierbarkeit, Datenkonsistenz, leichtere Wartbarkeit, aber auch Fehlertoleranz und Autonomie bei Netzausfall. Dafür sind sie teurer.

Aufgabe 6.6-1 Middleware

- a) Angenommen, Sie haben 100 verschiedene Programme, die über ein Netzwerk auf einer von fünf verschiedenen Datenbanken arbeiten. Dabei wird jeweils eine bestimmte Netzwerkfunktionalität (Protokoll) von zwei möglichen vorausgesetzt. Wie viele Programmversionen können Sie abdecken, wenn Sie dafür eine Middleware einführen und wie viele Versionen müssen damit nicht neu programmiert werden?
- b) Was sind die Unterschiede und die Gemeinsamkeiten von einem RPC-Aufruf und einer CORBA-Anfrage?
- a) Ohne Middleware gibt es 100 Programme. Mit Middleware haben Sie $100 \cdot 5 \cdot 2 = 1000$ Funktions-Versionen, die möglich sind. Sie ersparen sich also 900 Programmversionen.
- b) **Gemeinsamkeiten:** Ein RPC-Aufruf fragt eine Dienstleistung an, ebenso ein CORBA-Aufruf. Beide nutzen meist dazu die normalen Transportschichten.
- Unterschiede:** Ein RPC-Aufruf kennt genau seinen Server und baut eine Direktverbindung für die Dienstleistung auf. Im Unterschied dazu ist beim CORBA-Aufruf nur der CORBA-Vermittlungsrechner bekannt, der prüft, ob für die Dienstleistung ihm ein Server bekannt ist. Danach vermittelt er den Dienst, führt ihn aber nicht aus.
- CORBA kann für die Anfragen RPCs benutzen; ebenso können RPCs für die Dienstleistung selbst verwendet werden.

9.7 Lösungen zu Kap. 7

Aufgabe 7.5-1 Sicherheit

Ein beliebtes Mittel zum Abhören von Passwörtern besteht in einem Programm, das „login:“ auf den Bildschirm schreibt. Jeder Benutzer, der sich an den Rechner setzt, wird seinen Namen und sein Passwort eintippen. Registriert dieses Programm die Daten, erscheint eine Meldung wie „Passwort falsch, bitte wiederholen Sie“ oder ähnliches, und es beendet sich. So fällt es nicht einmal auf, dass hier ein Passwort gestohlen worden ist. Wie können Sie als Benutzer oder Administrator den Erfolg eines solchen Programms verhindern?

- Ein solches Programm ist sinnvollerweise nur einmal aktiv und geht mit dem Diebstahl des Passwortes zu Ende. Da derartige Programme meist nicht nachprüfen, ob das Passwort auch korrekt ist, können Sie als Benutzer grundsätzlich einen falschen Benutzernamen eingeben und die erneute Abfrage abwarten, die dann vom echten login-Programm stammt.
- Als Administrator können Sie die Systeme statt auf Passwörter direkt auf ein Authentifizierungsprotokoll umstellen. Dies ist leicht mit Chipkarten oder Magnetkarten möglich, wobei anstelle einer Tastatureingabe eine Codekarte eingelesen und überprüft wird. Der Zugriff auf das Kartenlesegerät kann aber für normale Benutzer prinzipiell unterbunden werden, so dass ein normaler Benutzer kein *login*-äquivalentes Programm starten kann. Angriffe über das Netz oder von unberechtigten, internen Personen werden so unterbunden.

Aufgabe 7.5-2 Viren

- a) *Angenommen, Sie bemerken einen Virus auf Ihrem PC. Unter welchen Umständen reicht es nicht aus, die Massenspeicher mit einem Virensuchprogramm zu „reinigen“?*

Außer in Programmen kann ein Virus sich auch im Hauptspeicher verstecken. Wird nun ein Systemstart durchgeführt, so wird zwar ein „reset“-Signal ausgelöst, aber dieses Signal setzt nicht auch den Zustand der RAM-Zellen im Hauptspeicher zurück: Der Virus bleibt auch nach dem Systemstart erhalten. Abhilfe schafft hier das völlige Abschalten des Rechners für einige Minuten.

Andere Möglichkeiten für einen Virus bestehen darin, statt ausführbarer Programme nur binäre, ladbare Bibliotheken zu infizieren, sich selbst zu kodieren und damit unkenntlich zu machen oder durch Hardwaretricks (MMU-Manipulation) sich zu verstecken („stealth“-Viren).

- b) *Recherchieren Sie: Wie kann es sein, dass ein Virus in einem Server vorhanden ist, obwohl er weder im Hauptspeicher noch auf der Festplatte sich als Programm nachweisen lässt? Erklären Sie jede der Möglichkeiten ausführlich.*
- Innerhalb des *network stacks* gibt es reservierte Datenfelder, die nicht benutzt werden. Befindet sich dort ein Virus, so ist er unsichtbar und kann über Ausnutzen

von anderen Schwachstellen (*exploits*) aktiviert werden. Benutzt er zusätzlich solche Datenfelder in *Headern* von Paketen, so kann er auch unbemerkt nach außen kommunizieren. Wird ein weiterer Server infiziert, so kann auch ein Neustart des Servers den Virus wegen der Reinfektion nicht mehr abschalten, ohne dass der Virus auf dem Massenspeicher enthalten ist.

- Eine weitere Möglichkeit ist die Infektion von Firmware, z. B. beim BIOS oder bei Festplattencontrollern. Wird eine Firmware überschrieben, so kann sich der Virus bereits beim Booten festsetzen. Dies wurde bereits von der NSA genutzt.
- Eine dritte Möglichkeit ist die Infektion bereits bei der Chipherstellung. Wird ein Chipdesign unauffällig ohne Wissen des Herstellers modifiziert (etwa durch Infektion der Bibliotheken), so kann die Hardware später in Rechnern ausgenutzt werden.
- Eine weitere Möglichkeit besteht darin, in einer Virtuellen Maschinen-Umgebung, den Virus außerhalb der virtuellen Maschine in den Hypervisor im laufenden Betrieb einzuschleusen und so vor dem Virens Scanner innerhalb der VM zu verbergen.
- Ebenso alle „Ring-3- rootkits“, siehe <http://invisiblethingslab.com/resources/bh09usa/Ring%20-3%20Rootkits.pdf>.

Aufgabe 7.5-3 Anti-Virenprogramme

a) Was sind die gängigen Strategien, die Antivirenprogramme zum Erkennen von Viren und Ähnlichem nutzen? Wie funktionieren diese und wann versagen sie?

- **CodeEmulation**

Ein verdächtiges Programm wird in einer *Sandbox* ausgeführt und das Verhalten analysiert.

- **Verhaltensanalyse** (Echtzeitüberwachung)

Das Verhalten eines Programmes wird überwacht und bei einer bestimmten Menge/Intensität an verdächtigen Aktionen Alarm geschlagen.

- **Virensignatur**

Ein verdächtiges Programm wird mit den Virensignaturen (Codeabschnitten) aus einer aktuellen Datenbank verglichen.

- **Heuristik**

Erkennung noch unbekannter Viren mit Hilfe von Heuristiken, etwa der Frequenz bestimmter Codefolgen

- **Cloud-Technik**

Analyse basierend auf Hash-Werten der Programme, die sich auf den Servern der Anti-Virenherstellern befinden.

Diese Techniken versagen, wenn es sich um einen neuen Virus handelt (keine ex. Signatur) oder eine komplette neue Angriffsart verwendet wird, z. B. Underflow in JPG-Dateien, Aktionscode in Flash-Dateien oder Zero-day *exploits* des Betriebssystems. Hier sind sog. „generische“ Signaturen wichtig, die nur allgemeine Virencharakteristiken widerspiegeln.

- b) *Beurteilen Sie die Aussage „Je mehr verschiedene Antiviren Programme ich auf meinem Computer installiert habe, desto sicherer wird dieser.“*

Die Aussage ist falsch. Verschiedene Antivirenprogramme können sich gegenseitig behindern: Die Viren-Signaturen der anderen Programme können als Virus erkannt werden oder das Verhalten des einen Antivirenprogrammes selbst kann verdächtig auf das andere wirken. Neben Einschränkungen der Funktionalität der Programme kann es auch das ganze System instabil werden. Der größte Nachteil aber ist die starke Systembelastung, ohne dabei eine höhere Sicherheit zu bieten.

Aufgabe 7.7-1 ACL und Rollen

- a) *Wie funktionieren access control lists ACL und wie das capability-oriented model COM ?*
- b) *Was sind die Vorteile von ACL und von COM?*
- c) *Welche Vorteile bietet darüber hinaus das Rollen-Modell?*
- a) Für jedes Objekt gibt es eine ACL, in der die Zugriffsrechte einzelner Benutzer und Benutzergruppen stehen. Jede(r) Nutzer/in hat auch eine COM-Liste, in der für ihn/sie alle Berechtigungen aller Objekte aufgeführt sind.
- b) Damit ist es möglich, bei jeder ACL sehr feine und genaue Zugriffsrechte aufzuführen. Dies gilt auch für COM. Der Unterschied liegt im Speicherort und in der Zugriffsgeschwindigkeit: Gibt es sehr viele Objekte, so ist es einfacher, sie in einer COM des Nutzers aufzuführen, als bei jedem Objekt eine eigene ACL zu speichern.
- c) Das Rollenmodell biete darüber hinaus die Möglichkeit, die Zugriffsbeschränkungen von einem konkreten Nutzer zu abstrahieren und sie nur einmal für einen generischen Benutzer und seine möglichen Arbeitsaktivitäten festzulegen. Zum einen spart man sich so, gleiche Rechte auf die COM von Nutzern mit gleichem Arbeitsauftrag zu kopieren, zum anderen ist es auch leichter, die Rechte zu ändern, wenn der Arbeitsauftrag sich ändert.

Aufgabe 7.11-1 Firewall

Was sind die Nachteile eines firewall-Systems, das alle Datenpakete untersucht?

Ein firewall-System, das alle Datenpakete und nicht nur die aufgerufenen Programme auf Zulässigkeit untersucht, stellt einen Flaschenhals bezüglich der Systemleistung dar. Ist der Benutzer auf starken Datendurchsatz des Netzes angewiesen, so muss eine sehr schnelle, teure Maschine dafür beschafft werden. Aus diesem Grund verzichten viele Administratoren auf firewall-Rechner und schließen nur unkritische Dienste (E-Mail etc.) an das Internet an.

Aufgabe 7.11-2 Anonymität im Netz

a) *Recherchieren Sie und nennen Sie mindestens vier verschiedene Kriterien, anhand derer man beim Surfen im Internet identifiziert werden kann. Erklären Sie, wie bzw. warum anhand dieser Kriterien eine Identifikation des Nutzers möglich ist.*

- **IP-Adresse:** Die Kommunikation im Internet basiert auf dem Austausch von Datenpaketen. Diese werden mit den IP-Adressen von Sender und Empfänger versehen. Diese Datenpakete können während der Wegfindung angefangen und einem Nutzer zugeordnet werden.
- **Cookies:** Datensätze, auf dem Rechner des Nutzers vom Browser gespeichert. Auf diese sind viele personalisierte Dienste (z. B. Webshops) angewiesen. Man kann sie einsehen und löschen.
- **aktive Inhalte:** Ist z. B. Flash, Java Script, ... aktiv? Diese können auch auf den lokalen Rechner zugreifen.
- **Super Cookies** (Flash Cookies, Storage Cookies): Wie normale Cookies. Dies sind Datensätze, die sich auf dem Rechner befinden, jedoch nicht so einfach zu löschen sind und teilweise browserübergreifend arbeiten.
- **Browser-Fingerabdruck:** Cookie-Akzeptanz, Bildschirmauflösung, Zeitzone und die installierten Browser-Plugins, Betriebssystem, CPU-Nummer, installierte Schriftarten ...
- **„Der Versuch Anonymisierung selbst“** Da der Versuch der Anonymisierung ein extrem untypisches Verhalten darstellt, verhält man sich damit auffällig. Angreifer/Überwacher können deswegen der Meinung sein, dass die entsprechende Person etwas zu verbergen hat und entsprechende Gegenmaßnahmen einleiten.

Quellen:

- <http://www.elektronik-kompodium.de/sites/net/1809161.htm>
- <http://www.golem.de/1005/75176.html>

b) *Recherchieren Sie und erklären Sie Gegenmaßnahmen, um eine Identifikation anhand der unter a) genannten Kriterien zu verhindern oder zu erschweren.*

- **IP-Adresse:** IP-Adresse verschleiern (Proxy, VPN-Gateway, Tor-Netzwerk, JonDonym).
- **Cookies:** Nach Möglichkeit gar nicht erst akzeptieren und regelmäßig löschen.
- **aktive Inhalte:** Nach Möglichkeit deaktivieren, schränkt aber Nutzbarkeit von Webseiten extrem ein.
- **Super Cookies** (Flash Cookies, Storage Cookies): Nach Möglichkeit gar nicht erst akzeptieren und regelmäßig löschen.
- **Browser-Fingerabdruck:** Seltene Konfigurationen vermeiden, besser anonymisierten Browser verwenden.
- **„Der Versuch Anonymisierung selbst“:** Webseiten von entsprechenden Anbietern von Anonymisierungssoftware erst unter Verwendung solcher Software besuchen. Also: Erst Tor-Browser installieren, dann mittel Tor-Netzwerk weitere Software suchen und installieren.

Aufgabe 7.11-3 Recherchieren Sie: Netzattacken

- a) Erklären sie, worum es sich bei einem so genannten *Honey Pot* handelt und wie ein solcher funktioniert.

Ein *honey pot* (Honigtopf) ist ein Computer (oder Simulation eines solchen) in einem Netzwerk, der als eine Art Köder für Angreifer dient. Ein Beispiel dafür ist ein Windows 2000-server ohne *service packs*, auf dem zusätzlich eine Spezialsoftware zur Überwachung installiert ist.

Honey pots haben an sich keinerlei Funktion, so dass jede Art von Datenverkehr als möglicher Angriff gewertet wird und geheim protokolliert wird. Auf diese Art lassen sich viele Informationen über Angreiferprogramme sammeln.

Ebenfalls kann er von dem besser geschützten realen Netzwerk ablenken.

Auch gibt es:

Honey nets, ein Netzwerk aus *honey pots*.

Honey clients, Rechner, die einen Nutzer simulieren (z. B. Webseiten aufrufen) und Server auf infizierte Webseiten testen.

- b) Wobei handelt es sich bei den so genannten *DoS Attacks*? Wie funktionieren diese und wie kann man sich gegen sie schützen?

Bei einer *DoS* (*Denial of Service*) handelt es sich um einen Angriff, der darauf abzielt, das Ziel betriebsunfähig zu machen. Dazu wird eine große Anzahl an Anfragen an das Ziel gesendet (z. B. Webseitenaufrufe). Dabei kommen die Anfragen schneller als der Server diese abarbeiten kann. So ist er nicht mehr dazu in der Lage, seine normalen Tätigkeiten auszuführen. Beispielsweise ist dann ein online-Shop nicht mehr für alle anderen Kunden erreichbar.

Wird die Überlastung von einer größeren Anzahl anderer Systeme verursacht, so spricht man von *DDoS* (*Distributed Denial of Service*).

Als Gegenmaßnahme bietet sich unter anderem an:

- Sperrliste für angreifende IP-Adressen
- Rate Limiting
- Automatische Server-Lastverteilung auf viele Rechner
- ...

- c) Erklären sie das Prinzip des „*Port Knocking*“.

Port knocking ist ein Verfahren zur Absicherung einzelner Serverdienste vor unbefugten Zugriffen. Dazu werden bestimmte Ports anfangs von der Firewall blockiert. Ankommende Daten-Pakete werden überprüft. Sollten diese einen bestimmten Inhalt aufweisen („Klopfschlag“) so wird der entsprechende Port dann geöffnet.

Aufgabe 7.11-4 Kerberos

- a) Was ist der Unterschied zwischen Autorisierung und Authentifikation?

Bei der Authentifikation wird die *Identität* des Benutzers geprüft und nachgesehen, ob er registriert ist, und wenn ja, mit welchen Rechten. Erst danach können

mit der Autorisierung die *Rechte* eines Benutzers oder Programms festgelegt und geprüft werden. Dies kann anhand von Listen (ACL) oder anderer Statusinformation geschehen.

- b) *Wozu werden in Kerberos zwei Schlüssel S_1 und S_2 statt einem benötigt? Was ist der Unterschied zwischen den Schlüsseln?*

Die beiden Schlüssel dienen zwei Zwecken: der Authentifizierung des Benutzers und der Autorisierung der Programmaktionen und Dateizugriffen. Der erste Schlüssel ist ein Langzeitschlüssel für die gesamte Sitzung und dient nur dazu, die Grundrechte des Programms bzw. der Person festzulegen. Der zweite ist ein Kurzzeitschlüssel und ist nur für die aktuelle Transaktion gültig. Mit dem ersten Schlüssel kann der zweite angefordert werden.

9.8 Lösungen zu Kap. 8

Aufgabe 8.2-1 Benutzerschnittstelle und visuelle Programmierung

Wenn der zentrale Lösungsmechanismus eines Problems in analoger Form in der Wirklichkeit bekannt ist, so kann man ihn in der Benutzerschnittstelle grafisch danach modellieren.

- a) *Dabei sollen einfache, konsistente Beziehungen zwischen dem Modell und dem Algorithmus aufgebaut werden. Die Visualisierung soll sehr einfach konzipiert werden, wobei Vieldeutigkeit zu vermeiden ist, z. B. durch Benutzung von Ikonen statt Symbolen.*
- b) *Muss der Benutzer die interne Logik des Programms erlernen, so soll dies als aufbauendes Lernen konzipiert werden, bei dem die Komplexität zunächst versteckt wird und erst im Laufe der Benutzung deutlich wird.*

Verwirklichen Sie diese Grundsätze an einem einfachen Beispiel. Für die Verwaltung von Daten kann man als Programmiermetapher die Vorgänge in einem Lagerhaus verwenden. Hier ist das Liefern, Einsortieren von Paketen, Lagerung, Aussondern etc. gut bekannt und als Referenz beim Benutzer vorhanden.

Konzipieren Sie dazu ein Schema, bei dem die Dateiobjekte (Attribute und Methoden) den visuellen Objekten zugeordnet werden.

Unserem Beispiel entspricht folgendes Schema:

Objekte	
Textdateien	Behälter mit Büchern
Unterverzeichnisse	Regale mit Behältern
visuelle Modell (Ikon)	Behälter mit Bild
Operationen	
Verzeichnis untersuchen	durch das Lager gehen
Datei im Dateisystem abspeichern	Behälter in einem Regal ablegen
Datei löschen	Behälter in den Müll werfen
Datei kopieren	Behälter in eine Kopiermaschine legen

Das aufbauende Lernen kann dabei derart konzipiert werden, dass die Eigenschaften der Objekte (Behälter, Lager, Bücher) erst bei einem Doppelklick auf das Objekt angezeigt und interaktiv geändert werden können.

Aufgabe 8.2-2 Color Lookup Table

- a) *Angenommen, wir beschreiben für eine anspruchsvolle Grafik die Farbe eines Pixels direkt ohne CLUT mit 24-Bit-Farbtiefe. Wie groß muss der Bildwiederholtspeicher für ein 1024×768 großes Bild mindestens sein? Wie groß, wenn er drei Ebenen davon abspeichern soll?*

Das Bild enthält $1024 \times 768 = 786.432$ Bildpunkte je 24 Bit und damit insgesamt $786.432 \times 24 = 18.874.368$ Bits oder $2.359.296$ Bytes = 2,3 MB.

Möchten wir für bewegte Grafik Vordergrund, Hintergrund und Figuren in drei Ebenen extra abspeichern, so ist die dreifache Menge, also 6,9 MB, nötig.

- b) *Wie groß muss der Bildwiederholtspeicher für das Bild mindestens sein, wenn gleichzeitig $65.536 = 16$ Bit Farben sichtbar sein sollen und eine einstufige Umsetzung mit einer CLUT möglich ist?*

Eine einstufige Umsetzung erfordert bei 24-Bit-Farbtiefe und $65.536 = 2^{16}$ möglichen Farben eine CLUT der Länge 3 Byte $\times$ 65.536 Einträge = 196.608 Byte = 196 KB.

Zur Adressierung einer der 2^{16} Farben werden 16 Bit = 2 Byte je Pixel benötigt, so dass für ein solches Bild nur noch $2 \times 786.432 = 1,5$ MB Speicherplatz nötig sind. Brauchen wir noch weniger Farben, die gleichzeitig sichtbar sein sollen, so reduziert sich die erforderliche Speichergröße weiter.

- c) *Verallgemeinern Sie und stellen Sie eine Formel auf für den Speicherbedarf s ohne CLUT bei der Farbtiefe von f Bits pro Pixel und der Bildpixelzahl N sowie dem Speicherbedarf s_{CLUT} mit CLUT, wenn gleichzeitig n Farben sichtbar sein sollen. Bilden Sie das Verhältnis s_{CLUT}/s , und zeigen Sie, dass eine Voraussetzung dafür, dass es kleiner als eins ist, in der Bedingung $N > n$ liegt (was durchaus plausibel erscheint).*

Es gilt $s = f N$

und $s_{\text{CLUT}} = \text{CLUT-Speicherplatz} + \text{Bildspeicher} = nf + N \lg n$.

Also ist $s_{\text{CLUT}}/s = (nf + N \lg n)/(f N) = n/N + (\lg n)/f$.

Da $f, n > 1$ und somit $(\lg n)/f > 0$ folgt, dass $(s_{\text{CLUT}}/s < 1) \Rightarrow (1 > n/N)$ oder $N > n$, q.e.d.

Aufgabe 8.2-3 Benutzeroberflächen in verteilten Systemen

Im obigem Abschnitt wurde das Konzept des Displayserver eingeführt. Als Alternative gibt es dazu Programme, die Texte und Bilder über das Internet laden und anzeigen können, sog. Hypertext-Browser wie Netscape und Internet Explorer. Zusätzliche Programme (sog. cgi-Programme) auf dem Internet-Datenserver sorgen dafür, dass Suchanfragen und ähnliche Aufgaben interaktiv mit dem Browser durchgeführt werden

können. Damit kann man sehr viele Aufgaben, die mit einem *application client/display server*-System durchgeführt werden, auch mit einem derartigen Browser-System durchführen.

a) *Vergleichen Sie die Client-Server-Beziehungen in beiden Systemen.*

Im *application client/display server*-System befindet sich die Grafik der Benutzeroberfläche und ihre direkten Interaktionen und Wechselwirkungen auf dem Serversystem; der Client ist über das Netz getrennt. Die Rechenleistung ist also beim Client; die Anzeige beim Server.

Bei den Browser-Systemen ist dies genau umgekehrt: Der Kunde ist in diesem Fall der anfragende Rechner, auf dem sich der Browser und sein grafisches Fenster befindet; der Server ist die Datenquelle, auf dem sich die Hypertextdokumente und die Auswertungsprogramme befinden. Die Rechenleistung ist also beim Server; die Anzeige beim Client.

b) *Welche Arten der Kommunikation herrschen zwischen Client und Server?*

In [Abschn. 2.4](#) lernten wir zwei verschiedene Kommunikationsarten kennen: verbindungsorientierte und verbindungslose Kommunikation.

Für den Aufbau einer *application client/display server*-Beziehung ist eine Initialisierungssequenz nötig, bei der Fensternummern, Kontext, Speicherbedarf usw. für den Display des Client festgelegt werden. Diese Art der Darstellung benutzt also eine verbindungsorientierte Kommunikation; ein Anhalten des Client führt beim Server zu einer unnötigen Ressourcenbelegung, die nicht ohne Probleme beseitigt werden kann.

Im Gegensatz dazu ist beim Internet Browser jede Anfrage an den Datenserver nach Übermittlung der Daten sofort abgeschlossen; es findet also eine verbindungslose Kommunikation statt. Eine Initialisierung wird nur für ein Sicherheitsprotokoll einer besonders sicheren Sitzungskommunikation benötigt; der Sitzungsschlüssel reserviert aber sehr wenig Ressourcen und besitzt ein Zeitlimit. Ein defekter Client kann also einen solchen Server normalerweise ressourcenmäßig nicht blockieren.

Aufgabe 8.4-1 Client-Server-Architektur

Welche Vor- und Nachteile hat das lokale Konzept von Windows NT gegenüber dem verteilten X-Window-System? Denken Sie dabei an Applikationen wie die Prozesssteuerung eines Stahlwerks mit einem Rechnernetz, die Datenauswertung (data mining) verschiedener Abteilungen über das Intranet eines Betriebs, die Softwareinstallation und Wartung für einen vernetzten Rechnerpool usw.

Generell können wir zwischen der Erzeugung und der Darstellung der Grafikdaten unterscheiden. Beim X-Window-System kann man im Unterschied zu Windows NT den Ort (Rechner) der Anwendung und seiner Erzeugung von *display*-Daten völlig verschieden von dem Ort (Rechner) der Darstellung bzw. Visualisierung zu wählen. Dies bedeutet, dass man beispielsweise die interaktive Beeinflussung der Parameter einer

Prozesssteuerung eines Stahlwerkteils zusammen mit dem eigentlichen Kontrollprogramm in ein Modul packen und auf den jeweiligen Rechner am Kontrollort (z. B. an der Walzstraße) ablaufen lassen kann. Die gesamte Rechenleistung für diese Kontrolle wird also ihrem Kontrollrechner erbracht; der *display*-Server in der Leitzentrale muss nur die jeweiligen Grafikbefehle für das Modul auf dem Überwachungsbildschirm ausführen.

Damit wird die Kontrolle skalierbar: Man kann weitere Prozesse mit ihren Kontrollrechnern in das System integrieren, ohne die Rechenkapazität der Leitzentrale erhöhen zu müssen. Eine solche Modularisierung und Aufteilung begünstigt Fehlersuche, Programmänderungen und heterogenes Systemwachstum.

Im Gegensatz dazu existiert bei Windows NT keine Netzwerk-Grafikschnittstelle. Für jede Kontrollanwendung muss ein eigener Prozess auf dem Leitrechner geschrieben und gestartet werden; die Prozesse kommunizieren mit den Anwendungen über eine selbst geschriebene und damit fehlerträchtige, nichtstandardisierte Datenschnittstelle und benutzen auf dem Leitrechner eigene Grafiksysteme.

Diese Problematik gilt auch für das *data mining* und die Softwarewartung: Alle Stellvertreterprozesse haben in Windows NT eine inhaltlich bestimmte, sehr verschieden ausgeprägte Datenkopplung zu den Anwenderprozessen und keine normierte Schnittstelle.

Literatur

Dal Cin M.: Grundlagen der systemnahen Programmierung. Teubner Verlag, Stuttgart 1988

Lamport L.: A New Solution of Diskstra's Concurrent Programming Problem. Commun. of the ACM 17, 453–455 (1974)

Quade J., Kunst E.: Linux-Treiber entwickeln, dpunkt.verlag Heidelberg, 4.Auflage 2016

Tanenbaum, A.: Verteilte Betriebssysteme. Prentice Hall Verlag, München

Inhaltsverzeichnis

10.1 Modellierung von Thrashing 465

10.1 Modellierung von Thrashing

In [Abschn. 3.3](#) zeigten wir informativ, dass der thrashing-Effekt eintritt, wenn die Seitenwechselzeit größer wird als die Job-Bearbeitungszeit. Wann tritt nun dieser Effekt genau ein? Wann ist $t_w = t_s$? Dazu modellieren wir das System mit Wahrscheinlichkeiten. Mit der mittleren Zeitdauer t_T und der Wahrscheinlichkeit ρ für das Austauschen von Seiten ist

$$t_w = \rho t_T \tag{10.1}$$

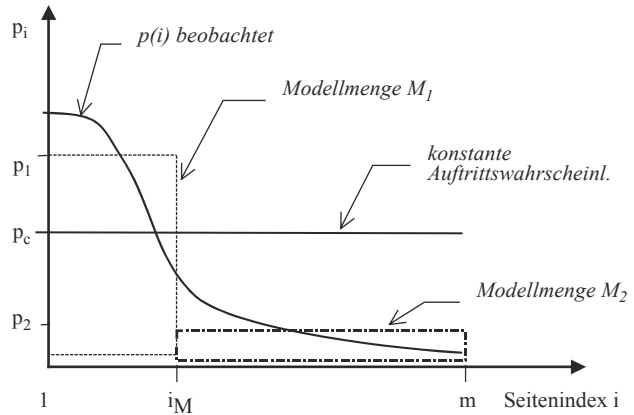
Wie erhalten wir nun aber den „Seitenaustauschgrad“ ρ ?

Angenommen, wir ordnen alle Seitenindizes so, dass die am häufigsten referierten Seiten den kleinsten Index erhalten. Dividieren wir bei jeder Seite die Anzahl der Referenzen durch die Gesamtanzahl aller Seitenreferenzen während der Programmausführung für einen bestimmten Zeitpunkt, so erhalten wir für jede Seite i eine **Referenzwahrscheinlichkeit** p_i . In [Abb. 10.1](#) ist dies aufgetragen.

Die Summe aller Seitenwahrscheinlichkeiten, also das Integral über die Fläche unter der Kurve in [Abb. 10.1](#), muss dabei natürlich eins ergeben.

Für den Seitenaustausch ist nun entscheidend, dass die Funktion p_i nicht konstant ist, also etwa alle Seiten gleichwahrscheinlich referiert werden (Gerade $p_i = p_c$ in [Abb. 10.1](#)), sondern die Adressreferenzen im Code meist lokal erfolgen und damit zu einem Zeitpunkt immer nur auf wenige, bestimmte Seiten zugegriffen wird. Diese **Lokalitätseigenschaft** führt zu einem starken Abfall der Funktion p_i bei größeren Indizes und bildet die

Abb. 10.1 Wahrscheinlichkeitsverteilung der Seitenreferenzen



Implementationsgrundlage des virtuellen Speichersmodells: Wären alle möglichen virtuellen Seiten des Prozesses gleichwahrscheinlich referenziert, so wären mehrstufige Tabellen, die auf der Redundanz nicht-existenter Seiten beruhen, und effiziente Assoziativspeicher nicht möglich.

Für die folgenden Betrachtungen modellieren wir die möglichen Seiten eines Prozesses mit zwei Mengen: Die Seiten mit Index $i \leq i_M$, die mit einer konstanten (mittleren) Wahrscheinlichkeit p_1 referiert werden, seien die Menge M_1 der stark referierten Seiten des working set, und die restlichen Seiten mit Index $i_M < i \leq m$ mit konstanter Wahrscheinlichkeit p_2 gehören der Menge M_2 zu, siehe Abb. 10.1.

Die Wahrscheinlichkeit ρ , eine Seite auszutauschen, wird stark von dem **relativen Speicherangebot** $\sigma = k/m$ pro Prozess, dem Verhältnis von k für den Prozess nutzbaren Seiten des Hauptspeichers zu den m möglichen Seiten des Prozesses, bestimmt. Wären alle Seiten gleichwahrscheinlich referenziert, so wäre

$$\rho = \frac{\text{Zahl der zu ersetzenden Seiten}}{\text{Zahl der gesamten Seiten}} = \frac{m - k}{m} = 1 - \sigma$$

Dies ist aber glücklicherweise nicht so, da sonst die virtuelle Adressenumsetzung durch die damit notwendigen riesigen Tabellen nicht praktikabel wäre und ein Tabellencache sinnlos wäre.

Die Wahrscheinlichkeit $\alpha = 1 - \rho$, dass eine gewünschte Seite i in den k Seiten des Speichers vorhanden ist, hängt von ihrer Referenzwahrscheinlichkeit p_i ab. Wenden wir eine Seitenersetzungsstrategie an, die die am häufigsten referenzierten Seiten am wenigsten ersetzt, so sind Referenzwahrscheinlichkeit $p(i)$ und Anwesenheitswahrscheinlichkeit α miteinander gekoppelt. Die Wahrscheinlichkeit dafür, dass auch eine Seite mit Index k anwesend ist, ist identisch mit der Wahrscheinlichkeit dafür, dass alle Seiten mit einem Index $i \leq k$ anwesend sind. Für unser Modell aus zwei Seitenmengen erhalten wir also

$$\alpha_1(k) = P(i \leq k) = \sum_{i=1}^k p_1 = kp_1 \quad \text{für } 1 \leq k \leq i_M \quad (10.2)$$

$$\alpha_2(k) = \alpha_1(i_M) + \sum_{i=i_M+1}^k p_2 = i_M p_1 + (k - i_M) p_2 \quad \text{für } i_M \leq k \leq m \quad (10.3)$$

Wir bezeichnen die Wahrscheinlichkeit, dass eine Referenz zu der bevorzugten Menge M_1 mit $i \leq i_M$ geht, mit

$$v := P(1 \leq i \leq i_M) \quad \text{so dass } P(i_M < i \leq m) = 1 - v \text{ gilt.}$$

Dann ist

$$v = \alpha_1(i_M) = i_M p_1 \Leftrightarrow p_1 = \frac{v}{i_M}$$

$$1 - v = \sum_{i=i_M+1}^m p_2 = (m - i_M) p_2 \Leftrightarrow p_2 = \frac{1 - v}{m - i_M}$$

Man beachte, dass durch die Definition von M_1 beim Übergang i_M bzw. σ_T gerade die Wahrscheinlichkeit $\alpha_1(i_M) = v$ mit der tatsächlichen Wahrscheinlichkeit $\Sigma p(i)$ der beobachteten Seitenreferenzen übereinstimmt.

Nun können wir den unbekannten Seitentauschgrad ρ aus Gl. (10.1) mit Hilfe der Beziehungen (10.2) und (10.3) näher fassen. Seine Funktion setzt sich zusammen aus den zwei Funktionen ρ_1 und ρ_2 für die beiden Seitenmengen

$$\begin{aligned} \rho_1 &= 1 - \alpha_1 = 1 - kp_1 = 1 - kv/i_M & \text{für } 1 \leq k \leq i_M \\ \rho_2 &= 1 - \alpha_2 = 1 - v - (1 - v)(k - i_M)/(m - i_M) & \text{für } i_M \leq k \leq m \end{aligned}$$

Gehen wir von dem absoluten Speicherangebot k auf das relative Speicherangebot pro Prozess σ über, indem wir die Gleichungen mit $1/m$ erweitern und die Relationen auf allen Seiten mit $1/m$ multiplizieren, so ergibt sich mit der Bezeichnung $\sigma_T := i_M/m$

$$\rho_1 = 1 - \frac{kv}{i_M} = 1 - \frac{k \cdot m \cdot v}{m \cdot i_M} = 1 - \frac{v\sigma}{\sigma_T} \quad \text{für } \frac{1}{m} \leq \sigma = \frac{k}{m} \leq \sigma_T \quad (10.4)$$

$$\rho_2 = (1 - v) \left(1 - \frac{k/m - i_M/m}{1 - i_M/m} \right) = \frac{1 - v}{1 - \sigma_T} (1 - \sigma) \quad \text{für } \sigma_T \leq \sigma \leq 1 \quad (10.5)$$

An dem Übergang der beiden Funktionen bei $\sigma = \sigma_T$ ist $\rho_1(\sigma) = \rho_2(\sigma) \equiv \rho_T$, also $\rho_T = \rho(\sigma_T)$. Der Wert ist dabei

$$\rho_T = \rho_1(\sigma_T) = 1 - v \quad (10.6)$$

Damit haben wir für unser grobes Modell mit der Einteilung der Seiten in nur zwei Mengen eine Beziehung zwischen dem Seitenaustauschgrad ρ und dem Speicherangebot σ gefunden. In Abb. 10.2 ist dies illustriert.

Für gleichwahrscheinliche Seitenreferenzen ist die Funktion $\rho = 1 - \sigma$ als Gerade eingezeichnet. Dabei ist mit der Erweiterung auf $\sigma = 0$ auch der Funktionswert $\rho = 1$ hinzugekommen. Die Beziehungen (10.4) und (10.5) sind ebenfalls als Gerade visualisiert, die in einem Punkt zusammentreffen. Wie man sieht, ist bei Verkleinerung des Seitenangebots σ bis auf das working set nach dem Übergang von ρ_1 auf ρ_2 ein plötzlicher Anstieg der Seitenaustauschaktivität zu beobachten: Der thrashing-Effekt tritt auf. Bei der tatsächlichen Abhängigkeit ρ wäre dies der Punkt der stärksten Krümmung.

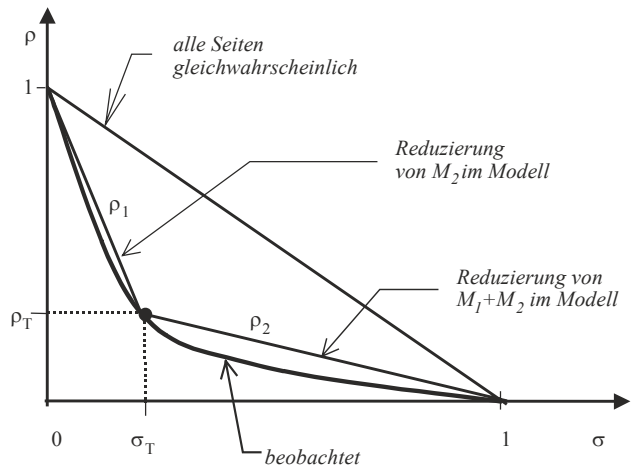
Ist $t_w > t_s$, so bestimmen die Wartezeiten die Gesamtbearbeitungsdauer B_G . Sie erhöht sich bei n gleichartigen Prozessen von $B_G = nB_1$ mit der Anzahl (B_1/t_s) von Wartezeiten auf $B_G = n(B_1/t_s) t_w$. Da wir nur an einer qualitativen Aussage unabhängig von der normalen Bearbeitungszeit B_1 interessiert sind, erhalten wir als relative Bearbeitungsdauer $G = B_G/B_1$ mit (10.1)

$$G = n \quad t_w \leq t_s \text{ proportionaler Bereich} \quad (10.7)$$

$$G = n t_w/t_s = n \rho t_T/t_s \quad t_w \geq t_s \text{ überproportionaler Bereich}$$

mit dem Übergang bei $t_w = t_s$. In diesem Fall ist $\rho t_T = t_s$ mit der Konstanten $\rho = t_s/t_T \equiv \rho_w$. Das entsprechende Speicherangebot sei mit σ_w bezeichnet, so dass $\rho(\sigma_w) = \rho_w$ gilt.

Abb. 10.2 Der Seitenaustauschgrad ρ in Abhängigkeit vom Speicherangebot σ pro Prozess





Die zweite Gleichung wird damit

$$G = \frac{n}{\rho_w} \rho \quad t_w \geq t_s \text{ überproportionaler Bereich} \quad (10.8)$$

Wir können also zwei wichtige Parameter unterscheiden: den Übergang ρ_T zwischen den beiden Seitenmengen M_1 und M_2 und den Übergang ρ_w zwischen der Bearbeitungsdauer t_s und der Wartedauer t_w . Wie hängt nun das Systemverhalten von diesen beiden Parametern ab?

Betrachten wir den Fall $\rho_T \leq \rho_w$ und somit $\sigma_T \geq \sigma_w$. Wir können damit das gesamte Intervall für σ in Abb. 10.2 in zwei Intervalle aufteilen: Intervall (A) und Intervall (B).

Im Bereich (A) ist mit $\sigma_w \leq \sigma$ auch $\rho \leq \rho_w$ und somit $t_w \leq t_s$. Also gilt im Bereich (A) Gl. (10.7)

$$(A) \quad G_A = n \quad \sigma_w \leq \sigma \quad \text{normaler Bereich}$$

und im Bereich (B) mit Gl. (10.8) und (10.4) und der Notation

$$\sigma = \frac{k}{n \cdot m} = \frac{s}{n}$$

für das relative Speicherangebot bei der Speicheranforderung von $n \cdot m$ Seiten durch n Prozesse und dem relativen Gesamtspeicher s

$$(B) \quad G_B = \frac{n}{\rho_w} \rho_1 = \frac{n}{\rho_w} \left(1 - \frac{\sigma \cdot v}{\sigma_T} \right) = \frac{n}{\rho_w} - \frac{sv}{\rho_w \sigma_T} \quad \text{Thrashing – Bereich}$$

Bei welcher Anzahl n_0 von Prozessen tritt nun der Übergang von einfach linear erhöhter Gesamtbearbeitungszeit (A) zu erhöht linearer Zeit (B) auf?

Dies ist der Fall bei

$$G_A(n_0) = G_B(n_0) \Leftrightarrow n_0 = \frac{n_0}{\rho_w} \rho_1 \Leftrightarrow \rho_w = \rho_1 = \left(1 - \frac{(s/n_0)v}{\sigma_T} \right)$$

$$\rho_w \sigma_T = \sigma_T - \frac{sv}{n_0} \Leftrightarrow n_0 = \frac{sv}{\sigma_T(1 - \rho_w)}$$

Beispiel Nichtlineare Auslastung bei geringem Seitenwechsel

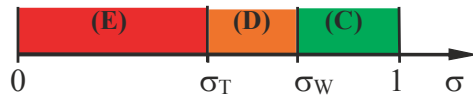
Sei ein System mit vollem Platz für 2 Prozesse ($s = 2$), einem working set, das die Hälfte der Prozessgröße beträgt ($\sigma_T = 0,5$), aber zu 90 % immer referenziert wird ($v = 0,9$) $\Rightarrow \rho_T = 0,1$, gegeben. Da $\rho_T \leq \rho_w$ ist, nehmen wir z. B. $\rho_w = 0,2$ an. Mit diesen Annahmen

errechnet sich $n_0 = 4,5$ und $G_B(n = 6) = 6 \cdot 5 - 2 \cdot 0,9 \cdot 5 \cdot 2 = 12$. In [Abb. 10.3](#) ist dies gezeigt.

Man sieht: Obwohl nur Platz für 2 vollständige Prozesse im Hauptspeicher da ist, können auch ohne Leistungsverlust 4 Prozesse ausgeführt werden. Erhöht man aber die Prozesszahl, z. B. auf $n = 6$ Prozesse, so ist nun eine 2fach überhöhte Last wirksam!

Betrachten wir den anderen Fall $\rho_T \geq \rho_w$ und somit $\sigma_T \leq \sigma_w$. Wir können damit das gesamte Intervall für σ in [Abb. 3.24](#) in drei Intervalle aufteilen: Intervall (C), Intervall (D) und Intervall (E).

Im Bereich (C) ist mit $\sigma_w \leq \sigma$ auch $\rho \leq \rho_w$ und somit $t_w \leq t_s$. Also gilt im Bereich (C) Gl. (3.7)



$$(C) \quad G_C = n \quad \sigma_w \leq \sigma \quad \text{normaler Bereich}$$

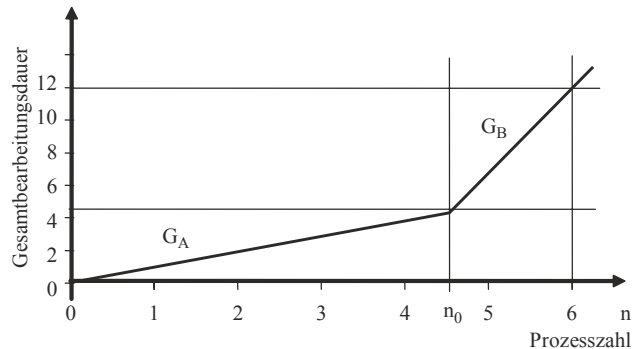
und im Bereich (D) mit einem Speicherangebot kleiner als die Thrashing-Grenze, aber größer als das working set, s. Gl. (10.8) und (10.5)

$$(D) \quad G_D = \frac{n}{\rho_w} \rho_2 = \frac{n}{\rho_w} \frac{1-v}{1-\sigma_T} \left(1 - \frac{s}{n}\right) = c(n-s) \quad \text{für } \sigma_T \leq \sigma \leq \sigma_w$$

$$\text{mit der konstanten } c = \frac{1}{\rho_w} \frac{1-v}{1-\sigma_T} \quad \text{schwaches Thrashing}$$

und für den dritten Bereich mit einem sehr kleinen Speicherangebot $0 \leq \sigma \leq \sigma_T$

Abb. 10.3 Anstieg der Bearbeitungsdauer mit wachsender Prozesszahl bei $\rho_T \leq \rho_w$



$$(E) \quad G_E = \frac{n}{\rho_w} \rho_1 = \frac{n}{\rho_w} \left(1 - \frac{\sigma v}{\sigma_T} \right) = \frac{1}{\rho_w} \left(n - \frac{sv}{\sigma_T} \right) \quad \text{starkes Thrashing}$$

Bei welcher Anzahl n von Prozessen tritt nun der Übergang von einfach linear erhöhter Gesamtbearbeitungszeit (C) zu erhöht linearen Zeiten (D) und (E) auf?

Dies ist der Fall bei

$$G_C(n_1) = G_D(n_1) \Leftrightarrow n_1 = c(n_1 - s) \Leftrightarrow n_1 = \frac{sc}{c-1}$$

$$G_D(n_2) = G_E(n_2) \text{ bei } \sigma = \sigma_T \Rightarrow \text{Mit } \sigma = \frac{s}{n_2} = \sigma_T \text{ ist } n_2 = \frac{s}{\sigma_T}$$

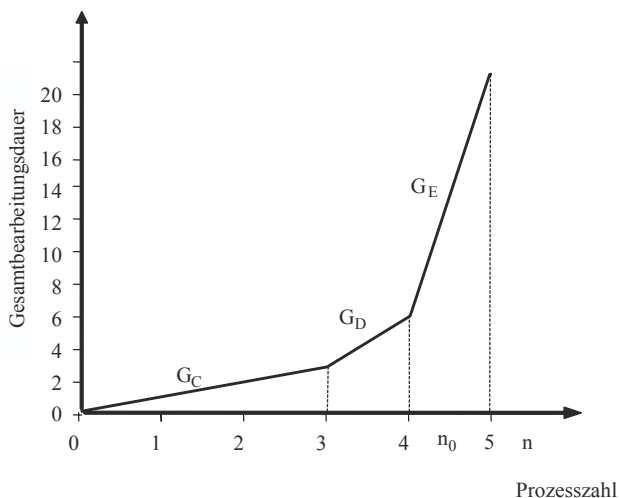
Veranschaulichen wir uns dies wieder an unserem obigen Beispiel.

Beispiel Nichtlineare Auslastung bei hohem Seitenwechsel

Sei $s = 2$, $\sigma_T = 0,5$, $v = 0,9 \Rightarrow \rho_T = 1/10$. Da $\rho_T \geq \rho_w$ ist nehmen wir z. B. $\rho_w = 1/15$ an und damit $c = \frac{15 \cdot 0,1}{0,5} = 3$. Mit diesen Annahmen errechnet sich $n_1 = \frac{2 \cdot 3}{2} = 3$ und $n_2 = \frac{2}{0,5} = 4$ sowie $G_D(n = 4) = 15 \cdot 0,2 \cdot (4 - 2) = 6$ und $G_E(n = 5) = 15 \cdot (5 - 2 \cdot 0,9 \cdot 2) = 21$. In Abb. 10.4 ist der Verlauf der relativen Gesamtbearbeitungsdauern gezeigt.

Bei der Erhöhung der Prozesszahl nimmt die Bearbeitungszeit zunächst nur linear zu; ja, wir können sogar drei Prozesse ohne Probleme laufen lassen, obwohl nur Platz für

Abb. 10.4 Anstieg der Bearbeitungszeit mit wachsender Prozesszahl bei $\rho_T \geq \rho_w$



zwei vollständige Prozesse da ist. Ermöglichen wir aber mit $n = 4$ einen Prozess mehr, so ist bereits das 1,5-fache an Bearbeitungszeit nötig. Mit weiteren Prozessen erfolgt der scharfe Einschnitt dann, wenn auch das working set im Speicher keinen Platz mehr hat. Die Bearbeitungszeit wächst nun mit dem Faktor $1/\rho_w = 15$ rapide an und erreicht schon bei $n = 5$ mehr als das 4-fache der Bearbeitungszeit im Normalzustand!

Weiterführende Literatur

- Achilles, A.: Betriebssysteme; Eine kompakte Einführung mit Linux. Springer Verlag, Heidelberg 2006
- Anderson, T., Dahlin, M.: Operating Systems: Principles and Practice. Recursive Books Ltd., 2nd ed., 2014
- Autenrieth, K., Dappa, H., Grevel, M.: Technik verteilter Betriebssysteme. Hüthig Verlag, Heidelberg 1990
- Bic, L., Shaw, A.C.: Operating Systems Principles. Prentice Hall 2002
- Davis, W.: Operating Systems: a Systematic View. Addison-Wesley, Reading, MA 1992
- Deitel, H., Deitel, P., Choffnes D.: Operating Systems. 3rd ed., Prentice Hall 2004
- Erl T.: Service-Oriented Architecture: A Field Guide to Integrating XML and Web Services, Prentice Hall, PTR 2004
- Flynn, I., McHoes, A.: Understanding Operating Systems. Brooks/Cole Publ., Pacific Grove, CA 1991
- Fortier, P.: Design of Distributed Operating Systems. Intertext Publ., New York 1988
- Garg, R., Verma, G.: Operating Systems: A Modern Approach. Mercury Learning & Information, 2017
- Glatz, E.: Betriebssysteme: Grundlagen, Konzepte, Systemprogrammierung. dpunkt.verlag GmbH; 3. Auflage, 2015
- Goscinski, A.: Distributed Operating Systems. Addison-Wesley, Sydney 1991
- Habermann, A.: Entwurf von Betriebssystemen. Springer-Verlag, Berlin 1981
- Hofmann, F.: Betriebssysteme: Grundkonzepte und Modellvorstellungen Teubner Verlag, Stuttgart 1984
- Koubaa, A.: Robot Operating System (ROS). Springer Verlag 2017
- Krakiwiak, S.: Principles of Operating Systems. MIT Press, Cambridge, MA 1989
- Lister, A., Eager, R.: Fundamentals of Operating Systems. Macmillan, New York 1993
- Maekawa, M., Oldehoeft, A., Oldehoeft, R.: Operating Systems. Benjamin Cummings, Menlo Park, CA 1987
- Mandl, P.: Grundkurs Betriebssysteme. Springer Vieweg; 4. Auflage, 2014
- McHoes, a., Flynn, I.: Understanding Operating Systems. Course Technology, 8th ed., 2017
- Milenkovic, M.: Operating Systems. McGraw-Hill, 2nd ed., 1992
- Mullender S.J., Tanenbaum A.S.: Immediate Files. Software – Practice and Experience 23, 365–368 (1984)
- Nutt, G.: Operating Systems, A Modern Perspective. Addison Wesley Longman, 2nd ed, 2002
- Peterson, J., Silberschatz, A.: Operating System Concepts. Addison-Wesley, 5th ed., 1996
- Pinkert, J., Wear, L.: Operating Systems. Prentice Hall, London 1989

- Richter, L.: Betriebssysteme. Teubner Verlag, Stuttgart 1985
- Schnupp, P.: Standard-Betriebssysteme. Oldenbourg Verlag, München, 1988
- Shay, W.: Introduction to Operating Systems. Harper Collins College, Glenview, IL, 1993
- Siebert, H.-J., Baumgarten, U.: Betriebssysteme: Eine Einführung. Oldenbourg Verlag, München 2001
- Silberschatz, A.: Operating System Concepts. John Wiley & Sons Inc; 10th ed. 2017
- Solomon d., Russinovich M.: Inside Microsoft Windows 2000, Microsoft Press, Redmond, WA, 3rd ed., 2000
- Stallings, W.: Operating Systems: Internals and Design Principles. Prentice Hall, Upper Saddle River, NJ, 8th ed, 2014
- Switzer, R.: Operating Systems: a Practical Approach. Prentice Hall, Englewood Cliffs, NJ 1993.
- Tanenbaum, A.: Verteilte Betriebssysteme. Prentice Hall Verlag, München 1995
- Tanenbaum, A., Bos, H.: Moderne Betriebssysteme. Pearson Studium, 4. Aufl., München 2016
- Tanenbaum, A., Bos, H.: Modern Operating Systems. Pearson Int. Edition, 4th ed, 2014
- Theaker, C., Brookes, G.: Concepts of Operating Systems. Macmillan, New York 1993
- Weck, G.: Prinzipien und Realisierung von Betriebssystemen. Teubner Verlag, Stuttgart 1989
- Wettstein, H.: Systemarchitektur. Hanser Verlag, München 1993

Stichwortverzeichnis

10BaseT, 271

A

ABI (application binary interface), 179
Access Control Lists (ACL), 338, 343–344
access violation, 172
ACL (Access Control Lists), 338, 343–344
Active Directory Service (ADS), 277
adaptive Prozessorzuteilung, 44
Adressbegrenzung, 133
Adressregister, 236
ADS (Active Directory Service), 277
Advanced Host Controller Interface (AHCI), 238
Aesthetics, 355
AHCI (Advanced Host Controller Interface), 238
Ankunftsrate, 61
ANSI, 357
Antikörper, 331
Antwortzeit, 41
Anwendungsschicht, 270
API (application programming interface), 34, 356, 375, 377–378
application binary interface, 179
application programming interface (API), 34, 356, 375, 377–378
Arbeitsmenge, 149, 156–157
ASCII, 356–358
Assoziativspeicher, 136–137, 466
asymmetrischer Pool, 14
asymmetrisches Multiprocessing, 59
asynchroner Verbindungsaufbau, 239, 259
atomare Aktion, 72–73, 176
Attribute, 176
attribute caching, 378
Ausfalltoleranz, 60, 242–243

Ausführungszeit, 40
Ausgaberegister, 240
Auslastung, 40
Auslastungsmonitor, 52
Authentication Protocol, 348
Authentication Server, 346
Authentifikation, 342, 348
Autorisierung, 342
average seek time, 233

B

bad cluster mapping, 228
batching, 379
Bedienrate, 61–62
Bedienzeit, 40
Befehlsregister, 236
Beladys Anomalie, 152
Belegungstabelle, 132
benchmark, 221–222
Benutzeranpassung, 353
Benutzererkennung, 341
Benutzeroberfläche, 351
Benutzerschnittstelle, 4
Bereit-Liste, 29, 31, 35, 41, 45, 64, 68
Bereit-Zustand, 29
BestFit, 125, 127
Betriebsmittel, 28
Betriebssystemkern, 4
binary translation, 20
block device, 252, 258
blocked, 30
bootstrap, 5, 184
btrfs (B-tree file system), 225
buddy, 126–127
buffer overflow, 340
busy wait, 73

C

C-library, 159
Cache, 136–137, 140, 143, 167–169, 237, 257–258, 427
call gate, 3
Capability-oriented System, 339
child-widget, 373
Client, 9, 25
Client-Server, 25, 266, 268, 363, 365, 368–369
clipping, 370
clock, 146
Color Lookup Table, 361
composite-widget, 373
computing server, 266, 268
context switch, 30, 41
copy back, 168–169
copy on write, 158, 162, 172
Coroutinen, 35
CPU bursts, 46
critical sections, 68

D

Dämonen, 160
DAP (Directory Access Protocol), 277
data cache, 167
Dateiordner, 177
Dateninkonsistenz, 168
Datenkonsistenz, 176
Datenpuffer, 236
Datex-P, 270
Deferred Procedure Call (DPC), 253
demand paging, 154
device mapper, 225–226
DFS (Distributed File System), 279
Dialogobjekt, 371, 377
Dienst, 264, 267–270, 327, 343–344
Dienstleistung, 11
direct memory access (DMA), 167
directness, 354
directory, 177
Directory Access Protocol (DAP), 277
Directory Information Shadowing Protocol (DISP), 277
directory server, 268
Directory Service Protocol (DSP), 277
Disk-to-Disk-Storage, 14
DISP (Directory Information Shadowing Protocol), 277
Dispatcher, 30, 39, 44

Distributed File System, 279
DMA (direct memory access), 168
DMA-Kanal, 231, 237
DNS (Domain Name Server), 277
domain controller, 343
Domain Name Server (DNS), 277
Domäne, 275
DPC (Deferred Procedure Call), 253
drag-and-drop, 366
dragging, 355
DSP (Directory Service Protocol), 277
dual homed host, 345
Durchsatz, 40
Dynamic Link Library, 159, 162, 378
dynamische Prioritäten, 43
dynamisches Scheduling, 39

E

Earliest Deadline First, 50
earliest scheduling, 55
Echtzeitsystem, 47
Eingaberegister, 240
Einprozessorsystem, 23
ELF (Executable and Linkable Format), 179
Elternprozess, 28, 32
Escape-Sequenzen, 362
ESCD, 231
Ethernet, 271
EUC (Extended-UNIX-Code), 358
Executable and Linkable Format, 179
Explorer, 181
extended markup language (XML), 368
Extension, 178
externer Verschnitt, 128, 130

F

Fähigkeiten, 338
fail-save, 60
fairness, 40
fairness condition, 69, 73
Fairness-Bedingung, 69, 73
Falltür, 5
FCFS (First Come First Serve), 42–43, 45, 62, 256
FDDI, 271
Fehlerkorrektur, 243, 245
fences, 130
fibers, 37
FIFO-Strategie, 146, 161

file descriptor table, 33
file server, 25, 266, 268
File Transfer Protocol, 276
fire wall, 343–345
FirstFit, 125, 127
floppy disk, 232
Flußsteuerung, 240
Fragmentierung, 128, 165
frames, 270
FtDisk, 228, 245–246

G

Gantt-Diagramm, 54
garbage collection, 53, 122, 238
gegenseitiger Ausschluß, 69–70
ghostview, 327
Global Description Table, 166
Graphic Device Driver, 378
Graphic Device Interface, 378
Graphical User Interface Siehe GUI, 356
Gruppenkennung, 341
Gruppenscheduling, 58
Guaranteed Percentage Scheduling, 51
GUI (Graphical User Interface), 356, 366–367

H

Hacker, 324
HAL, 10
hard link, 183
hardware emulation, 19
Hardware-Virtualisierung, 19
Hauptfenster, 375
Hauptspeicher, 122–123, 128–130, 132–134,
139–140, 143–144, 146, 149, 152–153, 156,
158, 161, 164, 167–169, 427, 470
heavyweight threads, 36
hibernate-Modus, 239
HID (Human Interface Device), 252
Hit, 167
Human Interface Device, 252
hyper calls, 18, 20
Hypervisor, 17, 20

I

i-node, 224
I/O Request Package, 227, 253
IaaS (Infrastructure as a Service), 17
idle, 31
Ikon, 352–353

Infrastructure as a Service, 17
Init-Prozess, 32
instruction cache, 167
Interix, 10
interner Verschnitt, 128
Internet, 275
Interrupt Request, 49
Interrupt Service Routine, 49
Interruptroutine, 133
Interruptsystem, 236, 240, 253–254
inverse Seitentabelle, 136
IP (Internet-Protokoll), 270, 272–274, 276
IRP (I/O Request Package), 227, 253–254
ISO-OSI Schichtenmodell, 269

J

Jail, 19
Job, 28

K

KDE (K Desktop Environment), 368
Kerberos, 345–346, 349
kernel mode, 6, 20
kernel-based virtual machine (KVM), 20
Key Distribution Protocol, 347
Kindsprozess, 28, 32
kobject, 225
Kompatibilität, 9
konkurrente Prozesse, 53
Konsole, 379
Kontrollregister, 236, 239
kritischer Abschnitt, 68–71, 73
Kurzzeitscheduler, 47
Kurzzeitscheduling, 40
Kurzzeitschlüssel, 460
KVM (kernel-based virtual machine), 20

L

LAN, 264, 278
Langzeitscheduler, 47
Langzeitscheduling, 40
LAPD (Link Access Procedure for D-channels), 271
Lastenausgleich, 16
latest scheduling, 47, 56
lazy evaluation, 158
Least Laxity First, 50
legacy devices, 231
Legacy-Anwendung, 16

Legacy-Betriebssystem, 16
Leichtgewichtsprozess, 35
lightweight threads, 36
Link Access Procedure for D-channels, 271
Linux, 368
Littlesche Formel, 61
LLC (Logical Link Control), 271
local area network, 264, 278
Local Description Table, 165
Lockvogel, 330
Logical Link Control, 271
Logical Volume Manager, 225
logische Festplatte, 13
logische Maschinen, 12
Lokalitätseigenschaft, 144, 155, 167, 465
look-and-feel, 364, 366, 371, 378–379
LPC (local procedure calls), 9
LRU-Strategie, 146, 148
LVM (Logical Volume Manager), 225

M

Mach, 7
Mach-Kern, 267
magic number, 179
Mailbox, 342
mainframe control unit, 255
Makrovirus, 329
MDI (Multi Document Interface), 376
Mehrbenutzerbetrieb, 28
Mehrprogrammbetrieb, 28
Mehrrechnersystem, 24
Memory Curtaining, 171
memory mapped devices, 138
memory mapped I/O, 230
memory mapping, 258
Memory-Management-Unit, 131
Menü, 353
message, 29
Mikrokern, 8
MIME (Multipurpose Internet Mail Extensions), 180–181
Minimal Deadline First, 50
Minimal Processing Time First, 50
mirror, 242
Miss, 167
MMU (Memory-Management-Unit), 131
mount, 225, 228
MS-DOS, 9, 179, 182
MS-DOS/Windows, 9

multi-level queue, 62
Multi-level scheduling, 46
Multi-level-Tabellen, 134
Multi-Master-Systembus, 23
multi-tasking, 28, 375
multi-user, 6, 375
multiple Kerne, 23
Multiprogramming, 6, 28
Multiprozessoranomalien, 58
Multiprozessorsystem, 23
Multipurpose Internet Mail Extensions (MIME), 180–181
mutual exclusion, 69
mutually commensurable, 55

N

Nachrichtenpakete, 269
name server, 277
named pipe, 273, 375
Namensraum, 183
NAS, 14
NBT, 273
Nebenfenster, 375
NetBIOS, 273
Network Access Layer, 271
Network File System Siehe NFS, 279
Netzwerkcontroller, 273, 276
Netzwerkschicht, 270
NextFit, 125, 127
NFS (Network File System), 279
NLS (Native Language Support), 358
non-preemptive scheduling, 41
Nutzungsgradmodell, 157

O

Object-oriented Database Management Systems, 176
operating system, 4
orthogonale Befehle, 353

P

page base table, 132
page fault, 133–134, 140, 144, 146
page frame, 133, 136, 140–141, 166
Page Frame Database, 141
page frame reclaiming algorithm, 159
page out pool, 159
page table, 141
page-fault-frequency-Modell, 157

page-out- pool, 158
pages, 132
paging, 237, 258
paging demon, 159
Parameterschätzungen, 44
Paravirtualisierung, 16, 18, 20
parent window, 376
parent-widget, 373
Paritätsbildung, 243
Passwort, 325–327, 341, 345–346
Persistent Storage Manager, 176
Persistenz, 176
Pfadname, 182
physical address extension, 171
physikalische Ebene, 271
physikalische Maschine, 12
physikalischer Speicher, 131
pipeline cache, 167
Pixelebene, 361, 367, 370–371, 461
Plattenspeicher, 231–232, 234
Plug-and-Play-Protokoll, 231
Polled Loop, 49
polymorphe Viren, 329
Portierbarkeit, 7, 9
Portierung, 7
POSIX, 7, 358
power failure, 255, 257
Präsentationsebene, 270
Präzedenzgraph, 57
Präzedenzpartition, 55
Präzedenzrelation, 53
preemptive scheduling, 44
Prefetch, 239
prepaging, 154
print server, 25, 266, 268
Priorität, 43, 45–46, 49–50, 62–65, 73, 393
Prioritätsinversion, 50
Priority Scheduling, 45
process server, 268
Protokoll, 269–270, 272–274, 276
Prototype Page Table, 140
Prozeßgruppe, 241
Prozess, 28
Prozessgruppe, 58
Prozesskontext, 30
Prozesskontrollblock, 32, 37, 47, 123
Prozessoraffinität, 64
Prozessorzuteilung, adaptive, 44
Prozessscheduling, 39, 67
Pufferung, 257

R

race condition, 68
RAID, 228, 240, 242, 244–245
RAID-linear, 241
RAM scabbing, 52
RAM-Disk, 237
Rastergrafik, 352, 360, 371
Rate Monotonic-Scheduling, 50
raw device, 258
raw interface, 226
read ahead, 152
ready, 29–30
real mode, 173
real time systems, 47
Rechnernetz, 24
Redirector, 273
reference string, 144
Referenzfolge, 144, 154
Referenzwahrscheinlichkeit, 465–466
Registration Editor, 181
relativer Pfadname, 183
relozieren, 129
resource construction toolkit, 366
response time, 41
RGB-System, 361
Robustheit, 9
Rolle, 338
root, 185
root mode, 20
root window, 371, 373
Rootkit, 332
Rot-Grün-Blau-System, 361
rotational delay, 233
Round Robin, 45, 65
router, 343–344
Ruckeleffekt, 379
running, 29–30, 154

S

SAN, 14
SAN-in-a-box, 14
Scheduler, 30
Scheduling, 32, 40–41, 43–45, 47, 49, 53, 55,
59–60, 63–65, 67, 144, 392, 396
Schedulingstrategie, 41
Schichtenmodell, 11
Schlüssel, 178
Schnittstelle, 12, 15, 222, 229, 239, 257
Schreibtischmetapher, 352
screening, 343

Sealed Storage, 171
second chance, 146
section object, 140, 258
segmentation violation, 170
Segmente, 164–166
Segmentregister, 164
Segmenttabelle, 164–165
Seitenaustausch, 155, 159, 465
Seitenaustauschgrad, 465, 468
Seitenrahmen, 133, 162–163, 166
Seitentabelle, 132–134, 136, 140, 142, 146, 151, 164
Sektor, 232, 234, 257
Semaphore, 71–74
Server, 9, 25
Serverkonsolidierung, 16
Servospuren, 232
shared memory, 137, 140, 159, 166, 169
shared pages, 159
shell, 6, 29, 32, 341
shell-widget, 373
Shortest Job First, 42–43, 45, 50
shutdown, 258
Sicherheitsprotokoll, 326
Signal, 30, 71
Simulation, 62
Single Sign on Protocol, 346
Sitzungsebene, 270
SJF (Shortest Job First), 42–43, 45
SLED (Single Large Expensive Disk), 240
SMB (Server Message Block-Protokoll), 273
snooter, 168
Software-Hardware-Migration, 255
Softwareinterrupts, 5
Solid State Disk, 237
speedup, 58
Speicherangebot, 154, 156, 466–469
Speicherbelegungstabelle, 123
Speicherkapazität, 233, 242
Speicherschutz, 130
Spiegelplatten, 242, 245
spin lock, 73, 141, 255
Spur, 231–234, 236, 251, 256–257
SSD (Solid State Disk), 237
Stack-Algorithmen, 154
Stapelverarbeitung, 28
starvation, 42
statische Prioritäten, 43
statisches Scheduling, 39

stopped, 30
Storage Attached Network, 14
stream system, 226
Streifen, 241–242
stripes, 241, 245
SuperFetch, 239
swapping, 47, 63, 122, 129, 156, 160–161
symbolic link, 183
symmetrischer Pool, 15
symmetrisches Multiprocessing, 60
synchroner Verbindungsaufbau, 239, 259
system call, 5
Systemaufruf, 5
SystemFS, 225

T

task, 28
TCP (Transmission Control Protocol), 270, 272–274, 276
TCP/IP, 274
terminal node, 55
thrashing, 154, 156, 161, 163, 426, 468
thread, 33, 36, 64, 259–260
throughput, 40
Time Slice Scheduling, 51
toolkit, 366, 371–372
Top level-Domäne, 275
top-level window, 373
Transaction Grant Server, 347
transfer time, 233
translation lookaside buffer, 137
Transmission Control Protocol, 270, 272–274, 276
Transportschicht, 270, 273
trap-function, 5, 133, 345
Treiber, 4, 222–223, 226, 228, 230, 237, 239–240, 250–251, 253–258
Treiberschnittstelle, 250, 253
TRIM, 238
Trojanisches Pferd, 326
trust relationship, 343
turnaround time, 40
twisted pair, 271

U

UDI (Uniform Driver Interface), 223
UDP (User Datagram Protocol), 49, 270
UIMS (User Interface Management System), 365–367

Unicode, 182, 357–358
Uniform Driver Interface, 223
Uniform Resource Locator, 276
UNIX-98, 7, 37
User Datagram Protocol, 49, 270
user mode, 6, 172
user structure, 32

V

VDM (virtuelle DOS-Maschine), 9
verbindungslose Kommunikation, 270
verbindungsorientierte Kommunikation, 265, 269
Verhungern, 42
Verklemmung, 62, 407, 415
Verschnitt, 125, 128, 151–152
verteiltes Betriebssystem, 267
verteiltes Computersystem, 266
VFS (virtuelles Dateisystem), 224
Viren, 328–329, 331
virtual machine monitor, 17
virtual memory-Manager, 140
virtual mode, 173
Virtual Private Network, 274
virtual server, 17
Virtualisierung mit BS-Containern, 19
virtuelle DOS-Maschine, 9
virtuelle Festplatte, 13
virtuelle Maschine, 10–12, 15, 26, 222, 269
virtuelle Realität, 365
virtueller Prozessor, 30
virtueller Speicher, 130–131, 164, 166
Visualisierung, 352
vollständige Virtualisierung, 19–20
volume, 228, 241
VPN (Virtual Private Network), 274

W

WAN, 277
Warteschlange, 29–30, 39–47, 53, 60–64, 68, 72
Warteschlangentheorie, 60
Wartezeit, 41
watch dog timer, 52
WDM, 252
wear level, 238
Weyland, 374
wide area network, 277
widget, 371–373

Win-32-Treibermodell, 252
Win32, 9
window manager, 363, 371
Windows 2000, 10, 252, 277
Windows 98, 252
Windows NT Executive, 9
working set, 150, 154, 156–157, 161, 258, 466, 468–469, 472
working-set Modell, 156
WorstFit, 125, 127
write back, 168
write through, 168–169
WYSIWYG, 353

X

X-Window, 369–370, 372–373, 375–376, 379, 462
X.25, 270
X/Open, 7
Xlib, 369–372
XML (extended markup language), 368
XON/XOFF, 240
Xtoolkit, 371

Z

Zeigeinstrument, 352, 360
Zeitscheibe, 44–45, 64, 68
Zeitscheibenverfahren, 45, 63–64
Zugriffsrechte, 138, 140, 170, 172, 326–327, 337–338, 341–344
Zugriffsschnelligkeit, 242
Zustand, 29
Zylindergruppe, 232, 236